A FORMALLY VERIFIED PROOF OF CRAMÉR'S THEOREM ON LARGE DEVIATIONS

Kaan Berk Erdoğmuş

A THESIS

in

Mathematics

Presented to the Faculties of the University of Pennsylvania in Partial Fulfillment of the Requirements for the Degree of Master of Arts

2026

Supervisor of Thesis

Henry Towsner

Professor of Mathematics & Undergraduate Chair

Graduate Group Chair

Angela Gibney

Presidential Professor of Mathematics

Acknowledgements

I would like to thank my thesis advisor, Henry Towsner, for his guidance, time, and support throughout.

I would also like to thank Robin Pemantle for his assistance; Philip Gressman and Jiaqi Liu for teaching me analysis and probability theory; and my master's advisor Tony Pantev for his mentorship.

Finally, I thank my family for their unwavering support and encouragement.

Abstract

We present a formally verified measure-theoretic proof of Cramér's theorem in the theory of Large Deviations, implemented in the Lean 4 proof assistant and building on top of the Mathlib mathematical library.

Cramér's theorem states that the empirical mean of independent, identically distributed random variables with finite moment generating function satisfies a large deviation principle: that is, for $a \geq \mathbb{E}[X_1]$, the tail probability $\mathbb{P}(\overline{X}_n \geq a)$ decays exponentially with rate $I(a)$, the Legendre transform of the cumulant generating function. To the best of our knowledge, this is the first formalization of any large deviation result in a proof assistant.

We introduce the Large Deviation Principle as a formal Lean proposition, consisting of the two sides of the bound. The upper bound follows from the Chernoff bound — an exponential application of Markov's inequality optimized over a parameter — applied to the sample mean. The lower bound is substantially more complex, involving a change of measure to an exponentially tilted measure $\mathbb{Q}_{n,t}$ that transforms the rare tail event $\overline{X}_n \geq a$ into a typical event that preserves mass at the limit, demonstrated through a Central Limit Theorem argument for the standardized empirical mean under the sequence of tilted measures.

The formal proof consists of approximately 2,003 lines of Lean 4 code across six files, extending Mathlib's existing infrastructure for measure theory, moment and cumulant generating functions, tilted measures, the extended reals, characteristic functions, and the strong law of large numbers. This work demonstrates the viability of formalizing advanced probability theory in a modern proof assistant and associated library, and identifies gaps in current proof assistants' formal math libraries.

Table of Contents

1 Introduction

Large Deviations Theory is the study of the asymptotic decay of the probabilities of tail events. A foundational result of the theory, and the primary focus of this thesis, is Cramér's theorem [1], which characterizes the asymptotic decay rate of the tail probability $\mathbb{P}(\overline{X}_n \geq a)$, where $\overline{X}_n$ is the empirical mean of independent, identically distributed random variables and $a \geq \mathbb{E}[X_1]$. In a sense, Cramér's theorem is a complement to the Central Limit Theorem (CLT): while the CLT characterizes the behavior of the mean around the center of the distribution, Cramér's theorem characterizes the tail probability, which asymptotically decays to 0 at a rate $I(a)$ given by the Legendre transform of X_1's cumulant generating function.

For more comprehensive treatments of Large Deviations Theory, we refer the reader to Dembo and Zeitouni [2], Varadhan [3], Ellis [4], and den Hollander [5]. For connections to concentration inequalities and high-dimensional probability, see Vershynin [6].

While standard mathematical proofs of Cramér's theorem are well established (see, e.g., [2, 5]), they often involve delicate arguments – reasoning about decay over different probability spaces, interchanging limits and integrals, edge-case events of 0 probability – that are glossed over. In contrast, other parts of measure and probability theory have already been formalized in various proof assistants (discussed further in Chapter 2), including formalizations of foundational results like the Central Limit Theorem. Such formalizations enforce complete rigor by necessity: for a proof to be formally verified, it must explicitly enumerate all its axioms and assumptions, state the theorem precisely and completely, and provide a type-theoretic proof that gives a derivation of the theorem which can be fully machine-checked, leaving no room for subtle errors or arguments lacking sufficient rigor.

This thesis fills this gap by presenting a formally verified proof of Cramér's theorem, implemented using the Lean 4 theorem prover [7] and building upon the Mathlib mathematical library [8].[1]

The main contributions are:

1. A formal definition of the Large Deviation Principle (LDP) in Lean, separating the upper and lower bounds as asymptotic limits over the extended real numbers,

2. A proof of the upper bound through the Chernoff bound, an exponential application of Markov's inequality followed by optimizing over the moment generating function,

3. A proof of the lower bound through a change of measure to exponentially tilted measures, which includes a proof of the Central Limit Theorem for the empirical mean under the tilted measures, where the newly constructed probability measures transform the rare tail event into a typical one, and

4. The final result combining the above to state a Large Deviation Principle for the mean $\overline{X}_n$.

The formalization consists of approximately 2,003 lines of Lean 4 code across six files: `Defs.lean` (the LDP structure), `Basic.lean` (definitions and foundational lemmas), `UpperBound.lean` and `LowerBound.lean` (the two bounds, both building upon `Basic`), `TiltedCLT.lean` (the Central Limit Theorem under the tilted measure, used by the lower bound), and `Theorem.lean` (combining the preceding results into an LDP statement of Cramér's theorem). The proof of the lower bound is significantly longer, with `LowerBound.lean` and `TiltedCLT.lean` together accounting for about 70% of the code, reflecting the more involved change-of-measure approach compared to the Chernoff bound.

The remainder of this thesis is organized as follows: Chapter 2 is a brief discussion of prior work on the formalization of probability theory and related areas. Chapter 3 introduces the mathematical background for the proof, including the moment and cumulant generating functions, the Legendre transform, and the large deviation principle. Chapter 4 provides a general description of mathematical formalization in Lean 4 and lists the Lean definitions used by the rest of the work, including the formal definition of the Large Deviation Principle. Chapter 5 states Cramér's theorem formally in Lean, building on the aforementioned definitions. Chapters 6–9 explain the proof: the basic properties, the upper bound, the CLT over tilted measures, the lower bound, and the main result that combines both bounds. For each chapter here, we first

[1] This work builds upon Mathlib as of commit `1708ec1fd9`.

present a mathematical overview of the proof, then list the associated Lean lemmas and theorems proven with the Lean proofs omitted. Chapter 10 concludes with reflections and future directions. Appendix A is the complete listing of the Lean code, including the proofs omitted in the preceding chapters. Appendix B is a listing of existing definitions and results in Mathlib that were used in the proof.

2 Related Work

Formalizing probability theory in a proof assistant is a substantial undertaking: the theory builds upon several areas, including measure theory, real analysis, and topology, which must be formalized before work on probability may begin. The proof assistant ecosystem has seen several languages and associated formal math libraries that have been developed sufficiently to express proofs in measure-theoretic probability theory. In this chapter, we survey them before describing our contribution.

Isabelle/HOL. Hölzl and Heller [9] built foundational measure-theoretic infrastructure in Isabelle/HOL, extending prior work that formalized the Lebesgue measure on the reals to include the extended real numbers, Borel σ-algebras for arbitrary topological spaces, the Radon-Nikodým Theorem, and Fubini's Theorem. Building on this foundation, Avigad, Hölzl, and Serafin [10] produced a formally verified proof of the Central Limit Theorem, using characteristic functions and Lévy's Continuity Theorem in a proof that follows the textbook by Billingsley [11].

Hirata [12] formalized the Lévy-Prokhorov metric, which is a metric defined on the measures on a metric space, and proved Prokhorov's theorem in Isabelle/HOL. Prokhorov's theorem provides conditions for the relative compactness of sets of finite measures and is relevant to Sanov's theorem [2] from information theory, an LDP result for empirical measures.

Rocq (formerly Coq). Affeldt and Cohen [13] provided an original formalization of measure and integration theory, building on the MathComp-Analysis library, where they constructed a Lebesgue measure as an extension from a semiring of sets — a standard approach that had not previously been formalized in a proof assistant with dependent type theory foundations.

More recently, Affeldt et al. [14] formalized concentration inequalities in Rocq, which are inequalities of the form $\mathbb{P}(X \geq a) \leq b$ such as the Markov or Chebyshev bounds. Of particular relevance, they provided a proof of Chernoff's inequality, which, as discussed, is used in the proof of the upper bound of Cramér's theorem.

Lean 4 and Mathlib. The Lean 4 proof assistant [7] and its surrounding ecosystem have seen rapid advancement in formalized mathematics in the past several years, centered primarily around the Mathlib library [8]. In stochastic processes, Ying and Degenne [15] formalized Doob's Martingale Convergence Theorems, starting from foundations in conditional expectation on Banach spaces, stopping times, and martingales, and building up to Lévy's generalized Borel-Cantelli lemma. Degenne [16] further described Mathlib's usage of Markov kernels and the disintegration theorem, which are used for the formal definitions of conditional probability and posterior distributions, as well as independence, conditional independence, and entropy.

Sonoda et al. [17] built on Mathlib's foundation of measure-theoretic probability to formalize generalization error bounds by Rademacher complexity, and formalized Dudley's entropy integral as a worked example.

Mathlib also contains a proof of the strong law of large numbers, which we use in our proof for the case $a < \mathbb{E}[X_1]$ in order to expand the statement of the Large Deviation Principle to all $a \in \mathbb{R}$. Mathlib's strong law, named `strong_law_ae_real`, is based on Etemadi's proof [18], which only requires pairwise independence. We also build upon the existing formalization of Chernoff's inequality and exponentially tilted measures present in Mathlib.

Lastly, Mathlib contains a very recently merged proof of the Central Limit Theorem [19], which follows the approach of using characteristic functions and Lévy's Continuity Theorem — the same approach used by Avigad, Hölzl, and Serafin [10] in their Isabelle/HOL proof.

Our contribution. To the best of our knowledge, no results in Large Deviations Theory — whether Cramér's theorem, Sanov's theorem, or the Gärtner-Ellis theorem — have been proven in any proof assistant. Nor has the concept of a Large Deviation Principle been formalized in any of them. Closest to this thesis are the aforementioned formalizations of Chernoff bounds (which we build upon for the upper bound) and the formalizations of the Central Limit Theorem.

Our contribution introduces the Large Deviation Principle as a formal structure in Lean 4, builds upon the Chernoff bound and tilted measures for the upper and lower bounds, and combines these into a proof of Cramér's theorem. As part of the lower bound, we also build upon Mathlib's results on weak convergence in terms of characteristic functions to prove a CLT statement for the empirical mean under the sequence of tilted measures, following a similar approach to the recently merged proof of the standard CLT.

3 Preliminaries

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. Let $X_1, X_2, \ldots$ be a sequence of independent, identically distributed real-valued random variables.

Definition 3.1 (Partial Sums and Empirical Mean)**.** *We define the partial sum and the empirical mean as:*

$$S_n = \sum_{i=1}^{n} X_i, \qquad \overline{X}_n = \frac{S_n}{n} \tag{1}$$

Definition 3.2 (MGF, CGF, and Rate Function)**.** *Let $M_X(t)$ and $\Lambda(t)$ denote the Moment Generating Function (MGF) and Cumulant Generating Function (CGF) of X_1 respectively:*

$$M_X(t) = \mathbb{E}[e^{tX_1}] \qquad \Lambda(t) = \log M_X(t) = \log \mathbb{E}[e^{tX_1}] \tag{2}$$

The rate function $I(x)$ is defined as the Legendre transform of the convex function Λ:

$$I(x) = \sup_{t \in \mathbb{R}} \{tx - \Lambda(t)\} \tag{3}$$

As $t = 0$ yields $0 \cdot x - \Lambda(0) = 0$, the rate function satisfies $I(x) \geq 0$ for all $x \in \mathbb{R}$. In our formalization, we will take the assumption `h_bdd` (see chapter 4) that, $\forall x \in \mathbb{R}$, the set $\{tx - \Lambda(t) \mid t \in \mathbb{R}\}$ is bounded above, which implies that the rate function is always finite.

Definition 3.3 (Large Deviation Principle)**.** *We say that a sequence of random variables Z_n taking values in a space Ξ satisfies a **Large Deviation Principle** (LDP) if there exists a rate function $I : \Xi \to \mathbb{R}$ such that for all $A \subseteq \Xi$, the scaled log-tail probability is bounded by $-I$ between the closure and the interior of A:*

$$\limsup_{n \to \infty} \frac{1}{n} \log \mathbb{P}(Z_n \in A) \leq - \inf_{x \in \overline{A}} I(x) \tag{4}$$

$$\liminf_{n \to \infty} \frac{1}{n} \log \mathbb{P}(Z_n \in A) \geq - \inf_{x \in \mathring{A}} I(x) \tag{5}$$

For the purposes of this formal proof, we will restrict ourselves to investigating upper tail sets $[a, \infty)$ for some $a \in \mathbb{R}$, which aligns with the statement of Cramér's theorem. Unlike the full LDP involving arbitrary sets with separate closure and interior conditions, for upper tail sets $[a, \infty)$, both conditions reduce to the same rate function $I(a)$, so that the upper and lower bounds have the same asymptotic logarithmic bound:

$$\limsup_{n \to \infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -I(a) \tag{6}$$

$$\liminf_{n \to \infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -I(a) \tag{7}$$

When both inequalities hold, we have $\lim_{n \to \infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) = -I(a)$: the probability decays according to the rate function, and this rate is asymptotically tight.

Note that Cramér's theorem makes a statement for $a \geq \mathbb{E}[X_1]$, and if $a < \mathbb{E}[X_1]$, the event $\{\overline{X}_n \geq a\}$ is not a rare event. However, we can still define an appropriate rate function: by the strong law of large numbers, $\overline{X}_n \to \mathbb{E}[X_1]$ a.s., so $\mathbb{P}(\overline{X}_n \geq a) \to 1$ whenever $a < \mathbb{E}[X_1]$. Therefore, it suffices to define $I(a) = 0$ in this case and the LDP bounds hold trivially.

Definition 3.4 (Tilted Measure)**.** *For a fixed tilting parameter $t \in \mathbb{R}$, we define the sequence of tilted probability measures $\mathbb{Q}_{n,t}$ on $(\Omega, \mathcal{F})$ with respect to $\mathbb{P}$ through the Radon-Nikodym derivative:*

$$\frac{d\mathbb{Q}_{n,t}}{d\mathbb{P}} = \frac{e^{tS_n}}{\mathbb{E}_{\mathbb{P}}[e^{tS_n}]} = \frac{e^{tS_n}}{M_{S_n}(t)} = \frac{e^{tS_n}}{(M_{X_0}(t))^n} = \frac{e^{tS_n}}{e^{n\Lambda(t)}} = e^{tS_n - n\Lambda(t)} \tag{8}$$

4 Formalization Framework & Assumptions

Lean 4 is built on dependent type theory, a foundation different from set theory in ways that affect formalization and the structure of the proof. Crucially, mathematical objects belong to *types* rather than sets, and unlike sets, types do not fully subsume each other and can require explicit coercions to move values between them. A concrete example encountered in this thesis is that the real numbers $\mathbb{R}$ and the extended real numbers $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$ (denoted `EReal` in Lean) are different types: although `EReal` is defined using $\mathbb{R}$, a real number x is not implicitly an extended real number and can require explicit casts, denoted $\uparrow x$. This difference manifests itself as several helper lemmas throughout the formalization where, for example in Chapter 7.3, we will prove statements in $\mathbb{R}$ but then explicitly lift them to `EReal` for the LDP statement.

Like other mathematical formalizations in Lean, our formalization builds upon the Mathlib library [8], which provides the measure-theoretic probability foundations: σ-algebras, probability measures, integrability, moment and cumulant generating functions, and tilted measures are available and utilized.

Below we briefly describe the Lean 4 syntax conventions that appear throughout the code listings.

- `Type*` is a type of an implicit universe level, enabling definitions to be polymorphic over type universes.
- Curly braces in variable, lemma, and theorem statements $\{\Omega\}$ denote *implicit* arguments inferred automatically from typed explicit arguments. Parentheses (X) denote *explicit* arguments supplied by the caller. Square brackets [MeasureSpace Ω] denote *type class instances*.
- The `noncomputable` keyword marks a function as not being compilable to executable code. These definitions typically involve non-constructive axioms like classical choice or the law of excluded middle. Although Lean cannot generate executable code for them, they are still part of the type system and can be used for specification and formalization, and the Lean compiler will fully type-check them for correctness, so that the logical consistency of proofs is preserved.
- The notation $\forall^f n$ in atTop reads "for all sufficiently large n" and is a generic definition of limits using *Filters*. Mathlib uses Filters to abstract and unify the notion of limits of functions, sequences, points that approach infinitesimal or infinite values, as well as statements that are true for sufficiently large n, and so on. Instead of providing distinct definitions for each of these, they are unified in a single framework. `limsup` and `liminf` in particular, which are used in our statement of the LDP, are defined via filters rather than via explicit ε-N arguments.

The definitions and assumptions presented in the following chapters are listed in full in Appendix A.1.

4.1 Definitions

Definition 4.1 (Sample Space)**.** *We define the sample space Ω as a type equipped with the `MeasureSpace` type-class instance.*

```
variable {Ω : Type*} [MeasureSpace Ω]
```

`MeasureSpace` in turn is defined in Mathlib as a space with a σ-algebra equipped with a measure.

```
/-- A measure space is a measurable space equipped with a
  measure, referred to as `volume`. -/
class MeasureSpace (α : Type*) extends MeasurableSpace α where
  volume : Measure α
```

Listing 1: Mathlib/MeasureTheory/Measure/MeasureSpaceDef.lean

Definition 4.2 (Probability Measure)**.** *Mathlib defines the notation* $\mathbb{P}$ *as the* `volume` *of a measure space. We further constrain* $\mathbb{P}$ *to be a probability measure through the type-class* `IsProbabilityMeasure`*.*

```
scoped[ProbabilityTheory] notation "ℙ" => MeasureTheory.MeasureSpace.volume
```

Listing 2: Mathlib/Probability/Notation.lean

```
variable [IsProbabilityMeasure (ℙ : Measure Ω)]
```

where `IsProbabilityMeasure` is defined as

```
/-- A measure ‘μ‘ is called a probability measure if ‘μ univ = 1‘. -/
class IsProbabilityMeasure (μ : Measure α) : Prop where
  measure_univ : μ univ = 1
```

Listing 3: Mathlib/MeasureTheory/Measure/Typeclasses/Probability.lean

Definition 4.3 (Sequence of Random Variables)**.** *Next, we define our sequence of random variables* X_n *as a function from the naturals to real-valued random variables on* Ω*.*

```
variable (X : ℕ → Ω → ℝ)
```

In Lean, we will index starting at 0, as is common in programming contexts, but continue to index from 1 in the remainder of this text following standard mathematical convention.

Definition 4.4 (Partial Sums and Empirical Mean)**.** *We also define the partial sums* S_n *and the empirical mean* $\overline{X}_n$ *as mappings from the naturals to random variables on* Ω*.*

```
/-- The partial sum X₁ + ... + Xₙ. -/
def partialSum (n : ℕ) : Ω → ℝ := Σ i ∈ Finset.range n, X i

/-- The empirical mean Sₙ / n. -/
noncomputable def empiricalMean (n : ℕ) : Ω → ℝ := fun ω => partialSum X n ω / n
```

Listing 4: LargeDeviations/Cramers/Basic.lean

Definition 4.5 (Moment and Cumulant Generating Functions)**.** *The moment and cumulant generating functions are defined in Mathlib with their usual definitions as the expectation of* e^{tX} *and its* log *respectively:*

```
/-- Moment-generating function of a real random variable ‘X‘: ‘fun t => μ[exp(t*X)]‘. -/
def mgf (X : Ω → ℝ) (μ : Measure Ω) (t : ℝ) : ℝ :=
  μ[fun ω => exp (t * X ω)]

/-- Cumulant-generating function of a real random variable ‘X‘: ‘fun t => log μ[exp(t*X)]‘. -/
def cgf (X : Ω → ℝ) (μ : Measure Ω) (t : ℝ) : ℝ :=
  log (mgf X μ t)
```

Listing 5: Mathlib/Probability/Moments/Basic.lean

Note that $\mu[f]$ is notation for the expectation of f with respect to the measure μ.

Definition 4.6 (Rate Function)**.** *The rate function is defined as the Legendre transform of the cumulant generating function:*

```
noncomputable def I (x : ℝ) : ℝ := ⨆ t : ℝ, t * x - cgf (X 0) ℙ t
```

Listing 6: LargeDeviations/Cramers/Basic.lean

where ⨆ is notation for the indexed supremum `iSup`.

As we will be defining a general Large Deviation Principle for all a in the domain of the rate function, but are only interested in $a \geq \mathbb{E}[X_1]$, we define the *effective* rate function as:

```
noncomputable def upperTailRateFunction (X : ℕ → Ω → ℝ) (a : ℝ) : ℝ :=
  if 𝔼[X 0] ≤ a then I X a else 0
```

Listing 7: LargeDeviations/Cramers/Basic.lean

so that the LDP is trivially satisfied for $a < \mathbb{E}[X_0]$.

Definition 4.7 (Exponentially Tilted Measure)**.** *The sequence of exponentially tilted measures $\mathbb{Q}_{n,t}$ from **definition 3.4** is formalized using Mathlib's `Measure.tilted`, tilted by the random variable $\omega \mapsto t \cdot S_n(\omega)$:*

```
noncomputable def ℚ_nt (X : ℕ → Ω → ℝ) (μ : Measure Ω) (n : ℕ) (t : ℝ) : Measure Ω :=
  Measure.tilted μ (fun ω => t * partialSum X n ω)
```

Listing 8: LargeDeviations/Cramers/Basic.lean

4.2 The Large Deviation Principle

The Large Deviation Principle describes the asymptotic exponential decay of the probability of tail events. In Lean, we formalize this principle as a structure `LargeDeviationPrinciple` over a sequence of real-valued random variables Y_n and a rate function $I : \mathbb{R} \to \mathbb{R}$.

Definition 4.8 (Large Deviation Principle)**.** *A sequence of random variables (Y_n) satisfies a Large Deviation Principle with rate function I if the scaled log-tail probability $\frac{1}{n} \log \mathbb{P}(Y_n \geq a)$ is asymptotically bounded above and below by $-I(a)$ for all $a \in \mathbb{R}$. The principle is split into an upper bound and a lower bound to facilitate independent proofs for each direction.*

```
structure LargeDeviationPrinciple (Y : ℕ → Ω → ℝ) (I : ℝ → ℝ) : Prop where
  /-- Upper bound: limsup_n (1/n) log ℙ(Y_n ≥ a) ≤ -I(a) -/
  upperBound : ∀ a : ℝ,
    limsup (fun n : ℕ => ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | Y n ω ≥ a}))
      atTop ≤ (- I a : EReal)
  /-- Lower bound: liminf_n (1/n) log ℙ(Y_n ≥ a) ≥ -I(a) -/
  lowerBound : ∀ a : ℝ,
    (- I a : EReal) ≤
      liminf (fun n : ℕ => ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | Y n ω ≥ a}))
        atTop
```

Listing 9: Mathlib/Probability/LargeDeviations/Defs.lean

While the general LDP gives bounds for all a, Cramér's theorem specifically provides a tight asymptotic bound when $a \geq \mathbb{E}[X_1]$. We've defined the modified rate function `upperTailRateFunction` in order to make a general LDP statement for all $a \in \mathbb{R}$.

4.3 Assumptions

We explicitly assume the following properties, corresponding to the `variable` blocks in the code. These are the only assumptions we make for the formal proof beyond the initial setup.

- **Independence:** The sequence X_i consists of mutually independent random variables.

```
variable (h_indep : iIndepFun X ℙ)
```

- **Identically Distributed:** The random variables X_i are identically distributed.

```
variable (h_ident : ∀ n, IdentDistrib (X n) (X 0) ℙ ℙ)
```

- **Measurability:** The random variables X_i are measurable.

```
variable (h_meas : ∀ n, Measurable (X n))
```

- **Integrability:** The random variables are integrable, meaning $\mathbb{E}[|X_i|] < \infty$. This is implied by the finiteness of the moment generating function, but we assume it explicitly for convenience in formalization.

```
variable (h_int : Integrable (X 0) ℙ)
```

- **Finite Moment Generating Function:** For all $t \in \mathbb{R}$, the exponential moment is finite, i.e., $M_X(t) := \mathbb{E}[\exp(tX_i)] < \infty$.

```
variable (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * X 0 ω)) ℙ)
```

- **Bounded Rate Function:** For any $a \in \mathbb{R}$, the set $\{ta - \Lambda(t) \mid t \in \mathbb{R}\}$ over which we take the supremum is bounded above. This implies that the rate function $I(a)$ exists and is finite for all $a \in \mathbb{R}$. Like integrability, this is also implied by the finiteness of the moment generating function.

```
variable (h_bdd : ∀ a : ℝ, BddAbove (Set.range (fun t => t * a - cgf (X 0) ℙ t)))
```

- **Non-Degeneracy:** The CGF has a strictly positive second derivative everywhere: $\forall t \in \mathbb{R}, \Lambda''(t) > 0$. When the cumulant generating function is finite, it is analytic and convex, and is strictly convex if and only if the distribution is non-degenerate [2]. This excludes degenerate (point mass) distributions, for which $\Lambda(t) = tc$ and $\Lambda''(t) = 0$. Large Deviations are trivial for such distributions since for all n, $\overline{X}_n = c$ a.s.

```
variable (h_non_deg : ∀ t : ℝ, 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)
```

- **Exposed Points:** All points $a \geq \mathbb{E}[X_1]$ are exposed. That is, they are in the range of the derivative of the CGF so that $\forall a \geq \mathbb{E}[X_1], \exists t \in \mathbb{R}$ such that $\Lambda'(t) = a$.

```
variable (h_exposed : ∀ a : ℝ, 𝔼[X 0] ≤ a → ∃ t, deriv (cgf (X 0) ℙ) t = a)
```

5 Formal Theorem Statement

With the setup complete, we can now state the main theorem formally.

Theorem 5.1 (Cramér's Theorem)**.** *The sequence of empirical means $\overline{X}_n$ satisfies a Large Deviation Principle with rate function* `upperTailRateFunction`*. That is, for $a \geq \mathbb{E}[X_1]$, the sequence satisfies an LDP with rate function $I(x)$, the Legendre transform of Λ; and for $a < \mathbb{E}[X_1]$, it satisfies an LDP with rate function 0.*

```
include h_indep h_meas h_ident h_mgf h_int h_bdd h_non_deg h_exposed in
/-- **Cramér's Theorem**: For i.i.d. random variables with finite MGF, the empirical mean
satisfies a Large Deviation Principle with rate function being the Legendre transform of the CGF.
    -/
theorem cramers_theorem :
    LargeDeviationPrinciple (empiricalMean X) (upperTailRateFunction X)
```

Listing 10: LargeDeviations/Cramers/Theorem.lean

6 Basic Properties

Having stated the theorem formally, we now develop the building blocks needed for its proof. We prove the following helper lemmas prior to delving into separate proofs for the upper and lower bounds. These lemmas are used in the formal proofs of subsequent statements and are listed in full in Appendix A.2.

Lemma 6.1 (Finite MGF of Identically Distributed Variables)**.** *The moment generating function $M_{X_i}(t) = \mathbb{E}[e^{tX_i}]$ is finite for all i and any $t \in \mathbb{R}$.*

```
lemma integrable_exp_of_identDistrib (i : ℕ) (t : ℝ) :
    Integrable (fun ω => Real.exp (t * X i ω)) ℙ
```

Listing 11: LargeDeviations/Cramers/Basic.lean

Lemma 6.2 (Measurability of Partial Sum)**.** *The partial sum S_n is a measurable function.*

```
lemma measurable_partialSum (n : ℕ) : Measurable (partialSum X n)
```

Lemma 6.3 (Measurability of Empirical Mean)**.** *The empirical mean $\overline{X}_n$ is a measurable function.*

```
lemma measurable_empiricalMean (n : ℕ) : Measurable (empiricalMean X n)
```

Lemma 6.4 (Interior of Integrable Exponential Set)**.** *Any $t \in \mathbb{R}$ lies in the interior of the set of parameters for which the exponential e^{tX_1} is integrable. That is, $\forall t \in \mathbb{R}, t \in \text{int}\{t \mid |\mathbb{E}[e^{tX_1}]| < \infty\}$.*

```
lemma mem_interior_integrableExpSet (t : ℝ) : t ∈ interior (integrableExpSet (X 0) ℙ)
```

The following lemmas require the measure $\mathbb{P}$ to be a probability measure. That is, they additionally require that $\mathbb{P}(\Omega) = 1$.

Lemma 6.5 (CGF Lower Bound)**.** *For any integrable (measurable with finite expectation) random variable Y with a finite MGF, we have $\Lambda_Y(t) \geq t\mathbb{E}[Y]$.*

```
lemma cgf_ge_mul_expect (Y : Ω → ℝ) (h_int : Integrable Y ℙ)
    (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * Y ω)) ℙ) (t : ℝ) :
    cgf Y ℙ t ≥ t * 𝔼[Y]
```

Lemma 6.6 (Rate Function is Negative for Negative Parameter)**.** *For an integrable random variable Y with finite MGF, if $t < 0$ and $a \geq \mathbb{E}[Y]$, the body of the supremum of the rate function, $ta - \Lambda_Y(t)$, is non-positive.*

```
lemma rate_function_neg_param_le_zero (Y : Ω → ℝ) (h_int : Integrable Y ℙ)
    (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * Y ω)) ℙ)
    (t a : ℝ) (ht : t < 0) (ha : 𝔼[Y] ≤ a) :
    t * a - cgf Y ℙ t ≤ 0
```

Lemma 6.7 (Finite MGF of Partial Sum)**.** *If each random variable X_i has a finite MGF, then the partial sum S_n also has a finite MGF for all $t \in \mathbb{R}$.*

```
lemma integrable_exp_sum (t : ℝ) (n : ℕ) :
    Integrable (fun ω => Real.exp (t * partialSum X n ω)) ℙ
```

Lemma 6.8 (Tilted Measure is a Probability Measure)**.** *$\forall t \in \mathbb{R}$, the tilted measure $\mathbb{Q}_{n,t}$ obtained from tilting by the factor $t \cdot S_n$ is a valid probability measure.*

```
lemma isProbabilityMeasure_tilted_partialSum (t : ℝ) (n : ℕ) :
    IsProbabilityMeasure (ℚ_nt X ℙ n t)
```

Lemma 6.9 (Interior of Integrable Exponential Set of Partial Sums)**.** *Any $t \in \mathbb{R}$ lies in the interior of the set of parameters for which the exponential e^{tS_n} is integrable for all $n \in \mathbb{N}$. That is, $\forall n \in \mathbb{N}, \forall t \in \mathbb{R}, t \in \text{int}\{t \mid |\mathbb{E}[e^{tS_n}]| < \infty\}$.*

```
lemma mem_interior_integrableExpSet_partialSum (t : ℝ) (n : ℕ) :
    t ∈ interior (integrableExpSet (partialSum X n) ℙ)
```

Lemma 6.10 (Rate Function Achieves Supremum at Non-Negative Parameter)**.** *If $a \geq \mathbb{E}[X_1]$, rate function $I(a) = \sup_{t \in \mathbb{R}}\{ta - \Lambda_{X_1}(t)\}$ has its supremum achieved by $t \geq 0$. That is, $I(a) = \sup_{t \geq 0}\{ta - \Lambda_{X_1}(t)\}$.*

```
lemma rateFunction_eq_sup_nonneg (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) :
    I X a = ⨆ t : {(x : ℝ) | 0 ≤ x}, (t : ℝ) * a - cgf (X 0) ℙ t
```

Lemma 6.11 (MGF of Partial Sum is a Power of the Single MGF)**.** *The MGF of the partial sum S_n, $M_{S_n}(t)$, is the product of individual MGFs, $M_{X_i}(t)$. In particular, as X_i are i.i.d., $M_{S_n}(t) = M_{X_1}(t)^n$.*

For convenience with future formalization, we state this in terms of the cumulant generating function of X_1: $M_{S_n}(t) = e^{n \cdot \Lambda_{X_1}(t)}$.

```
lemma mgf_sum_eq_exp_n_prod_cgf (n : ℕ) (t : ℝ) :
    mgf (partialSum X n) ℙ t = Real.exp (n * cgf (X 0) ℙ t)
```

Lemma 6.12 (CGF of Partial Sum is n Times the Single CGF)**.** *The cumulant generating function of the partial sum S_n is n times the cumulant generating function of X_1: $\Lambda_{S_n}(t) = n\Lambda_{X_1}(t)$.*

```
lemma cgf_sum_eq_n_prod_cgf (n : ℕ) (t : ℝ) :
    cgf (partialSum X n) ℙ t = (n : ℝ) * cgf (X 0) ℙ t
```

7 The Upper Bound (Chernoff Bound)

With the foundational definitions and basic properties established, we can now prove the two bounds of Cramér's theorem. We begin with the upper bound.

7.1 Mathematical Description

We first introduce the approach to the upper bound for Cramér's theorem in higher-level mathematical notation before delving into the formalized statements.

The upper bound relies on an exponential application of Markov's inequality, commonly known as the Chernoff bound (see, e.g., [2, 6]). We show that the tail probability $\mathbb{P}(\overline{X}_n \geq a)$ is bounded via the MGF, which reduces to an optimization problem. Then, given $a \geq \mathbb{E}[X_1]$, optimizing over non-negative parameters $t \geq 0$ suffices to recover the rate function $I(a)$.

Theorem 7.1 (Cramér Upper Bound)**.** *For any $a \geq \mathbb{E}[X_1]$:*

$$\limsup_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -I(a) \tag{9}$$

Proof. Since $x \mapsto e^{tx}$ is monotonically increasing, applying Markov's inequality to the non-negative random variable e^{tS_n} gives

$$\mathbb{P}(\overline{X}_n \geq a) = \mathbb{P}(S_n \geq na) = \mathbb{P}(e^{tS_n} \geq e^{tna}) \leq e^{-tna}\mathbb{E}[e^{tS_n}] \tag{10}$$

By independence, the expectation factors as a product:

$$\mathbb{E}[e^{tS_n}] = \mathbb{E}\left[\prod_{i=1}^{n} e^{tX_i}\right] = \prod_{i=1}^{n} \mathbb{E}[e^{tX_i}] = \left(\mathbb{E}[e^{tX_1}]\right)^n = e^{n\Lambda(t)} \tag{11}$$

Substituting,

$$\mathbb{P}(\overline{X}_n \geq a) \leq e^{-tna} e^{n\Lambda(t)} = e^{-n(ta-\Lambda(t))} \tag{12}$$

Taking logarithms and dividing by n:

$$\frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -(ta - \Lambda(t)) \tag{13}$$

As this bound is independent of n, taking the lim sup preserves it:

$$\limsup_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -(ta - \Lambda(t)) \tag{14}$$

Since the bound $-(ta-\Lambda(t))$ holds for all $t \geq 0$, taking the infimum over all t yields the tightest possible bound. Then, since $\inf_{t\geq 0}\{-(ta - \Lambda(t))\} = -\sup_{t\geq 0}\{ta - \Lambda(t)\}$:

$$\limsup_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -\sup_{t\geq 0}\{ta - \Lambda(t)\} \tag{15}$$

It remains to show that $\sup_{t\geq 0}\{ta-\Lambda(t)\} = I(a) = \sup_{t\in\mathbb{R}}\{ta-\Lambda(t)\}$. That is, the supremum is achieved by $t \geq 0$.

Define $g(t) = ta - \Lambda(t)$. Note that $g(0) = 0 - \Lambda(0) = -\log \mathbb{E}[e^0] = 0$ and, as $a \geq \mathbb{E}[X_1]$ by assumption,

$$g'(0) = a - \Lambda'(0) = a - \mathbb{E}[X_1] \geq 0 \tag{16}$$

As Λ is convex, g is concave. For a concave function with $g'(0) \geq 0$, we have $g(t) \leq g(0) = 0$ for all $t < 0$. Therefore, the supremum cannot be achieved by any negative t, and we conclude

$$\sup_{t\geq 0}\{ta - \Lambda(t)\} = \sup_{t\in\mathbb{R}}\{ta - \Lambda(t)\} = I(a) \tag{17}$$

□

7.2 A Worked Example: The Exponential Distribution

We illustrate a concrete example of the upper bound using the exponential distribution, working through the derivation of the Chernoff bound.

Let $X_1, X_2, \ldots$ be i.i.d. with $X_i \sim \text{Exp}(\lambda)$ for rate parameter $\lambda > 0$, so that the density is $f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$ and $f(x) = 0$ for $x < 0$. We have that $\mathbb{E}[X_1] = \frac{1}{\lambda}$.

7.2.1 Cumulant Generating Function

For $t < \lambda$, the MGF is finite:

$$\mathbb{E}[e^{tX_1}] = \int_0^\infty e^{tx} \lambda e^{-\lambda x}\, dx = \lambda \int_0^\infty e^{-(\lambda - t)x}\, dx = \frac{\lambda}{\lambda - t} \tag{18}$$

For $t \geq \lambda$, the integral diverges. Therefore the CGF is

$$\Lambda(t) = \begin{cases} \log \lambda - \log(\lambda - t) & \text{if } t < \lambda \\ \infty & \text{if } t \geq \lambda \end{cases} \tag{19}$$

Note that unlike the assumptions we've made for the formal proof in chapter 4.3, the exponential distribution's MGF is not finite everywhere.

7.2.2 The Rate Function

We compute $I(a) = \sup_{t \in \mathbb{R}} \{ta - \Lambda(t)\}$ for $a \geq \mathbb{E}[X_1] = \frac{1}{\lambda}$.

As $\Lambda(t) = \infty$ for $t \geq \lambda$, we search for the supremum for $t \in (-\infty, \lambda)$ where Λ is smooth and strictly convex. Letting $g(t) = ta - \log \lambda + \log(\lambda - t)$:

$$g'(t) = 0 \implies a - \frac{1}{\lambda - t} = 0 \implies t^\star = \lambda - \frac{1}{a} \tag{20}$$

We verify that $t^\star$ is in the domain $[0, \lambda)$: $a \geq \frac{1}{\lambda} \implies \frac{1}{a} \leq \lambda$, so $t^\star \geq 0$. Likewise, $a > 0 \implies \frac{1}{a} > 0$, so $t^\star < \lambda$.

Then, evaluating the rate function at $t^\star$:

$$I(a) = t^\star a - \Lambda(t^\star) = \left(\lambda - \frac{1}{a}\right) a - \log \lambda + \log\left(\frac{1}{a}\right) = \lambda a - 1 - \log(\lambda a) \tag{21}$$

7.2.3 Applying the Cramér Upper Bound

Theorem 7.2 (Exponential Upper Bound)**.** *Let $X_1, X_2, \cdots \sim Exp(\lambda)$ be i.i.d. For any $a \geq \frac{1}{\lambda}$:*

$$\limsup_{n \to \infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -(\lambda a - 1 - \log(\lambda a)) \tag{22}$$

At $a = \mathbb{E}[X_1] = \frac{1}{\lambda}$, the rate function evaluates to $I(\frac{1}{\lambda}) = 1 - 1 - \log(1) = 0$, which is consistent with the fact that the mean is not a rare event. As a increases, $I(a)$ becomes positive, demonstrating the exponential decay of the tail.

7.3 Formalized Results

We present the formalized steps for the upper bound of Cramér's theorem, giving a brief mathematical description of each statement, and omitting the proofs, which are listed in full in Appendix A.3.

In order to state a more general formal result that can account for outcomes of probability measure 0, we state the bounds in the extended real numbers $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$. In Mathlib, this is represented as creating a new type that wraps the reals twice; it is either the bottom value $\bot$, the top value $\top$, or a value from $\mathbb{R}$:

```
/-- The type of extended real numbers `[-∞, ∞]`, constructed as `WithBot (WithTop ℝ)`. -/
def EReal := WithBot (WithTop ℝ)
deriving Nontrivial, Zero, One, AddMonoid, AddCommMonoid, AddCommMonoidWithOne, CharZero, ...
```

Listing 12: Mathlib/Data/EReal/Basic.lean

```
/-- Attach `⊥` to a type. -/
@[to_dual /-- Attach `⊤` to a type. -/]
def WithBot (α : Type*) := Option α
```

Listing 13: Mathlib/Order/TypeTags.lean

Using the extended reals `EReal` enables us to use the function `ENNReal.log` which is the natural logarithm that maps $[0, \infty] \to [-\infty, \infty]$ and avoid `Real.log` which is defined so that $\log(0) = 0$ and $\log(-x) = \log(x)$. Therefore, we are able to correctly express the log-scaled probability $\frac{1}{n} \log \mathbb{P}(\{\omega \mid \overline{X}_n(\omega) \geq a\})$.

At a high level, the formalization follows the mathematical proof but requires some technical lemmas to map expressions on $\mathbb{R}$ to expressions on the extended reals $\overline{\mathbb{R}}$. Unlike set-theoretic foundations, where we'd consider $\mathbb{R}$ as a simple subset of $\overline{\mathbb{R}}$, these are fundamentally distinct types in Lean's type-theoretic foundations.

Lemma 7.3 (Infimum Coercion to Extended Reals)**.** *Let $S \subseteq \mathbb{R}$ be a nonempty set bounded below. Let $S_\infty = \{x_\infty \mid x \in S\}$ where x_∞ is x coerced to the extended reals $\overline{\mathbb{R}}$, corresponding to the Lean coercion from `Real` to `EReal`. Then $\inf S_\infty = (\inf S)_\infty$.*

```
private lemma ereal_sInf_coe_eq_coe_sInf {s : Set ℝ} (hne : s.Nonempty) (hbdd : BddBelow s) :
    sInf ((fun (x : ℝ) => (x : EReal)) '' s) = ↑(sInf s)
```

Listing 14: LargeDeviations/Cramers/UpperBound.lean

Lemma 7.4 (Infimum of Negations in Extended Reals)**.** *Let $S \subseteq \mathbb{R}$ be a nonempty bounded-above set. Then,*

$$\inf\{-x_\infty \mid x \in S\} = -(\sup S)_\infty \tag{23}$$

In the formalization, S is expressed as the range of an indexed function $f : \iota \to \mathbb{R}$ over a type ι.

```
private lemma ereal_sInf_neg_eq_neg_sSup {ι : Type*} (f : ι → ℝ)
    (hne : (Set.range f).Nonempty) (hbdd : BddAbove (Set.range f)) :
    sInf (Set.range fun i => (-(f i) : EReal)) = -((sSup (Set.range f) : ℝ) : EReal)
```

Lemma 7.5 (Chernoff Bound for Empirical Mean)**.** *For $t \geq 0$,*

$$\mathbb{P}(\overline{X}_n \geq a) \leq e^{-n(ta - \Lambda_{X_0}(t))} \tag{24}$$

```
lemma prob_mean_ge_le_exp (t a : ℝ) (ht : 0 ≤ t) (n : ℕ) (hn_pos : 0 < n) :
  (ℙ {ω | empiricalMean X n ω ≥ a}).toReal
    ≤ Real.exp ( - (n : ℝ) * (t * a - cgf (X 0) ℙ t))
```

Listing 15: LargeDeviations/Cramers/UpperBound.lean

Theorem 7.6 (Cramér's Theorem Upper Bound)**.** *For any* $a \geq \mathbb{E}[X_1]$,

$$\limsup_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -I(a) \tag{25}$$

```
theorem cramer_upper_bound (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) :
    limsup (fun n : ℕ =>
      ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a}))
        atTop ≤ (- I X a : EReal)
```

Listing 16: LargeDeviations/Cramers/UpperBound.lean

8 The Lower Bound (Exponential Tilting)

The upper bound was obtained through a direct application of Markov's inequality. The lower bound is substantially more involved: rather than bounding a probability from above, we must show that a rare event of decaying probability preserves enough mass asymptotically. We approach this with a change of measure: we transform our probability to an *exponentially tilted* measure that turns the rare tail event of decaying probability into a typical one that asymptotically preserves its mass [2].

The tilted measure $\mathbb{Q}_{n,t}$ is defined via the Radon-Nikodym derivative $\frac{\mathrm{d}\mathbb{Q}_{n,t}}{\mathrm{d}\mathbb{P}} = e^{tS_n - n\Lambda(t)}$, which is strictly positive everywhere (as $M_X(t) < \infty$ by assumption `h_mgf`). Therefore $\mathbb{Q}_{n,t}$ and $\mathbb{P}$ are mutually absolutely continuous, and the change of measure $\mathrm{d}\mathbb{P} = e^{-n(t\overline{X}_n - \Lambda(t))}\,\mathrm{d}\mathbb{Q}_{n,t}$ used in the lower bound proof is valid per the Radon-Nikodym theorem.

8.1 Motivation

Rearranging the Radon-Nikodym derivative of the tilted measure from **definition 3.4**, we get

$$\frac{\mathrm{d}\mathbb{Q}_{n,t}}{\mathrm{d}\mathbb{P}} = e^{tS_n - n\Lambda(t)} = e^{tn\overline{X}_n - n\Lambda(t)} \implies \mathrm{d}\mathbb{P} = e^{-n(t\overline{X}_n - \Lambda(t))}\mathrm{d}\mathbb{Q}_{n,t} \tag{26}$$

which matches almost exactly the upper Chernoff bound $\mathbb{P}(\overline{X}_n \geq a) \leq e^{-n(ta-\Lambda(t))}$, except that $\overline{X}_n$ is replaced by a.

Intuitively, if we could demonstrate that $\overline{X}_n$ heavily centers around a under $\mathbb{Q}_{n,t}$ as $n \to \infty$, and that the tilted measure of the overall event $\mathbb{Q}_{n,t}(\overline{X}_n \geq a)$ approaches some constant $c \in (0,1)$, the exponential term would dominate and we could approximate, for large n,

$$\frac{1}{n}\log \mathbb{P}(\overline{X}_n \geq a) = \frac{1}{n}\log \int_{\overline{X}_n \geq a} e^{-n(t\overline{X}_n - \Lambda(t))}\mathrm{d}\mathbb{Q}_{n,t} \approx \frac{1}{n}\log\left(e^{-n(ta-\Lambda(t))}\cdot c\right) \to -(ta - \Lambda(t)) \tag{27}$$

yielding the same bound as the Chernoff upper bound.

In fact, computing the expectation directly, for $1 \leq i \leq n$ we get

$$\begin{aligned}
\mathbb{E}_{\mathbb{Q}_{n,t}}[X_i] &= \int_\Omega X_i\,\mathrm{d}\mathbb{Q}_{n,t} = \int_\Omega X_i e^{tS_n - n\Lambda(t)}\,\mathrm{d}\mathbb{P} \\
&= \frac{1}{M_X(t)^n}\mathbb{E}_{\mathbb{P}}[X_i e^{tS_n}] = \frac{1}{M_X(t)^n}\mathbb{E}_{\mathbb{P}}\left[X_i e^{tX_i}\prod_{\substack{j=1\\ j\neq i}}^{n} e^{tX_j}\right] \\
&= \frac{1}{M_X(t)^n}\mathbb{E}_{\mathbb{P}}[X_i e^{tX_i}]\prod_{\substack{j=1\\ j\neq i}}^{n}\mathbb{E}_{\mathbb{P}}[e^{tX_j}] = \frac{1}{M_X(t)^n}\mathbb{E}_{\mathbb{P}}[X_i e^{tX_i}]M_X(t)^{n-1} \\
&= \frac{\mathbb{E}_{\mathbb{P}}[X_i e^{tX_i}]}{M_X(t)} = \frac{M_X'(t)}{M_X(t)} = \frac{d}{dt}\log M_X(t) = \Lambda'(t)
\end{aligned} \tag{28}$$

Then, by linearity of expectation, $\mathbb{E}_{\mathbb{Q}_{n,t}}[S_n] = \sum_{i=1}^n \mathbb{E}_{\mathbb{Q}_{n,t}}[X_i] = n\Lambda'(t)$ and $\mathbb{E}_{\mathbb{Q}_{n,t}}[\overline{X}_n] = \frac{1}{n}\mathbb{E}_{\mathbb{Q}_{n,t}}[S_n] = \Lambda'(t)$.

By assumption `h_exposed`, we can pick a t such that $\Lambda'(t) = a$, yielding $\mathbb{E}_{\mathbb{Q}_{n,t}}[\overline{X}_n] = \Lambda'(t) = a$. Then, effectively, the tilted measures all re-center the distributions of $\overline{X}_n$ so that the mean is always at a, and so the event $\overline{X}_n \geq a$ is no longer a rare tail event with decaying probability as n grows.

Thus, if we can make a Central Limit Theorem statement for the sample mean $\overline{X}_n$ under $\mathbb{Q}_{n,t}$, we could build toward justifying lower-bounding the integrand as $e^{-n(ta-\Lambda(t))}$.

To further motivate the Central Limit Theorem, we can also demonstrate that the variance of the mean is finite and decays $\Theta(n^{-1})$. First, the variance of a single X_i under a tilted measure is given by

$$\begin{aligned}
\mathbb{V}_{\mathbb{Q}_{n,t}}(X_i) &= \mathbb{E}_{\mathbb{Q}_{n,t}}[X_i^2] - \left(\mathbb{E}_{\mathbb{Q}_{n,t}}[X_i]\right)^2 \\
&= \int_\Omega X_i^2 e^{tS_n - n\Lambda(t)} \, \mathrm{d}\mathbb{P} - \left(\Lambda'(t)\right)^2 \\
&= \frac{1}{M_X(t)^n} \mathbb{E}_{\mathbb{P}} \left[X_i^2 e^{tX_i} \prod_{\substack{j=1 \\ j \neq i}}^{n} e^{tX_j} \right] - \left(\frac{M_X'(t)}{M_X(t)} \right)^2 \\
&= \frac{\mathbb{E}_{\mathbb{P}}[X_i^2 e^{tX_i}] M_X(t)^{n-1}}{M_X(t)^n} - \left(\frac{M_X'(t)}{M_X(t)} \right)^2 \\
&= \frac{M_X''(t)}{M_X(t)} - \left(\frac{M_X'(t)}{M_X(t)} \right)^2 = \frac{d}{dt} \left(\frac{M_X'(t)}{M_X(t)} \right) = \Lambda''(t).
\end{aligned} \tag{29}$$

Because we have not yet proven that the X_i remain independent under $\mathbb{Q}_{n,t}$, to determine the variance of the partial sum S_n we instead compute directly:

$$\begin{aligned}
\mathbb{V}_{\mathbb{Q}_{n,t}}(S_n) &= \mathbb{E}_{\mathbb{Q}_{n,t}}[S_n^2] - \left(\mathbb{E}_{\mathbb{Q}_{n,t}}[S_n]\right)^2 \\
&= \int_\Omega S_n^2 \frac{e^{tS_n}}{M_{S_n}(t)} \, \mathrm{d}\mathbb{P} - \left(\Lambda_{S_n}'(t)\right)^2 \\
&= \frac{1}{M_{S_n}(t)} \mathbb{E}_{\mathbb{P}}[S_n^2 e^{tS_n}] - \left(\frac{M_{S_n}'(t)}{M_{S_n}(t)} \right)^2 \\
&= \frac{M_{S_n}''(t)}{M_{S_n}(t)} - \left(\frac{M_{S_n}'(t)}{M_{S_n}(t)} \right)^2 = \frac{d}{dt} \left(\frac{M_{S_n}'(t)}{M_{S_n}(t)} \right) \\
&= \frac{d^2}{dt^2} \Lambda_{S_n}(t) = \frac{d^2}{dt^2} \left(n\Lambda(t) \right) \\
&= n\Lambda''(t)
\end{aligned} \tag{30}$$

Therefore, the variance of the empirical mean is $\mathbb{V}_{\mathbb{Q}_{n,t}}(\overline{X}_n) = \mathbb{V}_{\mathbb{Q}_{n,t}}\left(\frac{S_n}{n}\right) = \frac{1}{n^2}\mathbb{V}_{\mathbb{Q}_{n,t}}(S_n) = \frac{\Lambda''(t)}{n}$.

8.2 Central Limit Theorem on $\mathbb{Q}_{n,t}$

Before delving into the proof for the lower bound, we'll first demonstrate that the partial sums S_n, under their respective measures $\mathbb{Q}_{n,t}$, indeed satisfy a statement of the CLT.

Define

$$Y_i := \frac{X_i - \mathbb{E}_{\mathbb{Q}_{n,t}}[X_i]}{\sqrt{\mathbb{V}_{\mathbb{Q}_{n,t}}(X_i)}} = \frac{X_i - \Lambda'(t)}{\sqrt{\Lambda''(t)}} \tag{31}$$

and let

$$Z_n := \frac{S_n - \mathbb{E}_{\mathbb{Q}_{n,t}}[S_n]}{\sqrt{\mathbb{V}_{\mathbb{Q}_{n,t}}(S_n)}} = \frac{S_n - n\Lambda'(t)}{\sqrt{n\Lambda''(t)}} = \sum_{i=1}^{n} \frac{X_i - \Lambda'(t)}{\sqrt{n\Lambda''(t)}} = \frac{1}{\sqrt{n}} \sum_{i=1}^{n} Y_i \tag{32}$$

Note that the Y_i are independent of each other under $\mathbb{P}$ and that each Y_i is independent of X_j for $j \neq i$.

Theorem 8.1 (CLT under Tilted Measures)**.** *The sequence of random variables Z_n under $\mathbb{Q}_{n,t}$ satisfies a Central Limit Theorem. That is, the distributions of Z_n under $\mathbb{Q}_{n,t}$ converge to $\mathcal{N}(0,1)$ as $n \to \infty$.*

Proof. First, for $1 \leq i \leq n$, the characteristic function of Y_i under $\mathbb{Q}_{n,t}$ is given by

$$\begin{aligned}
\varphi_{Y_i,\mathbb{Q}_{n,t}}(\theta) = \mathbb{E}_{\mathbb{Q}_{n,t}}\left[e^{i\theta Y_i}\right] &= \int_\Omega e^{i\theta Y_i}\,\mathrm{d}\mathbb{Q}_{n,t} = \int_\Omega e^{i\theta Y_i} e^{tS_n - n\Lambda(t)}\,\mathrm{d}\mathbb{P} \\
&= e^{-n\Lambda(t)} \int_\Omega e^{i\theta Y_i} e^{tS_n}\,\mathrm{d}\mathbb{P} = e^{-n\Lambda(t)} \int_\Omega e^{i\theta Y_i + tX_i} e^{t(S_n - X_i)}\,\mathrm{d}\mathbb{P} \\
&= e^{-n\Lambda(t)} \mathbb{E}_{\mathbb{P}}[e^{i\theta Y_i + tX_i}]\ \mathbb{E}_{\mathbb{P}}[e^{t(S_n - X_i)}] \\
&= e^{-n\Lambda(t)} \mathbb{E}_{\mathbb{P}}[e^{i\theta Y_i + tX_i}]\ \mathbb{E}_{\mathbb{P}}\left[\prod_{\substack{j=1 \\ j\neq i}}^{n} e^{tX_j}\right] \\
&= e^{-n\Lambda(t)} \mathbb{E}_{\mathbb{P}}[e^{i\theta Y_i + tX_i}]\ \mathbb{E}_{\mathbb{P}}[e^{tX_0}]^{n-1} \\
&= e^{-n\Lambda(t)} \mathbb{E}_{\mathbb{P}}[e^{i\theta Y_i + tX_i}]\ e^{(n-1)\Lambda(t)} \\
&= \boxed{e^{-\Lambda(t)} \mathbb{E}_{\mathbb{P}}[e^{i\theta Y_i + tX_i}]}
\end{aligned} \tag{33}$$

The characteristic function of Z_n under $\mathbb{Q}_{n,t}$ is then:

$$\begin{aligned}
\varphi_{Z_n,\mathbb{Q}_{n,t}}(\theta) = \mathbb{E}_{\mathbb{Q}_{n,t}}\left[e^{i\theta Z_n}\right] &= \int_\Omega e^{i\theta Z_n}\,\mathrm{d}\mathbb{Q}_{n,t} = \int_\Omega e^{i\theta Z_n} e^{tS_n - n\Lambda(t)}\,\mathrm{d}\mathbb{P} \\
&= e^{-n\Lambda(t)} \int_\Omega e^{i\theta \sum_{i=1}^n \frac{Y_i}{\sqrt{n}}} e^{t\sum_{i=1}^n X_i}\,\mathrm{d}\mathbb{P} \\
&= e^{-n\Lambda(t)} \int_\Omega \prod_{j=1}^n e^{\frac{\theta}{\sqrt{n}} iY_j + tX_j}\,\mathrm{d}\mathbb{P} \\
&= e^{-n\Lambda(t)} \mathbb{E}_{\mathbb{P}}\left[\prod_{j=1}^n e^{\frac{\theta}{\sqrt{n}} iY_j + tX_j}\right] \\
&= e^{-n\Lambda(t)} \prod_{j=1}^n \mathbb{E}_{\mathbb{P}}\left[e^{\frac{\theta}{\sqrt{n}} iY_j + tX_j}\right] \\
&= e^{-n\Lambda(t)} \mathbb{E}_{\mathbb{P}}\left[e^{\frac{\theta}{\sqrt{n}} iY_1 + tX_1}\right]^n \\
&= \left(e^{-\Lambda(t)} \mathbb{E}_{\mathbb{P}}\left[e^{\left(i\frac{\theta}{\sqrt{n}}\right) Y_1 + tX_1}\right]\right)^n \\
&= \boxed{\left(\varphi_{Y_1,\mathbb{Q}_{n,t}}\left(\frac{\theta}{\sqrt{n}}\right)\right)^n}
\end{aligned} \tag{34}$$

The remainder of the proof follows the same format as the proof of the classical CLT.
By Taylor's theorem,

$$\begin{aligned}
\varphi_{Y_1,\mathbb{Q}_{n,t}}\left(\frac{\theta}{\sqrt{n}}\right) &= \mathbb{E}_{\mathbb{Q}_{n,t}}\left[1 + \frac{i\theta Y_1}{\sqrt{n}} - \frac{\theta^2 Y_1^2}{2n} + \sum_{j=3}^{\infty} \frac{(i\theta Y_1)^j}{j!\, n^{j/2}}\right] \\
&= 1 + \frac{i\theta\, \mathbb{E}_{\mathbb{Q}_{n,t}}[Y_1]}{\sqrt{n}} - \frac{\theta^2\, \mathbb{E}_{\mathbb{Q}_{n,t}}[Y_1^2]}{2n} + \mathbb{E}_{\mathbb{Q}_{n,t}}\left[\sum_{j=3}^{\infty} \frac{(i\theta Y_1)^j}{j!\, n^{j/2}}\right] \\
&= 1 + \frac{i\theta \cdot 0}{\sqrt{n}} - \frac{\theta^2 \cdot 1}{2n} + o\left(\frac{\theta^2}{n}\right) \\
&= 1 - \frac{\theta^2}{2n} + o\left(\frac{\theta^2}{n}\right)
\end{aligned} \tag{35}$$

and so

$$\varphi_{Z_n,\mathbb{Q}_{n,t}}(\theta) = \left(\varphi_{Y_1,\mathbb{Q}_{n,t}}\left(\frac{\theta}{\sqrt{n}}\right)\right)^n = \left(1 - \frac{\theta^2}{2n} + o\left(\frac{\theta^2}{n}\right)\right)^n \to \boxed{e^{-\theta^2/2}} \qquad \text{as } n \to \infty \tag{36}$$

Lévy's Continuity Theorem states that if a sequence of random variables Z_n, not necessarily on the same probability space, has characteristic functions $\varphi_n(\theta)$ that converge pointwise to a function $\varphi(\theta)$ that is itself the characteristic function of some random variable Z, then $Z_n \xrightarrow{\mathcal{D}} Z$ [11].

Therefore, Z_n converges to a random variable Z with characteristic function $e^{-\theta^2/2}$. As this is the characteristic function of the standard normal, we have $Z \sim \mathcal{N}(0,1)$. □

Corollary 8.1 (eventually_$\mathbb{Q}_{n,t}$_empiricalMean_mem_Icc_ge)**.** *For any target $a \in \mathbb{R}$ and tilting parameter $t \in \mathbb{R}$ such that $\Lambda'(t) = a$, and for any interval width $\delta > 0$ and $\varepsilon > 0$, there exists an integer N such that for all $n \geq N$:*

$$\mathbb{Q}_{n,t}\left(\overline{X}_n \in [a, a+\delta]\right) \geq \frac{1}{2} - \varepsilon \tag{37}$$

Proof. From the definition of Z_n, we can write:

$$Z_n = \frac{S_n - n\Lambda'(t)}{\sqrt{n\Lambda''(t)}} = \frac{n\overline{X}_n - n\Lambda'(t)}{\sqrt{n\Lambda''(t)}} = \frac{\sqrt{n}(\overline{X}_n - \Lambda'(t))}{\sqrt{\Lambda''(t)}} \tag{38}$$

As $\Lambda'(t) = a$, the event $\overline{X}_n \in [a, a+\delta]$ is equivalent to:

$$a \leq \overline{X}_n \leq a+\delta \iff 0 \leq \overline{X}_n - \Lambda'(t) \leq \delta \iff 0 \leq \frac{\sqrt{n}(\overline{X}_n - \Lambda'(t))}{\sqrt{\Lambda''(t)}} \leq \frac{\delta\sqrt{n}}{\sqrt{\Lambda''(t)}} \iff 0 \leq Z_n \leq \delta\sqrt{n/\Lambda''(t)} \tag{39}$$

By theorem 8.1, we have that $Z_n \xrightarrow{\mathcal{D}} \mathcal{N}(0,1)$. Therefore, for any sequence $b_n \to \infty$, we have:

$$\lim_{n\to\infty} \mathbb{Q}_{n,t}(0 \leq Z_n \leq b_n) = \frac{1}{2} \tag{40}$$

Because $\Lambda''(t) > 0$ and $\delta > 0$, the upper bound $b_n = \delta\sqrt{n/\Lambda''(t)} \to \infty$ as $n \to \infty$. So, the probability of the event converges to $1/2$:

$$\lim_{n\to\infty} \mathbb{Q}_{n,t}\left(0 \leq Z_n \leq \delta\sqrt{n/\Lambda''(t)}\right) = \lim_{n\to\infty} \mathbb{Q}_{n,t}\left(\overline{X}_n \in [a, a+\delta]\right) = \frac{1}{2} \tag{41}$$

Therefore, by definition of the limit, $\forall \varepsilon > 0, \exists N$ such that $\forall n \geq N$:

$$\mathbb{Q}_{n,t}\left(\overline{X}_n \in [a, a+\delta]\right) \geq \frac{1}{2} - \varepsilon \tag{42}$$

□

8.3 Lower Bound

Theorem 8.2 (Cramér's Theorem Lower Bound)**.** *For any $a \geq \mathbb{E}[X_1]$, the sequence of empirical means satisfies the large deviation lower bound:*

$$\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -I(a) \tag{43}$$

Proof. As shown above, we have that

$$\mathbb{E}_{\mathbb{Q}_{n,t}}[\overline{X}_n] = \Lambda'(t) = a \tag{44}$$

$$\mathbb{V}_{\mathbb{Q}_{n,t}}(\overline{X}_n) = \frac{1}{n}\Lambda''(t) \tag{45}$$

To bound $\mathbb{P}(\overline{X}_n \geq a)$ from below, we restrict the event to a smaller range $[a, a+\delta]$ for some arbitrary $\delta > 0$. Let $E_n = \{\overline{X}_n \in [a, a+\delta]\}$. Clearly, $\mathbb{P}(\overline{X}_n \geq a) \geq \mathbb{P}(E_n)$. Then, by the definition of the Radon-Nikodym derivative $\frac{d\mathbb{Q}_{n,t}}{d\mathbb{P}}$,

$$\mathbb{P}(\overline{X}_n \geq a) \geq \mathbb{P}(E_n) = \int_{E_n} e^{-tS_n + n\Lambda(t)}\, d\mathbb{Q}_{n,t} \tag{46}$$

For $\omega \in E_n$, we have $\overline{X}_n(\omega) \leq a+\delta \implies S_n \leq n(a+\delta)$. By **Lemma 6.10**, the optimal t for which the rate function achieves its supremum is non-negative. As such, we are interested in $t \geq 0$, and multiplying by $-t$ reverses the inequality, yielding $-tS_n \geq -tn(a+\delta)$. Substituting:

$$\begin{aligned} \mathbb{P}(E_n) &= \int_{E_n} e^{-tS_n + n\Lambda(t)}\, d\mathbb{Q}_{n,t} \\ &\geq \int_{E_n} e^{-tn(a+\delta) + n\Lambda(t)}\, d\mathbb{Q}_{n,t} \\ &= e^{-n(t(a+\delta) - \Lambda(t))} \mathbb{Q}_{n,t}(E_n) \end{aligned} \tag{47}$$

Taking the logarithm,

$$\frac{1}{n}\log \mathbb{P}(\overline{X}_n \geq a) \geq \frac{1}{n}\log \mathbb{P}(E_n) \geq -(ta - \Lambda(t)) - t\delta + \frac{1}{n}\log \mathbb{Q}_{n,t}(E_n) \tag{48}$$

so it suffices to bound $\mathbb{Q}_{n,t}(E_n)$ from below.

By **Corollary 8.1**, we can pick any $\varepsilon > 0$ such that $\exists N \in \mathbb{N}$ for which $n \geq N$ satisfies

$$\mathbb{Q}_{n,t}(E_n) = \mathbb{Q}_{n,t}\left(\overline{X}_n \in [a, a+\delta]\right) \geq \frac{1}{2} - \varepsilon \tag{49}$$

Because $\mathbb{Q}_{n,t}(E_n) \geq \frac{1}{2} - \varepsilon > 0$ for sufficiently large n, the error term $\frac{1}{n}\log \mathbb{Q}_{n,t}(E_n)$ is bounded between $\frac{1}{n}\log\left(\frac{1}{2} - \varepsilon\right)$ and 0 (since $\mathbb{Q}_{n,t}(E_n) \leq 1$). The key insight is that $\frac{1}{n}\log c \to 0$ for any constant $c > 0$: the CLT guarantees that $\mathbb{Q}_{n,t}(E_n)$ is bounded away from zero, so its logarithm is a constant that vanishes by the $1/n$ scaling.

Taking the limit inferior of both sides, we then obtain:

$$\liminf_{n\to\infty} \frac{1}{n}\log \mathbb{P}(\overline{X}_n \geq a) \geq -(ta - \Lambda(t)) - t\delta \tag{50}$$

As the choice of $\delta > 0$ was arbitrary, we can take the limit as $\delta \downarrow 0$:

$$\liminf_{n\to\infty} \frac{1}{n}\log \mathbb{P}(\overline{X}_n \geq a) \geq -(ta - \Lambda(t)) \tag{51}$$

Finally, recall the definition of the rate function $I(a) = \sup_{s\in\mathbb{R}}(sa - \Lambda(s))$. By assumption `h_exposed`, there exists $s = t$ with $\Lambda'(t) = a$, so $g(s) := sa - \Lambda(s)$ has a critical point at $s = t$: $g'(t) = a - \Lambda'(t) = 0$. As

$\Lambda''(s) > 0$ everywhere by assumption h_non_deg, Λ is strictly convex on $\mathbb{R}$, hence g is strictly concave. Any critical point of a concave function is a global maximum, and strict concavity guarantees uniqueness, so the supremum is attained at $s = t$: $I(a) = ta - \Lambda(t)$. Substituting gives the desired bound:

$$\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -I(a) \tag{52}$$

□

8.4 Worked Example: Exponential Distribution (Lower Bound)

We return to the exponential distribution from Chapter 7.2 to illustrate the lower bound on a concrete example. Recall that $X_i \sim \text{Exp}(\lambda)$ with $\mathbb{E}[X_1] = 1/\lambda$, $\Lambda(t) = \log\lambda - \log(\lambda - t)$ for $t < \lambda$, and the optimal tilting parameter is $t^\star = \lambda - \frac{1}{a}$ for $a \geq 1/\lambda$.

The tilted distribution. Under the tilted measure $\mathbb{Q}_{n,t^\star}$, each X_i has density proportional to

$$e^{t^\star x} \cdot \lambda e^{-\lambda x} = \lambda\, e^{-(\lambda - t^\star)x} = \lambda\, e^{-x/a} \tag{53}$$

Normalizing yields the density $\frac{1}{a}e^{-\frac{x}{a}}$, which has distribution $\text{Exp}(1/a)$, the exponential distribution of mean a. Perhaps unsurprisingly, the exponential distribution is *closed* under exponential tilting — the tilted exponential distribution is still exponential.

Shifted mean and variance. Under $\mathbb{Q}_{n,t^\star}$, the mean of each X_i is $\mathbb{E}_{\mathbb{Q}_{n,t^\star}}[X_i] = a$. Thus, the tilted measure has transformed the rare event $\{\overline{X}_n \geq a\}$ under $\mathbb{P}$ to a typical one, centered at the mean.

The variance is $\frac{1}{1/a^2} = a^2$, so the CLT under $\mathbb{Q}_{n,t^\star}$ gives $\sqrt{n}(\overline{X}_n - a) \xrightarrow{d} \mathcal{N}(0, a^2)$. In particular, $\mathbb{Q}_{n,t^\star}(\overline{X}_n \in [a, a+\delta]) \to \frac{1}{2}$ for any fixed $\delta > 0$.

The lower bound. Following the proof of the lower bound and applying the change-of-measure inequality, we obtain

$$\frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -\big(t^\star a - \Lambda(t^\star)\big) - t^\star\delta + \frac{1}{n} \log \mathbb{Q}_{n,t^\star}(E_n) \tag{54}$$

where $E_n = \{\overline{X}_n \in [a, a+\delta]\}$. Substituting $t^\star = \lambda - 1/a$ and $\Lambda(t^\star) = \log(\lambda a)$:

$$t^\star a - \Lambda(t^\star) = (\lambda - 1/a) \cdot a - \log(\lambda a) = \lambda a - 1 - \log(\lambda a) = I(a) \tag{55}$$

As $n \to \infty$, the error term $(1/n)\log \mathbb{Q}_{n,t^\star}(E_n) \to 0$ by the CLT argument. Taking $\delta \downarrow 0$:

$$\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -I(a) = -(\lambda a - 1 - \log(\lambda a)) \tag{56}$$

Matching bounds. Combining with the upper bound from Theorem 7.2, the two inequalities are tight:

$$\lim_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) = -(\lambda a - 1 - \log(\lambda a)) = -I(a) \tag{57}$$

which confirms the Large Deviation Principle for the exponential distribution with rate function $I(a)$.

8.5 Formalized Results

As before, we list the formalized statements that make up the proof, giving a brief mathematical description of each statement and omitting the proofs, which are listed in full in Appendix A.4 and Appendix A.5.

The lower bound is substantially more involved than the upper bound, and requires a greater number of intermediate lemmas, so we separate the formalization across two files: `TiltedCLT.lean` formalizes a CLT statement for the sample mean under the sequence of exponentially tilted measures $\mathbb{Q}_{n,t}$ and establishes **Corollary 8.2**. `LowerBound.lean` then uses the corollary through a change-of-measure argument on the sample mean to derive the lower bound.

8.5.1 Central Limit Theorem on Tilted Measures

We first present the formal statements from `TiltedCLT.lean`. At a high level, the approach here is similar to the informal proof of Theorem 8.1: We introduce standardized random variables Y_i and partial sums Z_n, identify their characteristic functions, and derive a convergence result to the standard normal using Lévy's Continuity Theorem. However, the formal proof takes a slightly different approach of proving that X_i are independent and identically distributed under $\mathbb{Q}_{n,t}$, unlike Theorem 8.1, which derived the expressions for the characteristic functions directly.

Definitions.

Definition 8.3 (Tilted Marginal Law)**.** *The law of X_0 under $\mathbb{P}$, pushed forward and tilted by $t \cdot x$.*

```
noncomputable def μt (X : ℕ → Ω → ℝ) (t : ℝ) : Measure ℝ :=
  ((ℙ : Measure Ω).map (X 0)).tilted (fun x => t * x)
```

Listing 17: LargeDeviations/Cramers/TiltedCLT.lean

Definition 8.4 (Standardized Tilted Marginal Law)**.** *The pushforward of μ_t along the transformation $x \mapsto (x - \Lambda'(t))/\sqrt{\Lambda''(t)}$, i.e., μ_t centered at $\Lambda'(t)$ and rescaled by $\sqrt{\Lambda''(t)}$.*

```
noncomputable def νt (X : ℕ → Ω → ℝ) (t : ℝ) : Measure ℝ :=
  (μt X t).map
    (fun x => (x - deriv (cgf (X 0) ℙ) t) / Real.sqrt (iteratedDeriv 2 (cgf (X 0) ℙ) t))
```

Definition 8.5 (Standardized Partial Sum)**.** *The standardized partial sum $Z_n := (S_n - n\Lambda'(t))/\sqrt{n\Lambda''(t)}$, to which we will apply Lévy's Continuity Theorem.*

```
noncomputable def Z (X : ℕ → Ω → ℝ) (t : ℝ) (n : ℕ) : Ω → ℝ := fun ω =>
  (partialSum X n ω - n * deriv (cgf (X 0) ℙ) t)
    / Real.sqrt (n * iteratedDeriv 2 (cgf (X 0) ℙ) t)
```

Moments and integrability under μ_t.

Lemma 8.6 (MGF of the Identity under μ_t)**.** $M_{\mathrm{id},\mu_t}(u) = M_{X_0,\mathbb{P}}(t+u)/M_{X_0,\mathbb{P}}(t)$.

```
lemma mgf_id_tiltedLaw (t u : ℝ) :
    mgf id (μ_t X t) u = mgf (X 0) ℙ (t + u) / mgf (X 0) ℙ t
```

Lemma 8.7 (Interior of the Integrable Exponential Set under the Pushforward)**.** *Every $t \in \mathbb{R}$ lies in the interior of the integrable exponential set of the identity under the pushforward law $\mathbb{P} \circ X_0^{-1}$.*

```
private lemma t_mem_interior_integrableExpSet_id (t : ℝ) :
    t ∈ interior (integrableExpSet id ((ℙ : Measure Ω).map (X 0)))
```

Lemma 8.8 (CGF under the Pushforward Equals CGF under $\mathbb{P}$)**.** *The CGF of the identity under the pushforward $\mathbb{P} \circ X_0^{-1}$ equals the CGF of X_0 under $\mathbb{P}$.*

```
private lemma cgf_id_map_eq :
    cgf id ((ℙ : Measure Ω).map (X 0)) = cgf (X 0) ℙ
```

Lemma 8.9 (Multiplication with Identity)**.** *$x \mapsto t \cdot x$ is $t \cdot \mathrm{id}$ as a function $\mathbb{R} \to \mathbb{R}$.*

```
private lemma t_mul_eq_mul_id (t : ℝ) : (fun x : ℝ => t * x) = (t * id ·)
```

Lemma 8.10 (Expectation of the Identity under μ_t)**.** $\mathbb{E}_{\mu_t}[\mathrm{id}] = \Lambda'(t)$.

```
lemma integral_id_tiltedLaw (t : ℝ) :
    ∫ x, x ∂(μ_t X t) = deriv (cgf (X 0) ℙ) t
```

Lemma 8.11 (Variance of the Identity under μ_t)**.** $\mathbb{V}_{\mu_t}(\mathrm{id}) = \Lambda''(t)$.

```
lemma variance_id_tiltedLaw (t : ℝ) :
    Var[id; μ_t X t] = iteratedDeriv 2 (cgf (X 0) ℙ) t
```

Lemma 8.12 (L^2 Integrability of the Identity under μ_t)**.** $\mathrm{id} \in L^2(\mu_t)$.

```
lemma memLp_id_tiltedLaw (t : ℝ) : MemLp (id : ℝ → ℝ) 2 (μ_t X t)
```

Connecting the empirical mean and standardized variables.

Lemma 8.13 (Event Equivalence)**.** *For $n \geq 1$ and $\Lambda'(t) = a$, the event $\{\overline{X}_n \in [a, a+\delta]\}$ is exactly the event $\{Z_n \in [0, \delta\sqrt{n/\Lambda''(t)}]\}$.*

```
lemma empiricalMean_mem_Icc_iff_Z_mem_Icc (t a δ : ℝ) (n : ℕ) (hn : 0 < n)
    (h_non_deg_t : 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) (ω : Ω) :
    empiricalMean X n ω ∈ Set.Icc a (a + δ) ↔
      Z X t n ω ∈ Set.Icc (0 : ℝ)
        (δ * Real.sqrt (n / iteratedDeriv 2 (cgf (X 0) ℙ) t))
```

Lemma 8.14 (Second Moment of the Identity under ν_t)**.** $\int x^2 \, d\nu_t = 1$*: the standardized tilted marginal has unit second moment.*

```
lemma integral_sq_id_stdTiltedLaw (t : ℝ) :
    ∫ x, x ^ 2 ∂(ν_t X t) = 1
```

Probability-measure instances.

Lemma 8.15 (Tilted Marginal is a Probability Measure)**.** μ_t *is a probability measure.*

```
lemma isProbabilityMeasure_tiltedLaw (t : ℝ) :
    IsProbabilityMeasure (μ_t X t)
```

Lemma 8.16 (Standardized Tilted Marginal is a Probability Measure)**.** ν_t *is a probability measure.*

```
lemma isProbabilityMeasure_stdTiltedLaw (t : ℝ) :
    IsProbabilityMeasure (ν_t X t)
```

Lemma 8.17 (Mean of the Identity under ν_t)**.** $\mathbb{E}_{\nu_t}[\mathrm{id}] = 0$.

```
lemma integral_id_stdTiltedLaw (t : ℝ) :
    ∫ x, x ∂(ν_t X t) = 0
```

Lemma 8.18 (L^2 Integrability of the Identity under ν_t)**.** $\mathrm{id} \in L^2(\nu_t)$.

```
lemma memLp_id_stdTiltedLaw (t : ℝ) :
    MemLp (id : ℝ → ℝ) 2 (ν_t X t)
```

Factorization of $X_1, \ldots, X_n$ under $\mathbb{Q}_{n,t}$.

Lemma 8.19 (Measurability of $\uparrow e^{t \cdot x}$). *$x \mapsto \uparrow e^{t \cdot x}$ is measurable as a function $\mathbb{R} \to [0, \infty]$, where $\uparrow$ denotes the coercion from $\mathbb{R}_{\geq 0}$ to ENNReal.*

```
private lemma measurable_ennreal_ofReal_exp_t_mul (t : ℝ) :
    Measurable (fun x : ℝ => ENNReal.ofReal (Real.exp (t * x)))
```

Lemma 8.20 (Factorization of $\uparrow e^{tS_n}$ as a Product). $\uparrow e^{tS_n} = \prod_{j<n} \uparrow e^{tX_j}$.

```
private lemma ennreal_ofReal_exp_t_partialSum_eq_prod (t : ℝ) (n : ℕ) (ω : Ω) :
    ENNReal.ofReal (Real.exp (t * partialSum X n ω)) =
    ∏ j ∈ Finset.range n, ENNReal.ofReal (Real.exp (t * X j ω))
```

Lemma 8.21 (Measurability of $\uparrow e^{tX_j}$). *For each j, $\uparrow e^{tX_j}$ is measurable.*

```
private lemma measurable_ennreal_ofReal_exp_t_mul_X (t : ℝ) (j : ℕ) :
    Measurable (fun ω => ENNReal.ofReal (Real.exp (t * X j ω)))
```

Lemma 8.22 (Independence of $\uparrow e^{tX_j}$). *The family $\{\uparrow e^{tX_j} : j \in \mathbb{N}\}$ is independent under $\mathbb{P}$.*

```
private lemma iIndepFun_ennreal_ofReal_exp_t_mul_X (t : ℝ) :
    iIndepFun (fun j ω => ENNReal.ofReal (Real.exp (t * X j ω))) ℙ
```

Lemma 8.23 (Lebesgue Integral of $\uparrow e^{tX_j}$). $\int_0^\infty \uparrow e^{tX_j} \, d\mathbb{P} = \uparrow M_{X_0}(t)$

```
private lemma lintegral_ennreal_ofReal_exp_t_mul_X (t : ℝ) (j : ℕ) :
    ∫⁻ ω, ENNReal.ofReal (Real.exp (t * X j ω)) ∂ℙ
      = ENNReal.ofReal (mgf (X 0) ℙ t)
```

Lemma 8.24 (MGF Preamble). *Both $M_{X_0,\mathbb{P}}(t)$ and $M_{S_n,\mathbb{P}}(t)$ are strictly positive, and $\uparrow M_{S_n,\mathbb{P}}(t) = (\uparrow M_{X_0,\mathbb{P}}(t))^n$.*

```
private lemma mgf_preamble (n : ℕ) (t : ℝ) :
    0 < mgf (X 0) ℙ t ∧
    0 < mgf (partialSum X n) ℙ t ∧
    ENNReal.ofReal (mgf (partialSum X n) ℙ t) =
      (ENNReal.ofReal (mgf (X 0) ℙ t)) ^ n
```

Lemma 8.25 (Pushforward of X_i under $\mathbb{Q}_{n,t}$). *For $i < n$, the pushforward of X_i under $\mathbb{Q}_{n,t}$ is the tilted marginal law μ_t.*

```
lemma map_X_ℚnt (t : ℝ) (n : ℕ) {i : ℕ} (hi : i < n) :
    (ℚnt X ℙ n t).map (X i) = μt X t
```

Lemma 8.26 (Independence of X_i under $\mathbb{Q}_{n,t}$)**.** *Under $\mathbb{Q}_{n,t}$, the random variables $\{X_i : i < n\}$ are mutually independent.*

```
lemma iIndepFun_range_ℚ_nt (t : ℝ) (n : ℕ) :
    iIndepFun (fun i : Finset.range n => X i.val) (ℚ_nt X ℙ n t)
```

Lemma 8.27 (Identical Distribution of X_i under $\mathbb{Q}_{n,t}$)**.** *For $i < n$, X_i has the same distribution as X_0 under $\mathbb{Q}_{n,t}$.*

```
lemma identDistrib_X_X0_ℚ_nt (t : ℝ) (n : ℕ) {i : ℕ} (hi : i < n) (h0 : 0 < n) :
    IdentDistrib (X i) (X 0) (ℚ_nt X ℙ n t) (ℚ_nt X ℙ n t)
```

Characteristic functions and Lévy's continuity theorem.

Lemma 8.28 (Characteristic Function of Z_n under $\mathbb{Q}_{n,t}$)**.** $\varphi_{Z_n,\mathbb{Q}_{n,t}}(s) = \left(\varphi_{\nu_t}\left(\frac{s}{\sqrt{n}}\right)\right)^n$.

```
lemma charFun_map_Z_ℚ_nt (t : ℝ) (n : ℕ) (s : ℝ) :
    charFun ((ℚ_nt X ℙ n t).map (Z X t n)) s =
      (charFun (ν_t X t) ((Real.sqrt n)⁻¹ * s)) ^ n
```

Lemma 8.29 (Pointwise CLT for the Standardized Tilted Characteristic Function)**.** $\left(\varphi_{\nu_t}\left(\frac{s}{\sqrt{n}}\right)\right)^n \to e^{-s^2/2}$ *as $n \to \infty$, for every $s \in \mathbb{R}$.*

```
lemma tendsto_charFun_stdTiltedLaw_pow (t : ℝ) (s : ℝ) :
    Tendsto (fun n : ℕ => (charFun (ν_t X t) ((Real.sqrt n)⁻¹ * s)) ^ n)
      atTop (𝓝 (Complex.exp (-s ^ 2 / 2)))
```

Theorem 8.30 (CLT for Z_n under $\mathbb{Q}_{n,t}$)**.** *The characteristic function of the pushforward of Z_n under $\mathbb{Q}_{n,t}$ converges pointwise to $e^{-s^2/2}$, so Z_n converges in distribution to the standard normal.*

```
theorem tendsto_charFun_Z_ℚ_nt (t : ℝ) (s : ℝ) :
    Tendsto (fun n : ℕ => charFun ((ℚ_nt X ℙ n t).map (Z X t n)) s)
      atTop (𝓝 (Complex.exp (-s ^ 2 / 2)))
```

Consequence: Concentration of the Empirical Mean.

Lemma 8.31 (Lower Bound on Half-Intervals)**.** *For every $M \geq 0$ and $\varepsilon > 0$, eventually $(\mathbb{Q}_{n,t}.\mathrm{map}\ Z_n)([0, M]) \geq \mathcal{N}(0,1)([0, M]) - \varepsilon$.*

```
lemma liminf_ℚ_nt_Z_Icc_ge (t : ℝ) (M : ℝ) (hM : 0 ≤ M) (ε : ℝ) (hε : 0 < ε) :
    ∀ᶠ n in atTop, ((gaussianReal 0 1) (Set.Icc (0 : ℝ) M)).toReal - ε ≤
      ((ℚ_nt X ℙ n t).map (Z X t n) (Set.Icc (0 : ℝ) M)).toReal
```

Lemma 8.32 (Standard Normal Half-Mass at 0)**.** $\mathcal{N}(0,1)((-\infty, 0]) = 1/2$.

```
private lemma gaussianReal_Iic_zero_eq_half :
    (gaussianReal 0 1) (Set.Iic (0 : ℝ)) = 1 / 2
```

Lemma 8.33 (Standard Normal Probability of Half-Intervals Tends to 1/2)**.** $\mathcal{N}(0,1)([0, M]) \to 1/2$ *as $M \to \infty$.*

```
lemma tendsto_gaussianReal_Icc_toReal_half :
    Tendsto (fun M : ℝ => ((gaussianReal 0 1) (Set.Icc (0 : ℝ) M)).toReal)
      atTop (𝓝 (1 / 2))
```

Corollary 8.2 (Lean formalization of **Corollary 8.1**). *For any tilting parameter $t \in \mathbb{R}$ with $\Lambda'(t) = a$ and any $\varepsilon, \delta > 0$, there exists N such that for all $n \geq N$, $\mathbb{Q}_{n,t}(\overline{X}_n \in [a, a+\delta]) \geq 1/2 - \varepsilon$.*

```
lemma eventually_ℚ_nt_empiricalMean_mem_Icc_ge
    (t a δ : ℝ) (hδ : 0 < δ) (ε : ℝ) (hε : 0 < ε)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    ∀ᶠ n in atTop,
      (1 / 2 - ε : ℝ) ≤ ((ℚ_nt X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}).toReal
```

8.5.2 The Lower Bound

With the tilted-measure CLT (**Corollary 8.2**) established, we now proceed with the main result of the lower bound. As before, we list the formal statements from `LowerBound.lean` in source order, giving a brief mathematical description of each.

Lemma 8.34 (Exponential Bound on Concentrated Set). *For $t \geq 0$, if $\overline{X}_n \in [a, a+\delta]$, then $e^{-tS_n} \geq e^{-tn(a+\delta)}$.*

```
private lemma exp_neg_mul_S_ge_on_set (t : ℝ) (n : ℕ) (a δ : ℝ) (ht : 0 ≤ t)
    (ω : Ω) (hω : empiricalMean X n ω ∈ Set.Icc a (a + δ)) :
    Real.exp (-t * partialSum X n ω) ≥ Real.exp (-t * n * (a + δ))
```

Listing 18: LargeDeviations/Cramers/LowerBound.lean

Lemma 8.35 (Non-Negativity of $1/n$ in Extended Reals). *For every $n \in \mathbb{N}$, the coercion of $1/n$ to $\overline{\mathbb{R}}$ is non-negative.*

```
private lemma ereal_one_div_nat_nonneg (n : ℕ) : (0 : EReal) ≤ ((1 : ℝ) / n : EReal)
```

Lemma 8.36 (Extended Reals Vanishing Scaled Constant). *In $\overline{\mathbb{R}}$, $\lim_{n\to\infty} \frac{c}{n} = 0$.*

```
lemma ereal_inv_nat_mul_const_tendsto_zero (c : ℝ) :
    Tendsto (fun n : ℕ => ((1 : ℝ) / n : EReal) * (c : EReal)) atTop (𝓝 0)
```

Lemma 8.37 (Log Product Splitting). *$\frac{1}{n}\log(\exp(x)y) = \frac{x}{n} + \frac{1}{n}\log y$ in $\overline{\mathbb{R}}$.*

```
private lemma log_product_split (n : ℕ) (x : ℝ) (y : ENNReal) (hn : n ≥ 1)
    (hy_ne_zero : y ≠ 0) (hy_ne_top : y ≠ ⊤) :
    ((1 : ℝ) / n : EReal) * ENNReal.log (ENNReal.ofReal (Real.exp x * y.toReal)) =
    ((1 : ℝ) / n : EReal) * (x : EReal) + ((1 : ℝ) / n : EReal) * ENNReal.log y
```

Lemma 8.38 (Simplified Logarithmic Expansion). *A specific instance of the log-splitting identity used in the lower-bound proof: for $y > 0$ finite and $n \geq 1$, $\frac{1}{n}\log\big(\exp(-n(t(a+\delta)-l))\cdot y\big) = -(ta-l) - t\delta + \frac{1}{n}\log y$, applied later with $l = \Lambda(t)$.*

```
private lemma log_exp_product_eq_neg_coef_plus_log (n : ℕ) (t a δ : ℝ) (l : ℝ)
    (y : ENNReal) (hn : n ≥ 1) (hy_ne_zero : y ≠ 0) (hy_ne_top : y ≠ ⊤) :
    ((1 : ℝ) / n : EReal) * ENNReal.log (ENNReal.ofReal
      (Real.exp (-n * (t * (a + δ) - l)) * y.toReal)) =
    (-(t * a - l) - t * δ : EReal) + ((1 : ℝ) / n : EReal) * ENNReal.log y
```

Lemma 8.39 (Constant and Vanishing Sequence Limit). *If $f_n \to 0$, then $c + f_n \to c$, evaluated in $\overline{\mathbb{R}}$.*

```
private lemma tendsto_const_add_vanishing (c : EReal) (f : ℕ → EReal)
    (h : Tendsto f atTop (𝓝 0)) : Tendsto (fun n => c + f n) atTop (𝓝 c)
```

Lemma 8.40 (EReal Limit Bound Inequality). *If $x - \varepsilon \leq y$ for all $\varepsilon > 0$, then $x \leq y$ in $\overline{\mathbb{R}}$.*

```
private lemma EReal.le_of_forall_pos_sub_le {x y : EReal}
    (h : ∀ ε : ℝ, 0 < ε → x - (ε : EReal) ≤ y) : x ≤ y
```

Lemma 8.41 (Tilted Window Probability Lower Bound). *Given $\Lambda'(t) = a$, there exists a constant $c > 0$ such that for sufficiently large n, $\mathbb{Q}_{n,t}(\overline{X}_n \in [a, a+\delta]) \geq c$.*

```
private lemma tilted_window_lower_bound_from_concentration (a t δ : ℝ) (hδ : 0 < δ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    ∃ c > 0, ∀ᶠ n in atTop,
      c ≤ ((ℚ_[n, t] X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}).toReal
```

Lemma 8.42 (Measure via Tilted Integral). $\mathbb{P}(E) = \mathbb{E}[\exp(f)]\int_E \exp(-f)d\mathbb{Q}$.

```
private lemma measure_eq_integral_exp_neg_tilted (f : Ω → ℝ) (E : Set Ω)
    (h_int : Integrable (fun ω => Real.exp (f ω)) ℙ)
    (hE : MeasurableSet E) :
    (ℙ E).toReal =
      (𝔼[fun ω => Real.exp (f ω)]) * (∫ ω in E, Real.exp (-f ω) ∂(Measure.tilted ℙ f))
```

Lemma 8.43 (Change of Measure Lower Bound). *For $E = \{\overline{X}_n \in [a, a+\delta]\}$ and $t > 0$, $\mathbb{P}(E) \geq e^{-n(t(a+\delta)-\Lambda(t))}\mathbb{Q}_{n,t}(E)$.*

```
lemma change_of_measure_lower_bound (a δ t : ℝ) (n : ℕ) (ht : 0 < t)
    (h_int : Integrable (fun ω => Real.exp (t * partialSum X n ω)) ℙ) :
    let E := {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}
    (ℙ E).toReal ≥
      Real.exp (-n * (t * (a + δ) - cgf (X 0) ℙ t)) *
      ((ℚ_[n, t] X ℙ n t) E).toReal
```

Listing 19: LargeDeviations/Cramers/LowerBound.lean

Lemma 8.44 (Error Term Tends to Zero). *Given* $\Lambda'(t) = a$, $\frac{1}{n} \log \mathbb{Q}_{n,t}(\overline{X}_n \in [a, a+\delta]) \to 0$.

```
private lemma error_term_vanishes (a t δ : ℝ) (hδ : 0 < δ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    Tendsto (fun n : ℕ =>
      ((1 : ℝ) / n : EReal) * ENNReal.log ((ℚ_{n t} X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)})) atTop (𝓝 0)
```

Lemma 8.45 (Liminf Bound via Tilting). *For* $\delta, t > 0$ *with* $\Lambda'(t) = a$, $\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -(ta - \Lambda(t)) - t\delta$.

```
private lemma lower_bound_via_tilted (a t δ : ℝ) (hδ : 0 < δ) (ht : 0 < t)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    liminf (fun n : ℕ =>
      ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop
    ≥ (-(t * a - cgf (X 0) ℙ t) : EReal) - (t * δ : EReal)
```

Lemma 8.46 (Tilting Parameter is Non-Negative). *For* $a \geq \mathbb{E}[X]$, *if* $\Lambda'(t) = a$, *then* $t \geq 0$.

```
private lemma deriv_cgf_nonneg_of_ge_mean (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) (t : ℝ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    0 ≤ t
```

Lemma 8.47 (Global Analyticity of the CGF). *The CGF* Λ *is analytic on all of* $\mathbb{R}$.

```
private lemma analyticOn_cgf_univ : AnalyticOn ℝ (cgf (X 0) ℙ) Set.univ
```

Lemma 8.48 (Legendre Transform Maximum). *Given* $\Lambda'(t) = a$, $I(a) = ta - \Lambda(t)$.

```
private lemma rateFunction_eq_of_deriv_eq (a t : ℝ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    I X a = t * a - cgf (X 0) ℙ t
```

Lemma 8.49 (Lower Bound at the Mean). *For* $a = \mathbb{E}[X_1]$, *the lower bound holds with the trivial rate:* $\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq 0$.

```
private lemma cramer_lower_bound_at_mean (a : ℝ) (ht_deriv : deriv (cgf (X 0) ℙ) 0 = a) :
    (0 : EReal) ≤
      liminf (fun n : ℕ =>
        ((1 : ℝ) / (n : ℝ) : EReal) *
          ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop
```

Theorem 8.50 (Cramér's Theorem Lower Bound). *For* $a \geq \mathbb{E}[X_0]$, $\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -I(a)$.

```
theorem cramer_lower_bound (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) :
    (- I X a : EReal) ≤
      liminf (fun n : ℕ =>
        ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop
```

Listing 20: LargeDeviations/Cramers/LowerBound.lean

The following lemmas are supplementary and not needed for the lower bound proof presented here. We retain them as results of additional interest based on the Chebyshev concentration of the tilted measure.

Lemma 8.51 (Vanishing Variance Term). *For constant C and $\delta > 0$, $\lim_{n\to\infty} \frac{1}{n}\frac{C}{\delta^2} = 0$.*

```
private lemma variance_term_tendsto_zero (C : ℝ) (δ : ℝ) :
    Tendsto (fun n : ℕ => ENNReal.ofReal ((1 / n) * C / δ ^ 2)) atTop (𝓝 0)
```

Lemma 8.52 (Equality from Integral Identity). *Under a probability measure μ, if f is integrable, $f \le c$ almost everywhere, and $\int f\, d\mu = c$, then $f = c$ almost everywhere.*

```
private lemma ae_eq_const_of_integral_eq_const_of_le {α : Type*} {m : MeasurableSpace α}
    (μ : Measure α) [IsProbabilityMeasure μ] {f : α → ℝ} (c : ℝ)
    (hf : Integrable f μ) (h_le : ∀ᵐ ω ∂μ, f ω ≤ c) (h_int : μ[f] = c) :
    ∀ᵐ ω ∂μ, f ω = c
```

Lemma 8.53 (CGF of Partial Sum as a Function). $\Lambda_{S_n}(t) = n\Lambda_{X_1}(t)$

```
private lemma cgf_partialSum_eq_smul_cgf (n : ℕ) :
    cgf (partialSum X n) ℙ = fun s => n * cgf (X 0) ℙ s
```

Lemma 8.54 (Derivative of CGF for Sum). $\Lambda'_{S_n}(t) = n\Lambda'_{X_1}(t)$.

```
private lemma deriv_cgf_sum (t : ℝ) (n : ℕ) :
    deriv (cgf (partialSum X n) ℙ) t = n * deriv (cgf (X 0) ℙ) t
```

Lemma 8.55 (Second Derivative of CGF for Sum). $\Lambda''_{S_n}(t) = n\Lambda''_{X_1}(t)$.

```
private lemma iteratedDeriv_two_cgf_sum (t : ℝ) (n : ℕ) :
    iteratedDeriv 2 (cgf (partialSum X n) ℙ) t =
      n * iteratedDeriv 2 (cgf (X 0) ℙ) t
```

Lemma 8.56 (Moments of Partial Sum under Tilting). *Under $\mathbb{Q}_{n,t}$, $\mathbb{E}[S_n] = \Lambda'_{S_n}(t)$ and $\mathbb{V}(S_n) = \Lambda''_{S_n}(t)$.*

```
private lemma tilted_Sn_moments (t : ℝ) (n : ℕ) :
    let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
    μ_t[partialSum X n] = deriv (cgf (partialSum X n) ℙ) t ∧
    variance (partialSum X n) μ_t = iteratedDeriv 2 (cgf (partialSum X n) ℙ) t
```

Lemma 8.57 (Moments of Empirical Mean under Tilting). *Under $\mathbb{Q}_{n,t}$, $\mathbb{E}[\overline{X}_n] = \Lambda'(t)$ and $\mathbb{V}(\overline{X}_n) = \frac{1}{n}\Lambda''(t)$.*

```
private lemma tilted_empirical_moments (t : ℝ) (n : ℕ) (hn : n ≠ 0) :
    let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
    μ_t[empiricalMean X n] = deriv (cgf (X 0) ℙ) t ∧
    variance (empiricalMean X n) μ_t = (1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ) t
```

Lemma 8.58 (Tilted Empirical Mean Equals a at Matching Derivative). *For $n \geq 1$, if $\Lambda'(t) = a$ then $\mathbb{E}_{\mathbb{Q}_{n,t}}[\overline{X}_n] = a$.*

```
private lemma tilted_empirical_mean_eq_of_deriv (t a : ℝ) (n : ℕ) (hn : n ≠ 0)
    (h_deriv : deriv (cgf (X 0) ℙ) t = a) :
    (Measure.tilted ℙ (fun ω => t * partialSum X n ω))[empiricalMean X n] = a
```

Lemma 8.59 (Chebyshev Deviation Bound for Tilted Measure). $\mathbb{Q}_{n,t}(|\overline{X}_n - a| \geq \delta) \leq \frac{1}{n}\frac{\Lambda''(t)}{\delta^2}$.

```
private lemma tilted_deviation_bound (t a : ℝ) (n : ℕ) (hn : n ≠ 0) (δ : ℝ) (hδ : 0 < δ)
    (h_match : deriv (cgf (X 0) ℙ) t = a) :
    let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
    μ_t {ω | δ ≤ |empiricalMean X n ω - a|} ≤
      ENNReal.ofReal ((1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ) t / δ ^ 2)
```

Lemma 8.60 (Window Concentration under Tilting). $\mathbb{Q}_{n,t}(|\overline{X}_n - a| < \delta) \to 1$ *as* $n \to \infty$.

```
private lemma tilted_measure_concentrates (t a δ : ℝ) (hδ : 0 < δ)
    (h_match : deriv (cgf (X 0) ℙ) t = a) :
    Tendsto (fun n => ((ℚ_nt X ℙ n t)
      {ω | |empiricalMean X n ω - a| < δ}).toReal) atTop (𝓝 1)
```

Lemma 8.61 (Positivity of the Tilted Tail at a). *Given $\Lambda'(t) = a$ and $n \geq 1$, $\mathbb{Q}_{n,t}(\overline{X}_n \geq a) > 0$.*

```
private lemma tilted_prob_ge_mean_pos (a t : ℝ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) (n : ℕ) (hn : n ≠ 0) :
    0 < (ℚ_nt X ℙ n t) {ω | a ≤ empiricalMean X n ω}
```

9 Putting it All Together

9.1 Cramér's Theorem

With both the upper and lower bounds established, we now combine them to prove the main theorem stated in chapter 5.

Proof of Theorem 5.1. We need to show that $\overline{X}_n$ satisfies a Large Deviation Principle with rate function I. That is, for all $a \in \mathbb{R}$:

$$\limsup_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq -I(a) \tag{58}$$

$$\liminf_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \geq -I(a) \tag{59}$$

Recall that we defined the *effective* rate function piecewise as

$$I(a) = \begin{cases} \sup_{t\in\mathbb{R}}\{ta - \Lambda(t)\} & \text{if } a \geq \mathbb{E}[X_1] \\ 0 & \text{if } a < \mathbb{E}[X_1] \end{cases} \tag{60}$$

We proceed by cases.

Case 1: $a \geq \mathbb{E}[X_1]$**.** In this case, $I(a) = \sup_{t\in\mathbb{R}}\{ta - \Lambda(t)\}$ is the Legendre transform. The upper bound (58) is precisely theorem 7.1, established by the Chernoff bound in chapter 7. The lower bound (59) is theorem 8.2, established by exponential tilting in chapter 8. Combined, they give the desired asymptotic result:

$$\lim_{n\to\infty} \frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) = -I(a) \tag{61}$$

Case 2: $a < \mathbb{E}[X_1]$**.** In this case, $I(a) = 0$ by definition of `upperTailRateFunction`, so we must show that both the limsup and the liminf of $\frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a)$ are 0.

Upper bound. This follows trivially from the fact that probabilities are bounded above by 1. We have

$$\frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \leq \frac{1}{n} \log 1 = 0 = -I(a) \tag{62}$$

for every $n \geq 1$. Taking the limsup preserves this bound, giving (58).

Lower bound. By the strong law of large numbers, $\overline{X}_n \to \mathbb{E}[X_1]$ almost surely. Since $a < \mathbb{E}[X_1]$, we have that $\mathbb{P}(\overline{X}_n \geq a) \to 1$. As the logarithm is continuous at 1, $\log \mathbb{P}(\overline{X}_n \geq a) \to 0$. Then as $\frac{1}{n} \to 0$,

$$\frac{1}{n} \log \mathbb{P}(\overline{X}_n \geq a) \to 0 = -I(a) \tag{63}$$

In particular, the liminf equals zero, giving (59). □

9.2 Formalized Results

As before, we list the formalized statements that make up the main theorem's proof, giving a brief mathematical description of each statement, and omitting the proofs, which are listed in full in Appendix A.6.

Lemma 9.1 (Typical Event Limit)**.** *For $a < \mathbb{E}[X_0]$, $\mathbb{P}(\overline{X}_n \geq a) \to 1$ as $n \to \infty$.*

```
private lemma tendsto_prob_empiricalMean_ge_of_lt_mean_one (a : ℝ) (h : a < 𝔼[X 0]) :
  Tendsto (fun n : ℕ => (ℙ {ω | a ≤ empiricalMean X n ω} : ENNReal)) atTop (𝓝 1)
```

Listing 21: LargeDeviations/Cramers/Theorem.lean

Theorem 9.2 (Cramér's Theorem)**.** *The empirical mean of i.i.d. random variables with finite MGF satisfies a Large Deviation Principle with rate function* `upperTailRateFunction`*.*

```
theorem cramers_theorem :
    LargeDeviationPrinciple (empiricalMean X) (upperTailRateFunction X)
```

Listing 22: LargeDeviations/Cramers/Theorem.lean

10 Conclusions and Future Work

10.1 Summary

We have presented a formally verified proof of Cramér's theorem on Large Deviations, developed in the Lean 4 proof assistant and building on top of the Mathlib mathematical library.

The formalization comprises approximately 2,003 lines of code across six files and, to our knowledge, constitutes the first formalization of any large deviation result in a proof assistant.

Our contributions include the introduction of the Large Deviation Principle as a formal Lean structure, a proof of the upper bound by the Chernoff bound, a proof of the lower bound by exponentially tilted measures, and a proof of the Central Limit Theorem for sample means under a sequence of exponentially tilted measures.

The formalization builds on Mathlib's existing infrastructure for measure and probability theory, limits and filters, tilted measures, the strong law of large numbers, and the extended reals, demonstrating that the library has sufficient maturity to support further work in advanced probability theory.

10.2 Reflections on Formalization

Several aspects of the formalization merit discussion as lessons for future work in formalization.

Type coercion overhead. The LDP statement we defined uses the extended real numbers $\overline{\mathbb{R}}$ to correctly express $\log 0 = -\infty$, while intermediate computations like bounding exponentials to get the Chernoff bound or optimizing over tilting parameters were done in $\mathbb{R}$. Transferring results between these required additional work in proving coercion lemmas throughout the formalization. In set-theoretic mathematics, the fact that $\mathbb{R} \subset \overline{\mathbb{R}}$ is taken for granted, whereas with Lean's type-theoretic foundations, even with Lean's extensive syntactic support, such type lifts often need to be made explicit.

Lower bound complexity. `LowerBound.lean` and `TiltedCLT.lean` together account for the majority of the total codebase. This is despite the lower bound having comparable length to the upper bound in many informal mathematical treatments, and despite Mathlib already containing formalizations of tilted measures and the standard CLT. The disparity primarily comes from being forced to express, in full rigor, every aspect and subtlety of the lower bound and the tilted CLT. Whereas informal proofs may hand-wave the argument around the change of measure and the sequence of tilted measures, we keep extensive formal bookkeeping. Many steps that may otherwise be considered routine or obvious — such as the fairly nontrivial property that the random variables are still i.i.d. under the tilted measures — require explicit justification in Lean.

Filter-based asymptotics. Mathlib's filters provide a unified formalization of limits, convergence, and asymptotic statements. Although the single unified framework is somewhat more elegant than the usual assortment of ad hoc ε-δ or ε-N proofs, working with it requires adjusting to a technique that differs from the conventions of standard mathematical literature.

Mathlib as an enabler. Building on top of Mathlib's foundations — including existing formalizations of measure-theoretic probability, Chernoff bounds, exponentially tilted measures, the strong law of large numbers, and miscellaneous results in analysis, probability, and limits — was crucial to the success of this work. Without such precursors, not only would the scope of implementation substantially increase, but the mental overhead of deciding how best to represent abstract mathematical concepts in Lean's type-theoretic foundations would have added another degree of challenge. The rapid growth of Mathlib's probability foundations made work of this level feasible, and recent additions on martingale convergence [15], Markov kernels [16], and sub-Gaussian variables are indicative of future avenues of formalization potential.

Conversely, our formalization contributed various results that could be applied more broadly in future proofs: In addition to the various intermediate results already stated, two general-purpose lemmas on cumulant generating functions were in fact implemented directly in Mathlib's existing Moments module, and are listed in Chapter B.1.1.

10.3 Future Work

Several natural extensions of this work present themselves.

Generalizing the LDP. We have provided a formalization of the LDP for upper tail events $\{\overline{X}_n \geq a\}$ of real-valued random variables. A more general statement of the principle, as given by Dembo and Zeitouni [2], is in terms of a family of measures $\{\mu_\varepsilon\}$ on a topological space $\mathcal{X}$ with an associated σ-algebra $\mathcal{B}$. We say that $\{\mu_\varepsilon\}$ satisfies an LDP with rate function I if, for all $B \in \mathcal{B}$,

$$-\inf_{x \in \mathring{B}} I(x) \leq \liminf_{\varepsilon \to 0} \varepsilon \log \mu_\varepsilon(B) \leq \limsup_{\varepsilon \to 0} \varepsilon \log \mu_\varepsilon(B) \leq -\inf_{x \in \overline{B}} I(x) \tag{64}$$

Formalizing this general statement is a natural next step and would build upon Mathlib's well-developed topological and measure-theoretic foundations.

Relaxing requirements on the MGF. As preliminaries for the formalization, we took as assumptions that the MGF is finite everywhere (`h_mgf`) and that the rate function is bounded everywhere.

For many distributions of interest these conditions do not hold, yet LDP statements can still be made. For instance, our worked example of deriving the bounds for the exponential distribution in Chapter 7.2 and Chapter 8.4 did not require the MGF to exist everywhere; the MGF was instead finite only on a particular domain. Similarly, the assumption `h_exposed` requires that all $a \geq \mathbb{E}[X_1]$ are in the range of the CGF, which excludes, for instance, the Bernoulli distribution. A formalization that relaxes the various requirements we've imposed could cover a much broader class of distributions.

Higher-dimensional and abstract extensions. There are many possible extensions of Cramér's theorem: the theorem can be extended directly to $\mathbb{R}^d$, where we have an LDP result with rate function given by the Legendre transform of the multidimensional CGF [2]. The Gärtner-Ellis theorem [4] relaxes the restriction that the random variables X_i be independent and identically distributed, providing looser conditions in terms of the existence of a limiting scaled cumulant generating function $\Lambda(\lambda) = \lim_{n\to\infty} \frac{1}{n} \log \mathbb{E}[e^{\langle \lambda, S_n \rangle}]$. Sanov's theorem [2] states an LDP result for the empirical measures of sequences of i.i.d. random variables with the Kullback-Leibler divergence as the rate function, while Mogulskii's theorem gives an LDP for random walks [2, 3]. The contraction principle [2] provides machinery for deriving new LDPs from existing ones through continuous mappings. Each of these generalizations would build upon Mathlib's infrastructure and require the development of new foundations.

AI-assisted formalization. Large Language Models (LLMs) have seen significant adoption by the Lean community as aids in developing formal proofs: the proof-writing process can be tedious, discovering relevant existing lemmas is challenging, and the selection and application of appropriate tactics is often an uninteresting and mechanical process. To assist with such work, a wide variety of tooling has been developed, ranging from those that simply augment the writing experience like Lean Copilot [20], which provides in-editor tactic suggestions, to attempts at complete automation like Aristotle [21] that combines higher-level LLM reasoning with lower-level formal proof search, achieving the ability to solve competition-level problems. In this project, we also used Claude Code as an assistant for Lean 4 coding and for broader research and mathematical reasoning, finding it particularly helpful for navigating Mathlib's API, discovering proofs and strategies, and helping debug with type errors.

Interactive proof assistants aided by LLMs hint at an interesting direction for the future of formal math: Unlike most other LLM-generated content, Lean proofs can be completely mechanically checked, allowing

for such models to be used as a smarter, directed search over the space of derivations for a proof. This hints at a model of research where a human engages in higher-level exploration of a theory and an LLM semi-autonomously formalizes and verifies the work.

A Lean 4 Formalization Code Listings

A.1 Mathlib/Probability/LargeDeviations/Defs.lean

```
module
public import Mathlib.Analysis.SpecialFunctions.Log.ENNRealLog
public import Mathlib.MeasureTheory.Measure.MeasureSpace
public import Mathlib.Probability.Notation
/-!
# Large Deviation Principles

This file defines the `LargeDeviationPrinciple` for sequences of random variables.

A large deviation principle (LDP) characterizes the rate at which the probability of
tail events decay. Formally, we say that a sequence of random variables `(Y_n)` satisfies
an LDP with rate function `I` if for every `a` the limsup (resp. liminf) of
`(1/n) * log ℙ(Y_n ≥ a)` is at most (resp. at least) `-I(a)`.

## Main definitions
- `LargeDeviationPrinciple`: A Prop-valued structure containing the upper and lower bound
## Notes
The definition splits the upper and lower bounds so that each side can be proven separately.

This statement is given for sequences of real-valued random variables. However, this
could be further generalized to be in terms of Borel-measurable sets in Polish spaces.
## References
* <https://en.wikipedia.org/wiki/Large_deviation_theory>
* <https://en.wikipedia.org/wiki/Cramér%27s_theorem_(large_deviations)>
-/

open MeasureTheory ProbabilityTheory Filter Topology
open scoped ENNReal

@[expose] public section
namespace ProbabilityTheory

variable {Ω : Type*} [MeasureSpace Ω]

/-- A sequence of real-valued random variables `(Y_n)` satisfies a **Large Deviation
Principle** with rate function `I : ℝ → ℝ` if for every `a` the limsup (resp. liminf)
of the scaled log-tail probability `(1/n) * log ℙ(Y_n ≥ a)` is at most (resp. at least)
`-I(a)`.
The sub-goals are defined in terms of the extended real numbers to account for events
with probability 0.
-/
structure LargeDeviationPrinciple (Y : ℕ → Ω → ℝ) (I : ℝ → ℝ) : Prop where
  /-- Upper bound: limsup_n (1/n) log ℙ(Y_n ≥ a) ≤ -I(a) -/
  upperBound : ∀ a : ℝ,
    limsup (fun n : ℕ => ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | Y n ω ≥ a}))
      atTop ≤ (- I a : EReal)
  /-- Lower bound: liminf_n (1/n) log ℙ(Y_n ≥ a) ≥ -I(a) -/
  lowerBound : ∀ a : ℝ,
    (- I a : EReal) ≤
      liminf (fun n : ℕ => ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | Y n ω ≥ a}))
        atTop
end ProbabilityTheory
```

A.2 Mathlib/Probability/LargeDeviations/Cramers/Basic.lean

```
module
public import Mathlib.Probability.IdentDistrib
public import Mathlib.Probability.Independence.Basic
public import Mathlib.Probability.Moments.Basic
public import Mathlib.Probability.Moments.IntegrableExpMul
public import Mathlib.Probability.Moments.Tilted
public import Mathlib.Probability.Independence.Integration
public import Mathlib.Analysis.Convex.Integral
public import Mathlib.Analysis.Convex.SpecificFunctions.Basic
public import Mathlib.Analysis.SpecialFunctions.Log.ENNRealLog
public import Mathlib.Analysis.SpecialFunctions.Log.ENNRealLogExp
public import Mathlib.Analysis.SpecificLimits.Basic
public import Mathlib.Analysis.Calculus.Deriv.Basic

/-!
# Cramér's Theorem - Basic Definitions and Infrastructure

This file contains the core definitions and shared lemmas for Cramér's theorem:
- The definitions `partialSum`, `empiricalMean`, `I` (rate function), `upperTailRateFunction`.
- Basic measurability and integrability results.
- MGF/CGF sum formulas via independence.

The upper and lower bounds are proven in `UpperBound.lean` and `LowerBound.lean`;
the main theorem is assembled in `Theorem.lean`.
-/

open ProbabilityTheory MeasureTheory Filter Topology
open scoped ENNReal

@[expose] public section

namespace ProbabilityTheory

variable {Ω : Type*} [MeasureSpace Ω]
variable (X : ℕ → Ω → ℝ)

/-- The partial sum X₁ + ... + Xₙ. -/
def partialSum (n : ℕ) : Ω → ℝ := Σ i ∈ Finset.range n, X i

/-- The empirical mean Sₙ / n. -/
noncomputable def empiricalMean (n : ℕ) : Ω → ℝ := fun ω => partialSum X n ω / n

/-- The Legendre transform of the CGF. This is the rate function for Cramér's theorem. -/
noncomputable def I (x : ℝ) : ℝ := ⨆ t : ℝ, t * x - cgf (X 0) ℙ t

/-- The "Effective" Rate Function for the upper tail probability `ℙ(empiricalMean X n ≥ a)`.
Cramér's theorem only holds when `a ≥ 𝔼[X 0]`, so to state a general Large Deviation Principle
for all `a`, we define the rate function to be `I(a)` for `a ≥ 𝔼[X 0]`, and `0` otherwise. -/
noncomputable def upperTailRateFunction (X : ℕ → Ω → ℝ) (a : ℝ) : ℝ :=
  if 𝔼[X 0] ≤ a then I X a else 0

/-- The exponentially tilted measure, tilted by `t · Sₙ`. -/
noncomputable def ℚ_nt (X : ℕ → Ω → ℝ) (μ : Measure Ω) (n : ℕ) (t : ℝ) : Measure Ω :=
  Measure.tilted μ (fun ω => t * partialSum X n ω)
```

```
/- Assumptions for Cramér's theorem -/
-- The random variables Xᵢ are independent.
variable (h_indep : iIndepFun X ℙ)
-- The random variables Xᵢ are identically distributed.
variable (h_ident : ∀ n, IdentDistrib (X n) (X 0) ℙ ℙ)
-- The random variables Xᵢ are measurable.
variable (h_meas : ∀ n, Measurable (X n))
-- The random variable X₀ is integrable.
-- Note: This is implied by h_mgf but we assume it directly for convenience.
variable (h_int : Integrable (X 0) ℙ)
-- The random variable X₀ has a finite moment generating function for all `t ∈ ℝ`.
variable (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * X 0 ω)) ℙ)
-- Assume that this is a "good" rate function, i.e. bounded above.
-- Note: This is implied by h_mgf but difficult to prove directly and beyond scope here.
variable (h_bdd : ∀ a : ℝ, BddAbove (Set.range (fun t => t * a - cgf (X 0) ℙ t)))
-- Assume the distribution is non-degenerate (has positive variance everywhere)
-- Note that `Λ''(0) = Var[X]`, so this implies `Var[X] > 0` which implies `X` is non-const a.e.
-- Cumulant generating functions, when they exist and are non-degenerate, are strictly convex.
variable (h_non_deg : ∀ t : ℝ, 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)
-- For the lower bound, we assume points are "exposed" (in range of cgf derivative).
-- This is equivalent to stating that we can find a t such that the probability measure
-- tilted by `t · X` has expectation `a`, and thus the measure tilted by `t · Sₙ`
-- has expectation `n · a`.
variable (h_exposed : ∀ a : ℝ, 𝔼[X 0] ≤ a → ∃ t, deriv (cgf (X 0) ℙ) t = a)

/-! ### Basic measurability and integrability helpers -/

include h_ident h_mgf in
/-- The random variables Xᵢ have finite moment generating functions. -/
lemma integrable_exp_of_identDistrib (i : ℕ) (t : ℝ) :
    Integrable (fun ω => Real.exp (t * X i ω)) ℙ :=
  ((h_ident i).comp (measurable_const.mul measurable_id).exp).integrable_iff.mpr (h_mgf t)

include h_meas in
/-- The partial sum `Sₙ` is measurable. -/
lemma measurable_partialSum (n : ℕ) : Measurable (partialSum X n) := by
  simpa [partialSum, ← Finset.sum_apply] using
    Finset.measurable_sum (Finset.range n) (fun i _ => h_meas i)

include h_meas in
/-- The empirical mean `Sₙ/n` is measurable. -/
lemma measurable_empiricalMean (n : ℕ) : Measurable (empiricalMean X n) :=
  (measurable_partialSum X h_meas n).div_const (n : ℝ)

include h_mgf in
/-- All `t ∈ ℝ` lie in the interior of the domain for which `exp(tX₀)` is integrable. -/
lemma mem_interior_integrableExpSet (t : ℝ) : t ∈ interior (integrableExpSet (X 0) ℙ) := by
  simp [Set.eq_univ_of_forall h_mgf (s := integrableExpSet (X 0) ℙ)]
```

```
/-! ### Lemmas requiring `IsProbabilityMeasure` -/

variable [IsProbabilityMeasure (ℙ : Measure Ω)]

/-- For a random variable Y with finite MGF, the CGF satisfies `Λ_Y(t) ≥ t · 𝔼[Y]`.
This follows from Jensen's inequality applied to the convex function `e^{tx}` for fixed `t`. -/
lemma cgf_ge_mul_expect (Y : Ω → ℝ) (h_int : Integrable Y ℙ)
    (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * Y ω)) ℙ) (t : ℝ) :
    cgf Y ℙ t ≥ t * 𝔼[Y] := by
  -- Λ(t) = log 𝔼[exp(tY)] ≥ log exp(t 𝔼[Y]) = t 𝔼[Y]
  rw [cgf, mgf]
  -- Apply Jensen's inequality: exp(𝔼[tY]) ≤ 𝔼[exp(tY)]
  have jensen := ConvexOn.map_integral_le
    (g := Real.exp) (s := Set.univ) (f := fun ω => t * Y ω)
    (convexOn_exp) Real.continuous_exp.continuousOn isClosed_univ
    (ae_of_all _ (fun _ => Set.mem_univ _))
    (h_int.const_mul t) (h_mgf t)
  -- Extract t: t 𝔼[Y] ≤ 𝔼[exp(tY)]
  rw [integral_const_mul] at jensen
  -- Take log of both sides
  exact (Real.log_exp _).symm.trans_le (Real.log_le_log (Real.exp_pos _) jensen)

/-- When `t < 0` and `a ≥ 𝔼[Y]`, we have `t · a - Λ_Y(t) ≤ 0`. -/
lemma rate_function_neg_param_le_zero (Y : Ω → ℝ) (h_int : Integrable Y ℙ)
    (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * Y ω)) ℙ)
    (t a : ℝ) (ht : t < 0) (ha : 𝔼[Y] ≤ a) :
    t * a - cgf Y ℙ t ≤ 0 := by
    -- Λ(t) ≥ t · 𝔼[Y] ≥ t · a  (since t < 0, inequality flips)
  nlinarith [cgf_ge_mul_expect Y h_int h_mgf t, mul_le_mul_of_nonpos_left ha ht.le]

include h_indep h_ident h_meas h_mgf in
/-- If each `X_i` has finite MGF, then `S_n` also has finite MGF. -/
lemma integrable_exp_sum (t : ℝ) (n : ℕ) :
    Integrable (fun ω => Real.exp (t * partialSum X n ω)) ℙ := by
  have h_rw : (fun ω => Real.exp (t * partialSum X n ω)) =
      fun ω => ∏ i ∈ Finset.range n, Real.exp (t * X i ω) := by
    ext ω; simp [partialSum, Finset.sum_apply, Finset.mul_sum, Real.exp_sum]
  rw [h_rw]; clear h_rw
  induction n with
  | zero => simp
  | succ n ih =>
      -- Show product `e^(t * X_i)` over range `n + 1`
      --   = (product `e^(t * X_i)` over range `n`) * `e^(t * X_n)`
    simp only [Finset.prod_range_succ]
    have h_indep_exp : iIndepFun (fun i ω => Real.exp (t * X i ω)) ℙ := by
      simpa [Function.comp_def] using h_indep.comp (fun _ x => Real.exp (t * x))
        (fun _ => (measurable_const.mul measurable_id).exp)
    -- `∏_i^{n-1} e^{t * X_i}` is independent of `e^{t * X_n}`
    have h_indep_prod : IndepFun (fun ω => ∏ i ∈ Finset.range n, Real.exp (t * X i ω))
        (fun ω => Real.exp (t * X n ω)) ℙ := by
      convert h_indep_exp.indepFun_finset_prod_of_notMem
        (fun i => (h_meas i).const_mul t |>.exp) (by simp : n ∉ Finset.range n) using 2
      simp [Finset.prod_apply]
    exact h_indep_prod.integrable_mul ih
      (integrable_exp_of_identDistrib X h_ident h_mgf n t)
```

```
include h_indep h_ident h_meas h_mgf in
/-- The tilted measure by `t · S_n` is a probability measure. -/
lemma isProbabilityMeasure_tilted_partialSum (t : ℝ) (n : ℕ) :
    IsProbabilityMeasure (ℚ_nt X ℙ n t) :=
  isProbabilityMeasure_tilted (integrable_exp_sum X h_indep h_ident h_meas h_mgf t n)

include h_indep h_ident h_meas h_mgf in
/-- All `t ∈ ℝ` is in the interior of the domain where `e^{t S_n}` is integrable. -/
lemma mem_interior_integrableExpSet_partialSum (t : ℝ) (n : ℕ) :
    t ∈ interior (integrableExpSet (partialSum X n) ℙ) := by
  simp [Set.eq_univ_of_forall
    (fun s => integrable_exp_sum X h_indep h_ident h_meas h_mgf s n)
    (s := integrableExpSet (partialSum X n) ℙ)]

include h_bdd h_mgf h_int in
/-- For `a ≥ 𝔼[X]`, the supremum in the rate function is achieved by non-negative `t`.
That is, `I(a) = sup_{t ∈ ℝ⁺} (tx - Λ(t))` -/
lemma rateFunction_eq_sup_nonneg (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) :
    I X a = ⨆ t : {(x : ℝ) | 0 ≤ x}, (t : ℝ) * a - cgf (X 0) ℙ t := by
  rw [I]
  have : Nonempty {x : ℝ | 0 ≤ x} := ⟨⟨0, by simp⟩⟩
  have h_bdd_restrict : BddAbove (Set.range fun t : {x : ℝ | 0 ≤ x} =>
      (t : ℝ) * a - cgf (X 0) ℙ t) :=
    let ⟨b, hb⟩ := h_bdd a
    ⟨b, fun _ ⟨t, ht⟩ => ht ▸ hb ⟨t.val, rfl⟩⟩
  refine le_antisymm (ciSup_le fun t => ?_) (ciSup_le fun t => le_ciSup (h_bdd a) (t : ℝ))
  by_cases ht : 0 ≤ t
  -- Case t ≥ 0: It's in the restricted set so the supremum exists by h_bdd
  · exact le_ciSup h_bdd_restrict ⟨t, ht⟩
  -- Case t < 0: It's bound by the value at t=0, so the supremum is always achievable with a t ≥ 0
  · calc t * a - cgf (X 0) ℙ t
        ≤ 0 := rate_function_neg_param_le_zero (X 0) h_int h_mgf t a (not_le.mp ht) h_mean
      _ = (0 : ℝ) * a - cgf (X 0) ℙ 0 := by simp [cgf_zero]
      _ ≤ _ := le_ciSup h_bdd_restrict ⟨0, by simp⟩

include h_indep h_ident h_meas h_mgf in
/-- `M_S_n(t) = exp(n · Λ_X₀(t))` -/
lemma mgf_sum_eq_exp_n_prod_cgf (n : ℕ) (t : ℝ) :
    mgf (partialSum X n) ℙ t = Real.exp (n * cgf (X 0) ℙ t) := by
  rcases n with _ | n
  · simp [partialSum, cgf]
  change mgf (Σ i ∈ Finset.range (n + 1), X i) ℙ t = _
  rw [mgf_sum_of_identDistrib h_meas h_indep
      (fun i _ j _ => (h_ident i).trans (h_ident j).symm)
      (Finset.mem_range.mpr n.succ_pos) t,
    Finset.card_range, cgf, mgf]
  conv_lhs => rw [← Real.exp_log (integral_exp_pos (h_mgf t))]
  rw [← Real.exp_nsmul, nsmul_eq_mul]

include h_indep h_ident h_meas h_mgf in
/-- `Λ_S_n(t) = n · Λ_X₀(t)` -/
lemma cgf_sum_eq_n_prod_cgf (n : ℕ) (t : ℝ) :
    cgf (partialSum X n) ℙ t = (n : ℝ) * cgf (X 0) ℙ t := by
  rw [cgf, mgf_sum_eq_exp_n_prod_cgf X h_indep h_ident h_meas h_mgf, Real.log_exp]

end ProbabilityTheory
```

A.3 Mathlib/Probability/LargeDeviations/Cramers/UpperBound.lean

```
module

public import Mathlib.Probability.LargeDeviations.Cramers.Basic

/-!
# Cramér's Theorem - Upper Bound

This file proves the upper (Chernoff) bound for Cramér's theorem:

- `cramer_upper_bound`: For any `a ≥ 𝔼[X 0]`, `limsup (1/n) * log ℙ(S_n/n ≥ a) ≤ -rateFunction X
    a`.

The proof applies Markov's inequality to `exp(t * S_n)` for each `t ≥ 0`,
takes the infimum over `t`, and uses the scaling identity `cgf(S_n) = n * cgf(X_0)`.

We use `ProbabilityTheory.measure_ge_le_exp_cgf` that gives the Chernoff bound for the upper
tail of a real-valued random variable.
-/

open ProbabilityTheory MeasureTheory Filter Topology
open scoped ENNReal

@[expose] public section

namespace ProbabilityTheory

variable {Ω : Type*} [MeasureSpace Ω] [IsProbabilityMeasure (ℙ : Measure Ω)]
variable (X : ℕ → Ω → ℝ)
variable (h_indep : iIndepFun X ℙ)
variable (h_ident : ∀ n, IdentDistrib (X n) (X 0) ℙ ℙ)
variable (h_meas : ∀ n, Measurable (X n))
variable (h_int : Integrable (X 0) ℙ)
variable (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * X 0 ω)) ℙ)
variable (h_bdd : ∀ a : ℝ, BddAbove (Set.range (fun t => t * a - cgf (X 0) ℙ t)))

/-! ### Helper lemmas for the upper bound proof -/

/-- For a nonempty bounded-below set in ℝ, the infimum of its elements coerced to EReal
equals the coercion of its infimum. -/
private lemma ereal_sInf_coe_eq_coe_sInf {s : Set ℝ} (hne : s.Nonempty) (hbdd : BddBelow s) :
    sInf ((fun (x : ℝ) => (x : EReal)) '' s) = ↑(sInf s) := by
  have h_bdd : BddBelow ((fun (x : ℝ) => (x : WithTop ℝ)) '' s) :=
    Monotone.map_bddBelow (fun _ _ h => WithTop.coe_le_coe.mpr h) hbdd
  rw [show (fun (x : ℝ) => (x : EReal)) '' s
        = (fun (y : WithTop ℝ) => (y : WithBot (WithTop ℝ))) ''
          ((fun (x : ℝ) => (x : WithTop ℝ)) '' s) from
      (Set.image_image _ (fun x : ℝ => (x : WithTop ℝ)) s).symm]
  exact (WithBot.coe_sInf' h_bdd).symm.trans (by rw [← WithTop.coe_sInf' hne hbdd]; rfl)
```

```
/-- For a bounded nonempty set in ℝ, the infimum of negations equals the negation of the supremum,
    stated in terms of coercions to EReal: `inf {↑-x | x ∈ S} = -sup {↑x | x ∈ S}` -/
private lemma ereal_sInf_neg_eq_neg_sSup {ι : Type*} (f : ι → ℝ)
    (hne : (Set.range f).Nonempty) (hbdd : BddAbove (Set.range f)) :
    sInf (Set.range fun i => (-(f i) : EReal)) = -((sSup (Set.range f) : ℝ) : EReal) := by
  obtain ⟨_, i₀, _⟩ := hne
  obtain ⟨B, hB⟩ := hbdd
  have h_ne_neg : (Set.range fun i => -f i).Nonempty := ⟨-f i₀, i₀, rfl⟩
  have h_bdd_neg : BddBelow (Set.range fun i => -f i) :=
    ⟨-B, fun _ ⟨i, hi⟩ => hi ▸ neg_le_neg (hB ⟨i, rfl⟩)⟩
  rw [show Set.range (fun i => (-(f i) : EReal)) =
        ((↑) : ℝ → EReal) '' Set.range (fun i => -f i) by
      rw [← Set.range_comp]; simp [Function.comp_def],
    ereal_sInf_coe_eq_coe_sInf h_ne_neg h_bdd_neg,
    show sInf (Set.range fun i => -f i) = -sSup (Set.range f) by
      rw [← Real.sInf_neg, ← Set.image_neg_eq_neg, ← Set.range_comp]; rfl,
    EReal.coe_neg]

include h_indep h_meas h_ident h_mgf in
/-- **Chernoff bound** for the empirical mean.
`ℙ(S_n/n ≥ a) ≤ exp(-n · (t · a - Λ_X₀(t)))` for `t ≥ 0`. -/
lemma prob_mean_ge_le_exp (t a : ℝ) (ht : 0 ≤ t) (n : ℕ) (hn_pos : 0 < n) :
  (ℙ {ω | empiricalMean X n ω ≥ a}).toReal
    ≤ Real.exp ( - (n : ℝ) * (t * a - cgf (X 0) ℙ t)) := by
  have h_n_pos : (0 : ℝ) < n := Nat.cast_pos.mpr hn_pos
  rw [show { ω | empiricalMean X n ω ≥ a } = { ω | partialSum X n ω ≥ (n : ℝ) * a } from by
    ext ω; simp [empiricalMean, ge_iff_le, le_div_iff₀ h_n_pos, mul_comm]]
  refine (measure_ge_le_exp_cgf _ ht
    (integrable_exp_sum X h_indep h_ident h_meas h_mgf t n)).trans ?_
  rw [cgf_sum_eq_n_prod_cgf X h_indep h_ident h_meas h_mgf n t]
  apply le_of_eq; congr 1; ring
```

```
include h_indep h_meas h_ident h_mgf h_bdd h_int in
/-- **Cramér's Theorem (Upper Bound)**: For any a ≥ 𝔼[X 0], the scaled log probability that the
empirical mean exceeds a is bounded above by the negative rate function:
‘limsup_{n→∞} log(ℙ(S_n/n ≥ a)) / n ≤ -I(a)‘
Uses ‘ENNReal.log‘ to properly handle the case when probability is 0 (giving -∞). -/
theorem cramer_upper_bound (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) :
    limsup (fun n : ℕ =>
      ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a}))
        atTop ≤ (- I X a : EReal) := by
  unfold I
  set L := limsup (fun n : ℕ =>
      ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop
  set f : {x : ℝ | 0 ≤ x} → ℝ := fun t => t.val * a - cgf (X 0) ℙ t
  have h0 : (0 : ℝ) ∈ {x : ℝ | 0 ≤ x} := by simp
  haveI : Nonempty {x : ℝ | 0 ≤ x} := ⟨⟨0, h0⟩⟩
  suffices h : ∀ t : ℝ, 0 ≤ t → L ≤ (-(t * a - cgf (X 0) ℙ t) : EReal) by
    calc L
        ≤ sInf (Set.range fun t : {x : ℝ | 0 ≤ x} => (-(f t) : EReal)) :=
          le_csInf (Set.range_nonempty _) <| by
            rintro b ⟨t, rfl⟩; exact h t.val t.property
      _ = (-(⨆ t : {x : ℝ | 0 ≤ x}, f t) : EReal) := by
          have h_bdd' : BddAbove (Set.range f) :=
            let ⟨b, hb⟩ := h_bdd a
            ⟨b, fun _ ⟨t, ht⟩ => ht ▸ hb ⟨t.val, rfl⟩⟩
          simpa [iSup] using ereal_sInf_neg_eq_neg_sSup f ⟨_, ⟨0, h0⟩, rfl⟩ h_bdd'
      _ = (- I X a : EReal) := by
          norm_cast
          exact congrArg Neg.neg (rateFunction_eq_sup_nonneg X h_int h_mgf h_bdd a h_mean).symm
  intro t ht
  refine limsup_le_of_le (isCoboundedUnder_le_of_le atTop (fun _ => bot_le))
    (eventually_atTop.mpr ⟨1, fun n hn => ?_⟩)
  have hn_pos : 0 < n := hn
  have hn_ne : (n : ℝ) ≠ 0 := Nat.cast_ne_zero.mpr hn_pos.ne'
  have h_ennreal : ℙ {ω | empiricalMean X n ω ≥ a} ≤
      ENNReal.ofReal (Real.exp (-(n : ℝ) * (t * a - cgf (X 0) ℙ t))) :=
    (ENNReal.ofReal_toReal_eq_iff.mpr (measure_ne_top _ _)).symm.le.trans
      (ENNReal.ofReal_le_ofReal
        (prob_mean_ge_le_exp X h_indep h_ident h_meas h_mgf t a ht n hn_pos))
  have h_log_exp : ENNReal.log (ENNReal.ofReal (Real.exp (-(n : ℝ) * (t * a - cgf (X 0) ℙ t))))
      = (((-(n : ℝ) * (t * a - cgf (X 0) ℙ t)) : ℝ) : EReal) := by
    rw [ENNReal.log_ofReal_of_pos (Real.exp_pos _), Real.log_exp]
  have h_arith : (1 : ℝ) / (n : ℝ) * (-(n : ℝ) * (t * a - cgf (X 0) ℙ t))
      = -(t * a - cgf (X 0) ℙ t) := by field_simp
  calc ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})
      ≤ ((1 : ℝ) / (n : ℝ) : EReal) *
          (((-(n : ℝ) * (t * a - cgf (X 0) ℙ t)) : ℝ) : EReal) := by
        rw [← h_log_exp]; gcongr
    _ = (-(t * a - cgf (X 0) ℙ t) : EReal) := by
        simp only [← EReal.coe_mul]; exact congrArg (fun x : ℝ => (x : EReal)) h_arith

end ProbabilityTheory
```

A.4 Mathlib/Probability/LargeDeviations/Cramers/TiltedCLT.lean

```
module

public import Mathlib.Probability.LargeDeviations.Cramers.Basic
public import Mathlib.Probability.CentralLimitTheorem
public import Mathlib.Probability.Independence.CharacteristicFunction
public import Mathlib.MeasureTheory.Measure.LevyConvergence
public import Mathlib.MeasureTheory.Measure.Portmanteau

/-!
# Cramér's Theorem - CLT over Tilted Measures

This file provides a proof of the Central Limit Theorem over a sequence of tilted measures,
used in the proof of the lower bound of Cramér's theorem.

Under the assumptions of Cramér's theorem, for `t` with `Λ'(t) = a` the family of tilted measures
`ℚ_nt := ℙ.tilted (t · S_n)` satisfies a CLT statement:
The normalized partial sum `Z_n := (S_n - n Λ'(t))/√(n Λ''(t))` converges in distribution to
    `𝒩(0,1)`
under `ℚ_nt` as `n → ∞`.
An immediate corollary is a concentration statement used in `LowerBound.lean`.

## Main Definitions

* `μ_t X t`: the pushforward measure of `ℙ` by `X_0`, tilted by `t · x`,
* `ν_t X t`: the pushforward measure of `μ_t X t` by a linear function to standardize it to zero mean
  and unit variance.
* `Z X t n`: the standardized partial sum `(S_n - n Λ'(t)) / √(n Λ''(t))`.

## Main Results

* `tendsto_charFun_Z_ℚ_nt`: under `ℚ_nt`, the characteristic function of `Z_n` converges pointwise to
  that of `𝒩(0,1)`.
* `eventually_ℚ_nt_empiricalMean_mem_Icc_ge`: the concentration statement for the lower bound proof:
  For every `δ, ε > 0` and `t` with `Λ'(t) = a`, eventually `1/2 - ε ≤ ℚ_nt(S_n/n ∈ [a, a+δ])`.

-/

open ProbabilityTheory MeasureTheory Filter Topology
open scoped ENNReal NNReal

@[expose] public section

namespace ProbabilityTheory

variable {Ω : Type*} [MeasureSpace Ω]

/-- The law of `X 0` under `ℙ`, tilted by `t · x`. -/
noncomputable def μ_t (X : ℕ → Ω → ℝ) (t : ℝ) : Measure ℝ :=
  ((ℙ : Measure Ω).map (X 0)).tilted (fun x => t * x)
```

```
/-- The push-forward of ‘μ_t‘ by the map ‘x ↦ (x - Λ'(t)) / √Λ''(t)‘,
  so that it has zero mean and unit variance. -/
noncomputable def ν_t (X : ℕ → Ω → ℝ) (t : ℝ) : Measure ℝ :=
  (μ_t X t).map
    (fun x => (x - deriv (cgf (X 0) ℙ) t) / Real.sqrt (iteratedDeriv 2 (cgf (X 0) ℙ) t))

/-- The standardized partial sum under ‘ℚ_nt‘: ‘Z = (S_n - n Λ'(t)) / √(n Λ''(t))‘. -/
noncomputable def Z (X : ℕ → Ω → ℝ) (t : ℝ) (n : ℕ) : Ω → ℝ := fun ω =>
  (partialSum X n ω - n * deriv (cgf (X 0) ℙ) t) /
    Real.sqrt (n * iteratedDeriv 2 (cgf (X 0) ℙ) t)

variable (X : ℕ → Ω → ℝ)

/- Assumptions for Cramér's theorem -/
variable (h_indep : iIndepFun X ℙ)
variable (h_ident : ∀ n, IdentDistrib (X n) (X 0) ℙ ℙ)
variable (h_meas : ∀ n, Measurable (X n))
variable (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * X 0 ω)) ℙ)
variable (h_non_deg : ∀ t : ℝ, 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)

include h_meas in
/-- The MGF of the identity under ‘μ_t‘ at ‘u‘ equals ‘mgf(X_0)(t + u)/mgf(X_0)(t)‘. -/
lemma mgf_id_tiltedLaw (t u : ℝ) :
    mgf id (μ_t X t) u = mgf (X 0) ℙ (t + u) / mgf (X 0) ℙ t := by
  have hX0 : AEMeasurable (X 0) ℙ := (h_meas 0).aemeasurable
  simp only [mgf, μ_t, id]
  rw [integral_exp_tilted (fun x => t * x) (fun x => u * x),
    integral_map hX0 (by fun_prop), integral_map hX0 (by fun_prop)]
  simp [add_mul]

include h_meas h_mgf in
private lemma t_mem_interior_integrableExpSet_id (t : ℝ) :
    t ∈ interior (integrableExpSet id ((ℙ : Measure Ω).map (X 0))) := by
  have h_univ : integrableExpSet id ((ℙ : Measure Ω).map (X 0)) = Set.univ := by
    ext s
    simp only [integrableExpSet, Set.mem_setOf_eq, Set.mem_univ, iff_true, id]
    exact (integrable_map_measure (by fun_prop) (h_meas 0).aemeasurable).mpr (h_mgf s)
  rw [h_univ, interior_univ]
  exact Set.mem_univ t

include h_meas in
private lemma cgf_id_map_eq : cgf id ((ℙ : Measure Ω).map (X 0)) = cgf (X 0) ℙ := by
  ext s; simp [cgf, mgf_id_map (h_meas 0).aemeasurable]

private lemma t_mul_eq_mul_id (t : ℝ) : (fun x : ℝ => t * x) = (t * id ·) := rfl

include h_meas h_mgf in
/-- The mean of ‘μ_t‘ is ‘Λ'(t)‘. -/
lemma integral_id_tiltedLaw (t : ℝ) : ∫ x, x ∂(μ_t X t) = deriv (cgf (X 0) ℙ) t := by
  simp only [μ_t]
  rw [t_mul_eq_mul_id, ← cgf_id_map_eq X h_meas]
  exact integral_tilted_mul_self (t_mem_interior_integrableExpSet_id X h_meas h_mgf t)
```

```
include h_meas h_mgf in
/-- The variance of `μ_t` is `Λ''(t)`. -/
lemma variance_id_tiltedLaw (t : ℝ) :
    Var[id; μ_t X t] = iteratedDeriv 2 (cgf (X 0) ℙ) t := by
  simp only [μ_t]
  rw [t_mul_eq_mul_id, ← cgf_id_map_eq X h_meas]
  exact variance_tilted_mul (t_mem_interior_integrableExpSet_id X h_meas h_mgf t)

include h_meas h_mgf in
/-- The identity is in `L^2` under `μ_t`. -/
lemma memLp_id_tiltedLaw (t : ℝ) : MemLp (id : ℝ → ℝ) 2 (μ_t X t) := by
  simp only [μ_t]
  rw [t_mul_eq_mul_id]
  exact memLp_tilted_mul (t_mem_interior_integrableExpSet_id X h_meas h_mgf t) 2

/-! ### Algebraic Identities -/

/-- Algebraic identity: for `t` with `Λ'(t) = a` and `n ≥ 1`, the event
`{S_n/n ∈ [a, a+δ]}` is exactly `{Z_n ∈ [0, δ · √(n / Λ''(t))]}`. -/
lemma empiricalMean_mem_Icc_iff_Z_mem_Icc (t a δ : ℝ) (n : ℕ) (hn : 0 < n)
    (h_non_deg_t : 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) (ω : Ω) :
    empiricalMean X n ω ∈ Set.Icc a (a + δ) ↔
      Z X t n ω ∈ Set.Icc (0 : ℝ)
        (δ * Real.sqrt (n / iteratedDeriv 2 (cgf (X 0) ℙ) t)) := by
  rw [Set.mem_Icc, Set.mem_Icc, empiricalMean, Z, ht_deriv]
  set v := iteratedDeriv 2 (cgf (X 0) ℙ) t
  have hn' : (0 : ℝ) < n := by exact_mod_cast hn
  have hnv : (0 : ℝ) < Real.sqrt (n * v) := Real.sqrt_pos.mpr (by positivity)
  rw [le_div_iff₀ hn', div_le_iff₀ hn', le_div_iff₀ hnv, div_le_iff₀ hnv]
  have key : Real.sqrt (n / v) * Real.sqrt (n * v) = n := by
    rw [← Real.sqrt_mul (by positivity), show (n / v * (n * v) : ℝ) = n ^ 2 by field_simp,
      Real.sqrt_sq hn'.le]
  refine ⟨fun ⟨h1, h2⟩ => ⟨by linarith, ?_⟩, fun ⟨h1, h2⟩ => ⟨by linarith, ?_⟩⟩ <;> nlinarith [key]

include h_meas h_mgf h_non_deg in
/-- The second moment of `ν_t` is `1`. -/
lemma integral_sq_id_stdTiltedLaw (t : ℝ) :
    ∫ x, x ^ 2 ∂(ν_t X t) = 1 := by
  have hv := h_non_deg t
  set m := deriv (cgf (X 0) ℙ) t
  set v := iteratedDeriv 2 (cgf (X 0) ℙ) t
  simp only [ν_t]
  rw [integral_map (by fun_prop) (by fun_prop)]
  have hμ_mean : ∫ x, x ∂μ_t X t = m :=
    integral_id_tiltedLaw X h_meas h_mgf t
  have hvar : Var[id; μ_t X t] = v :=
    variance_id_tiltedLaw X h_meas h_mgf t
  have hint_sq : ∫ x, (x - m) ^ 2 ∂μ_t X t = v := by
    rw [← hvar, variance_eq_integral aemeasurable_id]
    congr 1; ext x; simp [hμ_mean]
  rw [show (fun x : ℝ => ((x - m) / Real.sqrt v) ^ 2) = fun x => (x - m) ^ 2 / v from
    funext fun x => by rw [div_pow, Real.sq_sqrt hv.le], integral_div, hint_sq, div_self hv.ne']
```

```
/-! ### Lemmas requiring `ℙ` to be a probability measure -/

variable [IsProbabilityMeasure (ℙ : Measure Ω)]

include h_meas h_mgf in
/-- `μ_t` is a probability measure. -/
lemma isProbabilityMeasure_tiltedLaw (t : ℝ) :
    IsProbabilityMeasure (μ_t X t) := by
  haveI : IsProbabilityMeasure ((ℙ : Measure Ω).map (X 0)) :=
    Measure.isProbabilityMeasure_map (h_meas 0).aemeasurable
  exact isProbabilityMeasure_tilted
    ((integrable_map_measure (by fun_prop) (h_meas 0).aemeasurable).mpr (h_mgf t))

/-! ### Properties of `ν_t` -/

include h_meas h_mgf in
/-- `ν_t` is a probability measure. -/
lemma isProbabilityMeasure_stdTiltedLaw (t : ℝ) :
    IsProbabilityMeasure (ν_t X t) :=
  haveI := isProbabilityMeasure_tiltedLaw X h_meas h_mgf t
  Measure.isProbabilityMeasure_map (by fun_prop)

include h_meas h_mgf in
/-- The mean of `ν_t` is `0`. -/
lemma integral_id_stdTiltedLaw (t : ℝ) :
    ∫ x, x ∂(ν_t X t) = 0 := by
  simp only [ν_t]
  haveI : IsProbabilityMeasure (μ_t X t) :=
    isProbabilityMeasure_tiltedLaw X h_meas h_mgf t
  have hid : Integrable (fun x : ℝ => x) (μ_t X t) :=
    (memLp_id_tiltedLaw X h_meas h_mgf t).integrable (by norm_num)
  rw [integral_map (by fun_prop) (by fun_prop), integral_div,
    integral_sub hid (integrable_const _), integral_const, probReal_univ, one_smul,
    integral_id_tiltedLaw X h_meas h_mgf t, sub_self, zero_div]

include h_meas h_mgf in
/-- The identity is in `L^2` under `ν_t`. -/
lemma memLp_id_stdTiltedLaw (t : ℝ) :
    MemLp (id : ℝ → ℝ) 2 (ν_t X t) := by
  simp only [ν_t]
  rw [memLp_map_measure_iff (by fun_prop) (by fun_prop)]
  haveI : IsProbabilityMeasure (μ_t X t) :=
    isProbabilityMeasure_tiltedLaw X h_meas h_mgf t
  have h_sub : MemLp (fun x : ℝ => x - deriv (cgf (X 0) ℙ) t) 2 (μ_t X t) :=
    (memLp_id_tiltedLaw X h_meas h_mgf t).sub (memLp_const _)
  have h_div : MemLp (fun x : ℝ => (x - deriv (cgf (X 0) ℙ) t) *
      (Real.sqrt (iteratedDeriv 2 (cgf (X 0) ℙ) t))⁻¹) 2 (μ_t X t) :=
    h_sub.mul_const _
  exact h_div.congr_norm (by fun_prop) (Eventually.of_forall fun x => by
    simp [Function.comp, div_eq_mul_inv])
```

```
/-! ### Factorization of ‘X₀, ..., X_{n-1}‘ under ‘ℚ_{nt}‘

Under the tilted measure ‘ℚ_{nt} = ℙ.tilted(t · S_n)‘, the coordinates ‘X₀, ..., X_{n-1}‘ are
still independent and each has law ‘μ_t‘.
These follow from factoring the tilting density ‘exp(t S_n) = ∏_i exp(t X_i)‘. -/

private lemma measurable_ennreal_ofReal_exp_t_mul (t : ℝ) :
    Measurable (fun x : ℝ => ENNReal.ofReal (Real.exp (t * x))) :=
  (measurable_id.const_mul t |>.exp).ennreal_ofReal

omit [MeasureSpace Ω] [IsProbabilityMeasure (ℙ : Measure Ω)] in
/-- ‘↑e^(t · S_n) = ∏_i ↑e^(t X_i)‘ where ‘↑‘ is coercion to extended reals. -/
private lemma ennreal_ofReal_exp_t_partialSum_eq_prod (t : ℝ) (n : ℕ) (ω : Ω) :
    ENNReal.ofReal (Real.exp (t * partialSum X n ω)) =
    ∏ j ∈ Finset.range n, ENNReal.ofReal (Real.exp (t * X j ω)) := by
  rw [show Real.exp (t * partialSum X n ω) = ∏ j ∈ Finset.range n, Real.exp (t * X j ω) by
    simp [partialSum, Finset.sum_apply, Finset.mul_sum, Real.exp_sum],
    ENNReal.ofReal_prod_of_nonneg (fun _ _ => (Real.exp_pos _).le)]

omit [IsProbabilityMeasure (ℙ : Measure Ω)] in
include h_meas in
/-- ‘↑e^(t X_i)‘ is measurable for each ‘i‘. -/
private lemma measurable_ennreal_ofReal_exp_t_mul_X (t : ℝ) (j : ℕ) :
    Measurable (fun ω => ENNReal.ofReal (Real.exp (t * X j ω))) :=
  (measurable_ennreal_ofReal_exp_t_mul t).comp (h_meas j)

omit [IsProbabilityMeasure (ℙ : Measure Ω)] in
include h_indep in
/-- The family ‘↑e^(t X_i)‘ is independent. -/
private lemma iIndepFun_ennreal_ofReal_exp_t_mul_X (t : ℝ) :
    iIndepFun (fun j ω => ENNReal.ofReal (Real.exp (t * X j ω))) ℙ :=
  h_indep.comp (fun _ x => ENNReal.ofReal (Real.exp (t * x)))
    (fun _ => measurable_ennreal_ofReal_exp_t_mul t)

omit [IsProbabilityMeasure (ℙ : Measure Ω)] in
include h_ident h_mgf in
/-- The integral of ‘↑e^(t X_i)‘ against ‘ℙ‘ is ‘↑ mgf(X 0)(t)‘. -/
private lemma lintegral_ennreal_ofReal_exp_t_mul_X (t : ℝ) (j : ℕ) :
    ∫⁻ ω, ENNReal.ofReal (Real.exp (t * X j ω)) ∂ℙ = ENNReal.ofReal (mgf (X 0) ℙ t) :=
  ((h_ident j).comp (measurable_ennreal_ofReal_exp_t_mul t)).lintegral_eq.trans
    (ofReal_integral_eq_lintegral_ofReal (h_mgf t)
      (ae_of_all _ fun _ => (Real.exp_pos _).le)).symm

include h_indep h_ident h_meas h_mgf in
/-- ‘MGF(X₁)‘ and ‘MGF(S_n)‘ are positive and ‘↑MGF(S_n) = (↑MGF(X₁))^n‘ -/
private lemma mgf_preamble (n : ℕ) (t : ℝ) :
    0 < mgf (X 0) ℙ t ∧
    0 < mgf (partialSum X n) ℙ t ∧
    ENNReal.ofReal (mgf (partialSum X n) ℙ t) = (ENNReal.ofReal (mgf (X 0) ℙ t)) ^ n := by
  have hM₀_pos : 0 < mgf (X 0) ℙ t := integral_exp_pos (h_mgf t)
  refine ⟨hM₀_pos,
    integral_exp_pos (integrable_exp_sum X h_indep h_ident h_meas h_mgf t n), ?_⟩
  rw [show mgf (partialSum X n) ℙ t = (mgf (X 0) ℙ t) ^ n by
      rw [mgf_sum_eq_exp_n_prod_cgf X h_indep h_ident h_meas h_mgf n t, cgf, ← Real.log_pow,
        Real.exp_log (pow_pos hM₀_pos n)],
    ENNReal.ofReal_pow hM₀_pos.le]
```

```
include h_indep h_ident h_meas h_mgf in
/-- Under `ℚ_nt`, each `X_i` (for `i < n`) has the same law `μ_t`. -/
lemma map_X_ℚ_nt (t : ℝ) (n : ℕ) {i : ℕ} (hi : i < n) :
    (ℚ_nt X ℙ n t).map (X i) = μ_t X t := by
  obtain ⟨hM₀_pos, hMₙ_pos, hMₙ_pow⟩ := mgf_preamble X h_indep h_ident h_meas h_mgf n t
  have hi_mem : i ∈ Finset.range n := Finset.mem_range.mpr hi
  have hn_pos : 0 < n := Nat.lt_of_le_of_lt (Nat.zero_le _) hi
  set M₀ := mgf (X 0) ℙ t with hM₀_def
  set Mₙ := mgf (partialSum X n) ℙ t with hMₙ_def
  have hM₀_nn : ENNReal.ofReal M₀ ≠ 0 := (ENNReal.ofReal_pos.mpr hM₀_pos).ne'
  have hMₙ_nn : ENNReal.ofReal Mₙ ≠ 0 := (ENNReal.ofReal_pos.mpr hMₙ_pos).ne'
  have hMₙ_inv_ne_top : (ENNReal.ofReal Mₙ)⁻¹ ≠ ∞ := ENNReal.inv_ne_top.mpr hMₙ_nn
  have hM₀_inv_ne_top : (ENNReal.ofReal M₀)⁻¹ ≠ ∞ := ENNReal.inv_ne_top.mpr hM₀_nn
  have hk_meas : Measurable (fun x : ℝ => ENNReal.ofReal (Real.exp (t * x))) :=
    measurable_ennreal_ofReal_exp_t_mul t
  set e : ℕ → Ω → ℝ≥0∞ := fun j ω => ENNReal.ofReal (Real.exp (t * X j ω))
  have he_meas := measurable_ennreal_ofReal_exp_t_mul_X X h_meas t
  have he_indep := iIndepFun_ennreal_ofReal_exp_t_mul_X X h_indep t
  have he_lint := lintegral_ennreal_ofReal_exp_t_mul_X X h_ident h_mgf t
  apply Measure.ext_of_lintegral
  intro g hg
  have hg_exp : Measurable (fun x : ℝ => g x * ENNReal.ofReal (Real.exp (t * x))) :=
    hg.mul hk_meas
  set I₀ : ℝ≥0∞ := ∫⁻ ω, g (X 0 ω) * e 0 ω ∂ℙ with hI₀_def
  have hI_eq : (∫⁻ ω, g (X i ω) * e i ω ∂ℙ) = I₀ :=
    ((h_ident i).comp hg_exp).lintegral_eq
  have hmeas_gxe : Measurable (fun ω : Ω => g (X i ω) * e i ω) :=
    (hg.comp (h_meas i)).mul (he_meas i)
  have hmeas_prod : Measurable fun ω => ∏ j ∈ (Finset.range n).erase i, e j ω :=
    Finset.measurable_prod _ fun j _ => he_meas j
  set ψ : ℕ → ℝ → ℝ≥0∞ := fun j x =>
    if j = i then g x * ENNReal.ofReal (Real.exp (t * x))
    else ENNReal.ofReal (Real.exp (t * x))
  have hψ_meas : ∀ j, Measurable (ψ j) := fun j => by
    simp only [ψ]; split_ifs; exacts [hg_exp, hk_meas]
  have hindep : IndepFun (fun ω => g (X i ω) * e i ω)
      (fun ω => ∏ j ∈ (Finset.range n).erase i, e j ω) ℙ := by
    have h : IndepFun (∏ j ∈ (Finset.range n).erase i, (ψ j ∘ X j)) (ψ i ∘ X i) ℙ :=
      (h_indep.comp ψ hψ_meas).indepFun_finset_prod_of_notMem
        (fun j => (hψ_meas j).comp (h_meas j)) (Finset.notMem_erase i _)
    convert h.symm using 1
    · ext ω; simp [ψ, e]
    · ext ω; rw [Finset.prod_apply]
      exact Finset.prod_congr rfl fun j hj => by simp [ψ, e, Finset.ne_of_mem_erase hj]
  rw [lintegral_map hg (h_meas i)]
  simp only [ℚ_nt, μ_t, lintegral_tilted]
  have hsum_prod : ∀ ω, ENNReal.ofReal (Real.exp (t * partialSum X n ω) /
      ∫ x, Real.exp (t * partialSum X n x) ∂ℙ) * g (X i ω) =
      (g (X i ω) * e i ω) * (∏ j ∈ (Finset.range n).erase i, e j ω) *
        (ENNReal.ofReal Mₙ)⁻¹ := fun ω => by
    rw [show (∫ x, Real.exp (t * partialSum X n x) ∂ℙ) = Mₙ from rfl,
      ENNReal.ofReal_div_of_pos hMₙ_pos, div_eq_mul_inv,
      ennreal_ofReal_exp_t_partialSum_eq_prod X t n ω,
      ← Finset.mul_prod_erase _ (e · ω) hi_mem]
    ring
```

```
simp_rw [hsum_prod]
rw [lintegral_mul_const' _ _ hM_n_inv_ne_top,
  lintegral_mul_eq_lintegral_mul_lintegral_of_indepFun''
    hmeas_gxe.aemeasurable hmeas_prod.aemeasurable hindep,
  lintegral_prod_eq_prod_lintegral_of_indepFun _ _ he_indep he_meas,
  Finset.prod_congr rfl (fun j _ => he_lint j), Finset.prod_const,
  Finset.card_erase_of_mem hi_mem, Finset.card_range, hI_eq, hM_n_pow]
have hmap_mgf : (∫ x : ℝ, Real.exp (t * x) ∂Measure.map (X 0) ℙ) = M_0 :=
  integral_map (h_meas 0).aemeasurable ((measurable_id.const_mul t).exp).aestronglyMeasurable
conv_rhs => rw [show (fun x : ℝ => ENNReal.ofReal (Real.exp (t * x) /
    ∫ y, Real.exp (t * y) ∂Measure.map (X 0) ℙ) * g x) = fun x =>
  (g x * ENNReal.ofReal (Real.exp (t * x))) * (ENNReal.ofReal M_0)⁻¹ from funext fun x => by
    rw [hmap_mgf, ENNReal.ofReal_div_of_pos hM_0_pos, div_eq_mul_inv]; ring]
rw [lintegral_mul_const' _ _ hM_0_inv_ne_top, lintegral_map hg_exp (h_meas 0), ← hI_0_def]
rw [show (ENNReal.ofReal M_0) ^ n = (ENNReal.ofReal M_0) ^ (n - 1) * ENNReal.ofReal M_0 by
    conv_lhs => rw [show n = (n - 1) + 1 from (Nat.sub_add_cancel hn_pos).symm, pow_succ],
  ENNReal.mul_inv (Or.inl (pow_ne_zero _ hM_0_nn))
    (Or.inl (ENNReal.pow_ne_top ENNReal.ofReal_lt_top.ne)),
  mul_assoc, ← mul_assoc ((ENNReal.ofReal M_0) ^ (n - 1)),
  ENNReal.mul_inv_cancel (pow_ne_zero _ hM_0_nn) (ENNReal.pow_ne_top ENNReal.ofReal_lt_top.ne),
  one_mul]
```

```
include h_indep h_ident h_meas h_mgf in
/-- Under `ℚ_nt`, `X_i` (for `i < n`) are independent. -/
lemma iIndepFun_range_ℚ_nt (t : ℝ) (n : ℕ) :
    iIndepFun (fun i : Finset.range n => X i.val) (ℚ_nt X ℙ n t) := by
  classical
  obtain ⟨hM₀_pos, hMₙ_pos, hMₙ_pow⟩ := mgf_preamble X h_indep h_ident h_meas h_mgf n t
  set M₀ := mgf (X 0) ℙ t with hM₀_def
  set Mₙ := mgf (partialSum X n) ℙ t with hMₙ_def
  have hM₀_nn : ENNReal.ofReal M₀ ≠ 0 := (ENNReal.ofReal_pos.mpr hM₀_pos).ne'
  have hM₀_top : ENNReal.ofReal M₀ ≠ ∞ := ENNReal.ofReal_lt_top.ne
  have hMₙ_nn : ENNReal.ofReal Mₙ ≠ 0 := (ENNReal.ofReal_pos.mpr hMₙ_pos).ne'
  have hMₙ_inv_ne_top : (ENNReal.ofReal Mₙ)⁻¹ ≠ ∞ := ENNReal.inv_ne_top.mpr hMₙ_nn
  set e : ℕ → Ω → ℝ≥0∞ := fun j ω => ENNReal.ofReal (Real.exp (t * X j ω)) with he_def
  have he_meas := measurable_ennreal_ofReal_exp_t_mul_X X h_meas t
  have he_indep := iIndepFun_ennreal_ofReal_exp_t_mul_X X h_indep t
  have he_lint := lintegral_ennreal_ofReal_exp_t_mul_X X h_ident h_mgf t
  have hsum_eq : ∀ ω,
      ENNReal.ofReal (Real.exp (t * partialSum X n ω)) =
      ∏ j ∈ Finset.range n, e j ω :=
    fun ω => ennreal_ofReal_exp_t_partialSum_eq_prod X t n ω
  rw [iIndepFun_iff_measure_inter_preimage_eq_mul]
  intro S sets hsets
  let A' : ℕ → Set ℝ := fun j =>
    if hj : j < n then
      if h : ⟨j, Finset.mem_range.mpr hj⟩ ∈ S then sets ⟨j, Finset.mem_range.mpr hj⟩
      else Set.univ
    else Set.univ
  have hA'_meas : ∀ j, MeasurableSet (A' j) := fun j => by
    simp only [A']; split_ifs with hj hjS
    exacts [hsets _ hjS, MeasurableSet.univ, MeasurableSet.univ]
  have hA'_inS : ∀ (i : Finset.range n), i ∈ S → A' (i : ℕ) = sets i := by
    intro i hi
    have hi_lt : (i : ℕ) < n := Finset.mem_range.mp i.2
    have heq : (⟨(i : ℕ), Finset.mem_range.mpr hi_lt⟩ : Finset.range n) = i := Subtype.ext rfl
    simp only [A', dif_pos hi_lt]
    rw [dif_pos (by rw [heq]; exact hi), heq]
  let φ : ℕ → ℝ → ℝ≥0∞ := fun j x =>
    (A' j).indicator (fun _ => (1 : ℝ≥0∞)) x * ENNReal.ofReal (Real.exp (t * x))
  have hφ_meas : ∀ j, Measurable (φ j) := fun j =>
    (measurable_const.indicator (hA'_meas j)).mul (measurable_ennreal_ofReal_exp_t_mul t)
  have hφ_comp_indep : iIndepFun (fun j => φ j ∘ X j) ℙ := h_indep.comp φ hφ_meas
  have hφ_meas' : ∀ j, Measurable (fun ω => φ j (X j ω)) :=
    fun j => (hφ_meas j).comp (h_meas j)
  have hφ_lint_not_inS : ∀ j, (∀ hj : j < n, ⟨j, Finset.mem_range.mpr hj⟩ ∉ S) →
      ∫⁻ ω, φ j (X j ω) ∂ℙ = ENNReal.ofReal M₀ := by
    intro j hj_notmem
    refine (lintegral_congr fun ω => ?_).trans (he_lint j)
    simp only [φ, A']; split_ifs with hj hjS
    · exact absurd hjS (hj_notmem hj)
    · rw [Set.indicator_univ]; simp
    · rw [Set.indicator_univ]; simp
```

```
have hprod_indicator : ∀ ω,
    ∏ j ∈ Finset.range n, φ j (X j ω) =
    (⋂ i ∈ S, (fun ω' => X (i : ℕ) ω') ⁻¹' sets i).indicator (fun _ => (1 : ℝ≥0∞)) ω *
      ∏ j ∈ Finset.range n, e j ω := by
  intro ω
  by_cases hω : ω ∈ ⋂ i ∈ S, (fun ω' => X (i : ℕ) ω') ⁻¹' sets i
  · rw [Set.indicator_of_mem hω, one_mul]
    refine Finset.prod_congr rfl fun j hj => ?_
    have hjn : j < n := Finset.mem_range.mp hj
    change (A' j).indicator _ (X j ω) * _ = e j ω
    by_cases hjS : (⟨j, Finset.mem_range.mpr hjn⟩ : Finset.range n) ∈ S
    · rw [hA'_inS ⟨j, Finset.mem_range.mpr hjn⟩ hjS]
      have hmem : X j ω ∈ sets ⟨j, Finset.mem_range.mpr hjn⟩ :=
        Set.mem_iInter₂.mp hω _ hjS
      rw [Set.indicator_of_mem hmem, one_mul]
    · have hA'_univ : A' j = Set.univ := by
        simp only [A', dif_pos hjn, dif_neg hjS]
      rw [hA'_univ, Set.indicator_univ, one_mul]
  · rw [Set.indicator_of_notMem hω, zero_mul]
    have : ∃ i ∈ S, X (i : ℕ) ω ∉ sets i := by
      by_contra! h_all
      exact hω (Set.mem_iInter₂.mpr h_all)
    obtain ⟨i, hiS, hi⟩ := this
    apply Finset.prod_eq_zero (i.2 : (i : ℕ) ∈ Finset.range n)
    change (A' (i : ℕ)).indicator _ (X (i : ℕ) ω) * _ = 0
    rw [hA'_inS i hiS, Set.indicator_of_notMem hi, zero_mul]
have hIcap_meas : MeasurableSet
    (⋂ i ∈ S, (fun ω => X (i : ℕ) ω) ⁻¹' sets i) :=
  .biInter (Set.to_countable _) fun i hi => (h_meas _) (hsets i hi)
have hfun_eq : ∀ ω,
    ENNReal.ofReal (Real.exp (t * partialSum X n ω) / M_n) =
    ENNReal.ofReal (Real.exp (t * partialSum X n ω)) * (ENNReal.ofReal M_n)⁻¹ := fun ω => by
  rw [ENNReal.ofReal_div_of_pos hM_n_pos, div_eq_mul_inv]
have hLHS :
    (ℚ_nt X ℙ n t) (⋂ i ∈ S, (fun ω => X i.val ω) ⁻¹' sets i) =
    (ENNReal.ofReal M_n)⁻¹ *
      ∫⁻ ω, ∏ j ∈ Finset.range n, φ j (X j ω) ∂ℙ := by
  simp only [ℚ_nt]
  rw [tilted_apply' _ _ hIcap_meas]
  change ∫⁻ ω in _, ENNReal.ofReal (Real.exp (t * partialSum X n ω) / M_n) ∂ℙ = _
  rw [setLIntegral_congr_fun hIcap_meas (fun ω _ => hfun_eq ω),
    lintegral_mul_const' _ _ hM_n_inv_ne_top, mul_comm, ← lintegral_indicator hIcap_meas]
  congr 1
  refine lintegral_congr fun ω => ?_
  rw [hprod_indicator ω]
  by_cases hω : ω ∈ ⋂ i ∈ S, (fun ω' => X (i : ℕ) ω') ⁻¹' sets i
  · rw [Set.indicator_of_mem hω, Set.indicator_of_mem hω, one_mul, hsum_eq ω]
  · rw [Set.indicator_of_notMem hω, Set.indicator_of_notMem hω, zero_mul]
have hLHS_factor :
    ∫⁻ ω, ∏ j ∈ Finset.range n, φ j (X j ω) ∂ℙ =
    ∏ j ∈ Finset.range n, ∫⁻ ω, φ j (X j ω) ∂ℙ :=
  lintegral_prod_eq_prod_lintegral_of_indepFun _ _ hφ_comp_indep hφ_meas'
have hA_i_meas : ∀ i ∈ S, MeasurableSet ((X (i : ℕ)) ⁻¹' sets i) := fun i hi =>
  (h_meas _) (hsets i hi)
```

```
have h_single : ∀ (i : Finset.range n) (hi : i ∈ S),
    (ℚ_{nt} X ℙ n t) ((X (i : ℕ)) ⁻¹' sets i) =
    (ENNReal.ofReal M_n)⁻¹ *
      (∫⁻ ω, (sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) *
        e (i : ℕ) ω ∂ℙ) *
      ∏ j ∈ (Finset.range n).erase (i : ℕ), ENNReal.ofReal M_0 := by
  intro i hi
  have hmeas_i : MeasurableSet ((X (i : ℕ)) ⁻¹' sets i) := hA_i_meas _ hi
  simp only [ℚ_{nt}]
  rw [tilted_apply' _ _ hmeas_i]
  change ∫⁻ ω in _, ENNReal.ofReal (Real.exp (t * partialSum X n ω) / M_n) ∂ℙ = _
  rw [setLIntegral_congr_fun hmeas_i (fun ω _ => hfun_eq ω),
    lintegral_mul_const' _ _ hM_n_inv_ne_top, mul_right_comm, ← lintegral_indicator hmeas_i]
  have hi_range : (i : ℕ) ∈ Finset.range n := i.2
  have hrewrite : ∀ ω,
      ((X (i : ℕ)) ⁻¹' sets i).indicator
        (fun a => ENNReal.ofReal (Real.exp (t * partialSum X n a))) ω =
      ((sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) * e (i : ℕ) ω) *
        ∏ j ∈ (Finset.range n).erase (i : ℕ), e j ω := by
    intro ω
    rw [Set.indicator]
    by_cases hω : X (i : ℕ) ω ∈ sets i
    · rw [hsum_eq ω]
      have hhω : ω ∈ (X (i : ℕ)) ⁻¹' sets i := hω
      simp only [hhω, if_true]
      rw [Set.indicator_of_mem hω]
      rw [← Finset.mul_prod_erase _ _ hi_range]
      ring
    · have hhω : ω ∉ (X (i : ℕ)) ⁻¹' sets i := hω
      simp only [hhω, if_false]
      rw [Set.indicator_of_notMem hω]
      simp
  rw [lintegral_congr hrewrite]
  have hmeas_first : AEMeasurable
      (fun ω => (sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) *
        e (i : ℕ) ω) ℙ :=
    ((measurable_const.indicator (hsets _ hi)).comp (h_meas _)).mul
      (he_meas _) |>.aemeasurable
  have hmeas_rest : AEMeasurable
      (fun ω => ∏ j ∈ (Finset.range n).erase (i : ℕ), e j ω) ℙ :=
    (Finset.measurable_prod _ (fun j _ => he_meas j)).aemeasurable
```

```
  have hindep_first_rest :
      IndepFun (fun ω => (sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) *
          e (i : ℕ) ω)
        (fun ω => ∏ j ∈ (Finset.range n).erase (i : ℕ), e j ω) ℙ := by
    let ψ : ℕ → ℝ → ℝ≥0∞ := fun j x =>
      if j = (i : ℕ) then
        (sets i).indicator (fun _ => (1 : ℝ≥0∞)) x * ENNReal.ofReal (Real.exp (t * x))
      else ENNReal.ofReal (Real.exp (t * x))
    have hψ_meas : ∀ j, Measurable (ψ j) := by
      intro j
      simp only [ψ]
      split_ifs
      · exact (measurable_const.indicator (hsets _ hi)).mul
          (measurable_ennreal_ofReal_exp_t_mul t)
      · exact measurable_ennreal_ofReal_exp_t_mul t
    convert ((h_indep.comp ψ hψ_meas).indepFun_finset_prod_of_notMem
      (fun j => (hψ_meas j).comp (h_meas j))
      (Finset.notMem_erase (i : ℕ) (Finset.range n))).symm using 1
    · ext ω
      show (sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) * e (i : ℕ) ω
          = (ψ (i : ℕ) ∘ X (i : ℕ)) ω
      simp [ψ, e]
    · ext ω
      show ∏ j ∈ (Finset.range n).erase (i : ℕ), e j ω
          = (∏ j ∈ (Finset.range n).erase (i : ℕ), (ψ j ∘ X j)) ω
      rw [Finset.prod_apply]
      apply Finset.prod_congr rfl
      intro j hj
      have hj_ne : j ≠ (i : ℕ) := Finset.ne_of_mem_erase hj
      simp [ψ, e, hj_ne]
  rw [lintegral_mul_eq_lintegral_mul_lintegral_of_indepFun''
    hmeas_first hmeas_rest hindep_first_rest]
  rw [lintegral_prod_eq_prod_lintegral_of_indepFun _ e he_indep he_meas]
  rw [Finset.prod_congr rfl (fun j _ => he_lint j)]
  ring
rw [hLHS, hLHS_factor]
rw [show (∏ i ∈ S, (ℚ_[n] X ℙ n t) ((fun ω => X i.val ω) ⁻¹' sets i)) =
    ∏ i ∈ S, (ℚ_[n] X ℙ n t) ((X (i : ℕ)) ⁻¹' sets i) from rfl]
rw [Finset.prod_congr rfl (fun i hi => h_single i hi)]
have hprod_const_erase : ∀ (i : Finset.range n),
    (∏ j ∈ (Finset.range n).erase (i : ℕ), ENNReal.ofReal M₀) =
      (ENNReal.ofReal M₀) ^ (n - 1) := fun i => by
  rw [Finset.prod_const, Finset.card_erase_of_mem i.2, Finset.card_range]
have h_image_S : (S.image (Subtype.val : Finset.range n → ℕ)) ⊆ Finset.range n :=
  fun j hj => by obtain ⟨i, _, rfl⟩ := Finset.mem_image.mp hj; exact i.2
have h_g_inS : ∀ (i : Finset.range n) (hi : i ∈ S),
    ∫⁻ ω, φ (i : ℕ) (X (i : ℕ) ω) ∂ℙ =
    ∫⁻ ω, (sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) *
      e (i : ℕ) ω ∂ℙ := fun i hi =>
  lintegral_congr fun ω => by
    change (A' (i : ℕ)).indicator _ _ * _ = _; rw [hA'_inS i hi]
```

```
  have h_g_notinS : ∀ j ∈ Finset.range n,
      j ∉ S.image (Subtype.val : Finset.range n → ℕ) →
      ∫⁻ ω, φ j (X j ω) ∂ℙ = ENNReal.ofReal M₀ := fun j _ hj_notmem =>
    hφ_lint_not_inS j fun hj hjS =>
      hj_notmem (Finset.mem_image.mpr ⟨⟨j, Finset.mem_range.mpr hj⟩, hjS, rfl⟩)
  rw [show ∏ j ∈ Finset.range n, ∫⁻ ω, φ j (X j ω) ∂ℙ =
      (∏ j ∈ S.image (Subtype.val : Finset.range n → ℕ), ∫⁻ ω, φ j (X j ω) ∂ℙ) *
        (∏ j ∈ Finset.range n \ S.image (Subtype.val : Finset.range n → ℕ),
          ∫⁻ ω, φ j (X j ω) ∂ℙ) from by rw [← Finset.prod_sdiff h_image_S, mul_comm],
    Finset.prod_congr rfl (fun j hj => h_g_notinS j (Finset.mem_sdiff.mp hj).1
      (Finset.mem_sdiff.mp hj).2), Finset.prod_const,
    show ∏ j ∈ S.image (Subtype.val : Finset.range n → ℕ), ∫⁻ ω, φ j (X j ω) ∂ℙ =
        ∏ i ∈ S, ∫⁻ ω, (sets i).indicator (fun _ => (1 : ℝ≥0∞)) (X (i : ℕ) ω) *
          e (i : ℕ) ω ∂ℙ from by
      rw [Finset.prod_image (fun _ _ _ _ => Subtype.ext)]
      exact Finset.prod_congr rfl h_g_inS,
    show (Finset.range n \ S.image (Subtype.val : Finset.range n → ℕ)).card = n - S.card from by
      rw [Finset.card_sdiff_of_subset h_image_S, Finset.card_range,
        Finset.card_image_of_injective _ fun _ _ hxy => Subtype.ext hxy]]
  simp_rw [hprod_const_erase, Finset.prod_mul_distrib, Finset.prod_const, hM_n_pow,
    ENNReal.inv_pow]
  have hS_card_le : S.card ≤ n :=
    (Finset.card_le_card (Finset.subset_univ _)).trans_eq <| by
      rw [Finset.card_univ, Fintype.card_coe, Finset.card_range]
  have hcancel : (ENNReal.ofReal M₀)⁻¹ * ENNReal.ofReal M₀ = 1 :=
    ENNReal.inv_mul_cancel hM₀_nn hM₀_top
  have hcancel_pow : ∀ k, (ENNReal.ofReal M₀)⁻¹ ^ k * (ENNReal.ofReal M₀) ^ k = 1 :=
    fun k => by rw [← mul_pow, hcancel, one_pow]
  set s := S.card with hs_def
  set P : ℝ≥0∞ := ∏ i ∈ S, ∫⁻ ω, (sets i).indicator (fun _ => (1 : ℝ≥0∞))
    (X (i : ℕ) ω) * e (i : ℕ) ω ∂ℙ
  conv_lhs => rw [show (ENNReal.ofReal M₀)⁻¹ ^ n =
      (ENNReal.ofReal M₀)⁻¹ ^ s * (ENNReal.ofReal M₀)⁻¹ ^ (n - s) by
    rw [← pow_add, Nat.add_sub_cancel' hS_card_le]]
  rw [mul_assoc, ← mul_assoc ((ENNReal.ofReal M₀)⁻¹ ^ (n - s)), mul_comm _ P, mul_assoc,
    hcancel_pow, mul_one]
  rw [← pow_mul, ← pow_mul]
  have h_expand : n * s = s + (n - 1) * s := by
    rcases Nat.eq_zero_or_pos n with hn | hn
    · have hs0 : s = 0 := Nat.le_zero.mp (hn ▸ hS_card_le)
      simp [hn, hs0]
    · conv_lhs => rw [show n = 1 + (n - 1) from (Nat.add_sub_cancel' hn).symm]
      rw [Nat.add_mul, one_mul]
  rw [h_expand, pow_add, show (ENNReal.ofReal M₀)⁻¹ ^ s * (ENNReal.ofReal M₀)⁻¹ ^ ((n - 1) * s) *
      P * ENNReal.ofReal M₀ ^ ((n - 1) * s) = (ENNReal.ofReal M₀)⁻¹ ^ s * P *
      ((ENNReal.ofReal M₀)⁻¹ ^ ((n - 1) * s) * ENNReal.ofReal M₀ ^ ((n - 1) * s)) by ring,
    hcancel_pow, mul_one]

include h_indep h_ident h_meas h_mgf in
/-- Under `ℚ_nt`, `X_i` (for `i < n`) are identically distributed. -/
lemma identDistrib_X_X0_ℚ_nt (t : ℝ) (n : ℕ) {i : ℕ} (hi : i < n) (h0 : 0 < n) :
    IdentDistrib (X i) (X 0) (ℚ_nt X ℙ n t) (ℚ_nt X ℙ n t) where
  aemeasurable_fst := (h_meas i).aemeasurable
  aemeasurable_snd := (h_meas 0).aemeasurable
  map_eq := (map_X_ℚ_nt X h_indep h_ident h_meas h_mgf t n hi).trans
    (map_X_ℚ_nt X h_indep h_ident h_meas h_mgf t n h0).symm
```

```
/-! ### Characteristic function of ‘Z‘ under ‘ℚ_nt‘ -/

include h_indep h_ident h_meas h_mgf h_non_deg in
/-- Under ‘ℚ_nt‘, the characteristic function of ‘Z_n‘ at ‘s‘ is ‘[charFun ν_t ((√n)⁻¹ · s)]^n‘. -/
lemma charFun_map_Z_ℚ_nt (t : ℝ) (n : ℕ) (s : ℝ) :
    charFun ((ℚ_nt X ℙ n t).map (Z X t n)) s =
      (charFun (ν_t X t) ((Real.sqrt n)⁻¹ * s)) ^ n := by
  haveI : IsProbabilityMeasure (ℚ_nt X ℙ n t) :=
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  set m := deriv (cgf (X 0) ℙ) t
  set v := iteratedDeriv 2 (cgf (X 0) ℙ) t
  have hv_ne : Real.sqrt v ≠ 0 := (Real.sqrt_pos.mpr (h_non_deg t)).ne'
  set Y : ℕ → Ω → ℝ := fun k ω => (X k ω - m) / Real.sqrt v with hY_def
  have hY_meas : ∀ k, Measurable (Y k) := fun _ => by fun_prop
  -- ‘Z = (√n)⁻¹ * Σ^n Y_i‘.
  have hZ_eq : Z X t n = fun ω => (Real.sqrt n)⁻¹ * Σ k ∈ Finset.range n, Y k ω := by
    ext ω
    simp only [Z, hY_def, partialSum, Finset.sum_apply,
      Real.sqrt_mul (Nat.cast_nonneg _), ← Finset.sum_div, Finset.sum_sub_distrib,
      Finset.sum_const, Finset.card_range, nsmul_eq_mul]
    ring
  rw [hZ_eq, charFun_map_mul_comp
    (Finset.aemeasurable_fun_sum _ fun k _ => (hY_meas k).aemeasurable)]
  -- Independence of the family ‘Y_i‘ under ‘ℚ_nt‘.
  have hY_indep : iIndepFun ((Finset.range n).restrict Y) (ℚ_nt X ℙ n t) :=
    (iIndepFun_range_ℚ_nt X h_indep h_ident h_meas h_mgf t n).comp
      (fun _ : Finset.range n => fun x : ℝ => (x - m) / Real.sqrt v) (fun _ => by fun_prop)
  -- Apply the characteristic function factorization on ‘1..n‘.
  rw [hY_indep.charFun_map_fun_finset_sum_eq_prod (fun k _ => (hY_meas k).aemeasurable)]
  -- Every factor is ‘charFun (ν_t X t) ((√n)⁻¹ * s)‘.
  have hY_map : ∀ i ∈ Finset.range n,
      (ℚ_nt X ℙ n t).map (Y i) = ν_t X t := fun i hi => by
    rw [show Y i = (fun x : ℝ => (x - m) / Real.sqrt v) ∘ X i from rfl,
      ← Measure.map_map (by fun_prop) (h_meas i),
      map_X_ℚ_nt X h_indep h_ident h_meas h_mgf t n (Finset.mem_range.mp hi)]
    rfl
  rw [Finset.prod_apply, Finset.prod_congr rfl fun i hi => by rw [hY_map i hi],
    Finset.prod_const, Finset.card_range]

include h_meas h_mgf h_non_deg in
/-- ‘(charFun ν_t ((√n)⁻¹ · s))^n → e^(-s^2/2)‘ as ‘n → ∞‘, the characteristic function of ‘𝒩(0,
    1)‘ -/
lemma tendsto_charFun_stdTiltedLaw_pow (t : ℝ) (s : ℝ) :
    Tendsto (fun n : ℕ => (charFun (ν_t X t) ((Real.sqrt n)⁻¹ * s)) ^ n)
      atTop (𝒩 (Complex.exp (-s ^ 2 / 2))) := by
  haveI : IsProbabilityMeasure (ν_t X t) :=
    isProbabilityMeasure_stdTiltedLaw X h_meas h_mgf t
  have h0 : (ν_t X t)[id] = 0 := integral_id_stdTiltedLaw X h_meas h_mgf t
  have h1 : (ν_t X t)[(id : ℝ → ℝ) ^ 2] = 1 := by
    simpa using integral_sq_id_stdTiltedLaw X h_meas h_mgf h_non_deg t
  have hmap : (ν_t X t).map id = ν_t X t := Measure.map_id
  have := tendsto_charFun_inv_sqrt_mul_pow (P := ν_t X t) (X := id)
    aemeasurable_id h0 h1 s
  rwa [hmap] at this
```

```
/-! ### Central Limit Theorem under the tilted measures -/

include h_indep h_ident h_meas h_mgf h_non_deg in
/-- **CLT for tilted measures (characteristic function form):** under `ℚ_nt`, the
characteristic function of `Z_n` converges pointwise to that of `𝒩(0, 1)` (i.e `e^(-s^2/2)`). -/
theorem tendsto_charFun_Z_ℚ_nt (t : ℝ) (s : ℝ) :
    Tendsto (fun n : ℕ => charFun ((ℚ_nt X ℙ n t).map (Z X t n)) s)
      atTop (𝒩 (Complex.exp (-s ^ 2 / 2))) := by
  refine (tendsto_charFun_stdTiltedLaw_pow X h_meas h_mgf h_non_deg t s).congr' ?_
  exact Filter.Eventually.of_forall fun n =>
    (charFun_map_Z_ℚ_nt X h_indep h_ident h_meas h_mgf h_non_deg t n s).symm

/-! ### Consequence: concentration of the empirical mean

As a consequence of the CLT, we show that given `Λ'(t) = a`, `{S_n/n ∈ [a, a+δ]} → 1/2}`, which is
used in the proof of the lower bound of Cramér's theorem, by demonstrating that this is equivalent
to `{Z_n ∈ [0, δ · √(n/Λ''(t))]}` and using the convergence of `Z_n` to `𝒩(0, 1)`. -/

include h_indep h_ident h_meas h_mgf h_non_deg in
/-- For `t` with `Λ'(t) = a` and any `M > 0`, eventually the tilted probability that
`Z_n ∈ [0, M]` is at least `𝒩(0,1)([0, M]) - ε`. -/
lemma liminf_ℚ_nt_Z_Icc_ge (t : ℝ) (M : ℝ) (hM : 0 ≤ M) (ε : ℝ) (hε : 0 < ε) :
    ∀ᶠ n in atTop, ((gaussianReal 0 1) (Set.Icc (0 : ℝ) M)).toReal - ε ≤
      ((ℚ_nt X ℙ n t).map (Z X t n) (Set.Icc (0 : ℝ) M)).toReal := by
  haveI : ∀ n, IsProbabilityMeasure (ℚ_nt X ℙ n t) := fun n =>
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  haveI : ∀ n, IsProbabilityMeasure ((ℚ_nt X ℙ n t).map (Z X t n)) := fun n =>
    Measure.isProbabilityMeasure_map
      (((measurable_partialSum X h_meas n).sub_const _).div_const _).aemeasurable
  haveI : NoAtoms (gaussianReal 0 1) := noAtoms_gaussianReal one_ne_zero
  set μ_n : ℕ → ProbabilityMeasure ℝ := fun n =>
    ⟨(ℚ_nt X ℙ n t).map (Z X t n), inferInstance⟩
  set μ : ProbabilityMeasure ℝ := ⟨gaussianReal 0 1, inferInstance⟩
  -- apply Lévy's convergence theorem
  have h_weak : Tendsto μ_n atTop (𝒩 μ) :=
    ProbabilityMeasure.tendsto_iff_tendsto_charFun.2 fun s => by
      have h_gauss : charFun (gaussianReal 0 1 : Measure ℝ) s =
          Complex.exp (-s ^ 2 / 2) := by rw [charFun_gaussianReal]; push_cast; ring_nf
      simpa [μ_n, μ, h_gauss] using
        tendsto_charFun_Z_ℚ_nt X h_indep h_ident h_meas h_mgf h_non_deg t s
  have h_fr : (μ : Measure ℝ) (frontier (Set.Icc (0 : ℝ) M)) = 0 := by
    rw [show (μ : Measure ℝ) = gaussianReal 0 1 from rfl, frontier_Icc hM]
    exact Set.Finite.measure_zero (by simp) _
  have h_lt : ((μ : Measure ℝ) (Set.Icc (0 : ℝ) M)).toReal - ε <
      ((μ : Measure ℝ) (Set.Icc (0 : ℝ) M)).toReal := by linarith
  simpa using ((ENNReal.tendsto_toReal (measure_ne_top _ _)).comp
    (ProbabilityMeasure.tendsto_measure_of_null_frontier_of_tendsto'
      h_weak h_fr)).eventually_const_le h_lt
```

```
/-- The standard Gaussian has mass `1/2` on `(-∞, 0]`. -/
private lemma gaussianReal_Iic_zero_eq_half :
    (gaussianReal 0 1) (Set.Iic (0 : ℝ)) = 1 / 2 := by
  haveI : NoAtoms (gaussianReal 0 1) := noAtoms_gaussianReal one_ne_zero
  set N := gaussianReal 0 1
  -- By symmetry, `N((-∞, 0]) = N([0, ∞))`.
  have hIoi_eq_Iic : N (Set.Ioi (0 : ℝ)) = N (Set.Iic 0) := by
    have hsym : N.map (fun x : ℝ => -x) = N := by
      simpa using gaussianReal_map_neg (μ := (0 : ℝ)) (v := (1 : ℝ≥0))
    refine (measure_congr Ioi_ae_eq_Ici).trans ?_
    conv_lhs => rw [← hsym, Measure.map_apply (by fun_prop) measurableSet_Ici]
    congr 1; ext x; simp
  -- `N((-∞, 0]) = 1/2` as `N((-∞, 0]) + N([0, ∞)) = N(ℝ) = 1`.
  have hcompl : N (Set.Iic (0 : ℝ)) + N (Set.Iic 0) = 1 := by
    have := measure_add_measure_compl (μ := N) (s := Set.Iic (0 : ℝ)) measurableSet_Iic
    rwa [Set.compl_Iic, measure_univ, hIoi_eq_Iic] at this
  rw [ENNReal.eq_div_iff (by norm_num) (by norm_num), two_mul]; exact hcompl

/-- The standard Gaussian's mass on `[0, M]` tends to `1/2` as `M → ∞`. -/
lemma tendsto_gaussianReal_Icc_toReal_half :
    Tendsto (fun M : ℝ => ((gaussianReal 0 1) (Set.Icc (0 : ℝ) M)).toReal)
      atTop (𝓝 (1 / 2)) := by
  haveI : NoAtoms (gaussianReal 0 1) := noAtoms_gaussianReal one_ne_zero
  set N := gaussianReal 0 1
  have hIic0 : N (Set.Iic (0 : ℝ)) = 1 / 2 := gaussianReal_Iic_zero_eq_half
  have hIio_eq_Iic : N (Set.Iio (0 : ℝ)) = N (Set.Iic 0) := measure_congr Iio_ae_eq_Iic
  -- For `M ≥ 0`, `N([0, M]) = N((-∞, M]) - 1/2`.
  have hIcc_eq : ∀ᶠ M : ℝ in atTop,
      N (Set.Iic M) - (1 / 2 : ℝ≥0∞) = N (Set.Icc (0 : ℝ) M) :=
    (Filter.eventually_ge_atTop 0).mono fun M hM => by
      have hunion : N (Set.Iio 0) + N (Set.Icc 0 M) = N (Set.Iic M) := by
        rw [← Set.Iio_union_Icc_eq_Iic hM]
        exact (measure_union ((Set.Iio_disjoint_Ici le_rfl).mono_right
          Set.Icc_subset_Ici_self) measurableSet_Icc).symm
      rw [← hIic0, ← hIio_eq_Iic]
      exact (ENNReal.eq_sub_of_add_eq (measure_ne_top N _)
        ((add_comm (N (Set.Iio 0)) _) ▸ hunion)).symm
  -- `N([0, M]) → 1/2` in ENNReal
  have hIcc_tendsto : Tendsto (fun M : ℝ => N (Set.Icc (0 : ℝ) M)) atTop (𝓝 (1 / 2)) := by
    have hIic_tendsto : Tendsto (fun M : ℝ => N (Set.Iic M)) atTop (𝓝 1) := by
      simpa using tendsto_measure_Iic_atTop N
    have key := (ENNReal.Tendsto.sub hIic_tendsto
      (tendsto_const_nhds (x := (1 / 2 : ℝ≥0∞))) (Or.inl ENNReal.one_ne_top))
    rw [show (1 : ℝ≥0∞) - 1 / 2 = 1 / 2 from by norm_num] at key
    exact key.congr' hIcc_eq
  rw [show (1 : ℝ) / 2 = ((1 : ℝ≥0∞) / 2).toReal from by norm_num]
  exact (ENNReal.tendsto_toReal (by norm_num)).comp hIcc_tendsto
```

```
include h_indep h_ident h_meas h_mgf h_non_deg in
/-- **Corollary of Tilted CLT:** Given `Λ'(t) = a`, for all `δ > 0` and `ε > 0`,
 `1/2 - ε ≤ ℚ_nt(S_n/n ∈ [a, a+δ])` holds for all sufficiently large `n`. -/
lemma eventually_ℚ_nt_empiricalMean_mem_Icc_ge (t a δ : ℝ) (hδ : 0 < δ) (ε : ℝ) (hε : 0 < ε)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    ∀ᶠ n in atTop,
      (1 / 2 - ε : ℝ) ≤ ((ℚ_nt X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}).toReal := by
  -- Let `v := Λ''(t)`
  set v := iteratedDeriv 2 (cgf (X 0) ℙ) t with hv_def
  have hv_pos : 0 < v := h_non_deg t
  haveI hℚ_prob : ∀ n, IsProbabilityMeasure (ℚ_nt X ℙ n t) := fun n =>
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  have hZ_meas : ∀ n, Measurable (Z X t n) := fun n => by
    unfold Z
    exact ((measurable_partialSum X h_meas n).sub_const _).div_const _
  -- Pick `M₀ ≥ 0` with `1/2 - ε/2 < N([0, M₀])`.
  obtain ⟨M₀, hM₀_lb, hM₀_nonneg⟩ :=
    ((tendsto_gaussianReal_Icc_toReal_half.eventually_const_lt
        (show (1/2 - ε/2 : ℝ) < 1/2 by linarith)).and (Filter.eventually_ge_atTop 0)).exists
  -- Apply `liminf_ℚ_nt_Z_Icc_ge` with `M₀` and `ε/2`.
  have h_liminf := liminf_ℚ_nt_Z_Icc_ge X h_indep h_ident h_meas h_mgf h_non_deg
    t M₀ hM₀_nonneg (ε/2) (by linarith)
  -- `δ · √(n/v) → ∞` as `n → ∞`.
  have h_Mn_ge : ∀ᶠ n : ℕ in atTop, M₀ ≤ δ * Real.sqrt ((n : ℝ) / v) :=
    ((Real.tendsto_sqrt_atTop.comp
      (Filter.Tendsto.atTop_div_const hv_pos tendsto_natCast_atTop_atTop)).const_mul_atTop
      hδ).eventually_ge_atTop M₀
  filter_upwards [h_liminf, h_Mn_ge, Filter.eventually_gt_atTop 0]
    with n h_lim hMn_ge hn_pos
  -- `M_n = δ * √(n / v) ≥ M₀ ≥ 0`.
  set M_n := δ * Real.sqrt ((n : ℝ) / v)
  haveI : IsProbabilityMeasure ((ℚ_nt X ℙ n t).map (Z X t n)) :=
    Measure.isProbabilityMeasure_map (hZ_meas n).aemeasurable
  -- Rewrite the preimage of `Z([0, Mₙ])` as `{ω | Sₙ(ω)/n ∈ [a, a+δ]}`
  have h_set_eq :
      (Z X t n) ⁻¹' Set.Icc (0 : ℝ) M_n =
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)} := by
    ext ω
    simp only [Set.mem_preimage, Set.mem_setOf_eq]
    rw [empiricalMean_mem_Icc_iff_Z_mem_Icc X t a δ n hn_pos (h_non_deg t) ht_deriv ω]
  -- `[0, M₀] ⊆ [0, M_n]` implies `Zₙ([0, M₀]) ≤ Zₙ([0, M_n])` under `ℚ_nt`.
  have h_toReal_mono :
      (((ℚ_nt X ℙ n t).map (Z X t n)) (Set.Icc (0 : ℝ) M₀)).toReal ≤
        (((ℚ_nt X ℙ n t).map (Z X t n)) (Set.Icc (0 : ℝ) M_n)).toReal :=
    ENNReal.toReal_mono (measure_ne_top _ _)
      (measure_mono (Set.Icc_subset_Icc le_rfl hMn_ge))
  -- Combine: `1/2 - ε ≤ N([0, M₀]) - ε/2 ≤ ℚ_nt.map(Z_n)([0, M_n]) = ℚ_nt(S_n / n ∈ [a, a+δ])`.
  rw [← h_set_eq, ← Measure.map_apply (hZ_meas n) measurableSet_Icc]
  linarith [h_lim, h_toReal_mono]

end ProbabilityTheory
```

A.5 Mathlib/Probability/LargeDeviations/Cramers/LowerBound.lean

```
module

public import Mathlib.Probability.LargeDeviations.Cramers.Basic
public import Mathlib.Probability.LargeDeviations.Cramers.TiltedCLT

/-!
# Cramér's Theorem - Lower Bound

This file proves the lower (exponential tilting) bound for Cramér's theorem:

- `cramer_lower_bound`: For any `a ≥ 𝔼[X 0]`, `liminf n⁻¹ * log ℙ(Sₙ/n ≥ a) ≥ -I(a)`.

The proof uses the change-of-measure approach with the family of tilted measures
`ℚₙₜ = ℙ.tilted(t * Sₙ)` where `t` is chosen so that `cgf'(t) = a`.
-/

open ProbabilityTheory MeasureTheory Filter Topology
open scoped ENNReal

@[expose] public section

namespace ProbabilityTheory

variable {Ω : Type*} [MeasureSpace Ω]
variable (X : ℕ → Ω → ℝ)

/- Assumptions for Cramér's theorem -/
variable (h_indep : iIndepFun X ℙ)
variable (h_ident : ∀ n, IdentDistrib (X n) (X 0) ℙ ℙ)
variable (h_meas : ∀ n, Measurable (X n))
variable (h_int : Integrable (X 0) ℙ)
variable (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * X 0 ω)) ℙ)
variable (h_bdd : ∀ a : ℝ, BddAbove (Set.range (fun t => t * a - cgf (X 0) ℙ t)))
variable (h_non_deg : ∀ t : ℝ, 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)
variable (h_exposed : ∀ a : ℝ, 𝔼[X 0] ≤ a → ∃ t, deriv (cgf (X 0) ℙ) t = a)

omit [MeasureSpace Ω] in
/-- `0 < t` and `Sₙ/n ∈ [a, a + δ]` implies `exp(-t · Sₙ) ≥ exp(-t · n · (a + δ))` -/
private lemma exp_neg_mul_S_ge_on_set (t : ℝ) (n : ℕ) (a δ : ℝ) (ht : 0 ≤ t)
    (ω : Ω) (hω : empiricalMean X n ω ∈ Set.Icc a (a + δ)) :
    Real.exp (-t * partialSum X n ω) ≥ Real.exp (-t * n * (a + δ)) := by
  apply Real.exp_le_exp.mpr
  rw [empiricalMean] at hω
  rcases eq_or_ne n 0 with hn | hn
  · simp [hn, partialSum]
  · have hn' : (0 : ℝ) < n := Nat.cast_pos.mpr (Nat.pos_of_ne_zero hn)
    nlinarith [(div_le_iff₀ hn').mp hω.2, mul_nonneg ht hn'.le]

/-- `0 ≤ (1 / n : EReal)` lifted from `ℝ` via `Nat.cast_nonneg`. -/
private lemma ereal_one_div_nat_nonneg (n : ℕ) : (0 : EReal) ≤ ((1 : ℝ) / n : EReal) :=
  EReal.coe_nonneg.mpr (div_nonneg zero_le_one (Nat.cast_nonneg n))
```

```
/-- ‘(1 / ↑n) · ↑c → 0‘ where we lift to ‘EReal‘ from ‘ℝ‘. -/
lemma ereal_inv_nat_mul_const_tendsto_zero (c : ℝ) :
    Tendsto (fun n : ℕ => ((1 : ℝ) / n : EReal) * (c : EReal)) atTop (𝓝 0) := by
  have h_real : Tendsto (fun n : ℕ => (1 / n * c : ℝ)) atTop (𝓝 0) := by
    simpa using ((tendsto_const_nhds (x := (1 : ℝ))).div_atTop
      tendsto_natCast_atTop_atTop).mul (tendsto_const_nhds (x := c))
  exact_mod_cast continuous_coe_real_ereal.continuousAt.tendsto.comp h_real

/-- For ‘y ∈ (0, ∞)‘ and ‘n ≥ 1‘, ‘n⁻¹ log(exp(x) · y) = n⁻¹x + n⁻¹ · log(y)‘, where we lift
    values to EReal and ENNReal where needed for later results. -/
private lemma log_product_split (n : ℕ) (x : ℝ) (y : ENNReal) (hn : n ≥ 1)
    (hy_ne_zero : y ≠ 0) (hy_ne_top : y ≠ ⊤) :
    ((1 : ℝ) / n : EReal) * ENNReal.log (ENNReal.ofReal (Real.exp x * y.toReal)) =
    ((1 : ℝ) / n : EReal) * (x : EReal) + ((1 : ℝ) / n : EReal) * ENNReal.log y := by
  have hy_pos : 0 < y.toReal := ENNReal.toReal_pos hy_ne_zero hy_ne_top
  rw [ENNReal.log_ofReal_of_pos (mul_pos (Real.exp_pos x) hy_pos),
    Real.log_mul (Real.exp_pos x).ne' hy_pos.ne', Real.log_exp,
    ENNReal.log_pos_real hy_ne_zero hy_ne_top, EReal.coe_add]
  exact EReal.left_distrib_of_nonneg_of_ne_top (EReal.coe_nonneg.mpr (by positivity))
    (EReal.coe_ne_top _) _ _

/-- For ‘y ∈ (0, ∞)‘ and ‘n ≥ 1‘,
  ‘n⁻¹ log(exp(-n (t · (a + δ) - l)) · y) = -(ta - l) - tδ + n⁻¹ log(y)‘
  which we will later use with ‘l := Λ(t)‘ -/
private lemma log_exp_product_eq_neg_coef_plus_log (n : ℕ) (t a δ : ℝ) (l : ℝ)
    (y : ENNReal) (hn : n ≥ 1) (hy_ne_zero : y ≠ 0) (hy_ne_top : y ≠ ⊤) :
    ((1 : ℝ) / n : EReal) * ENNReal.log (ENNReal.ofReal
      (Real.exp (-n * (t * (a + δ) - l)) * y.toReal)) =
    (-(t * a - l) - t * δ : EReal) + ((1 : ℝ) / n : EReal) * ENNReal.log y := by
  rw [log_product_split n _ y hn hy_ne_zero hy_ne_top]
  congr 1
  have h_eq : (1 / (n : ℝ)) * (-n * (t * (a + δ) - l)) = -(t * a - l) - t * δ := by
    field_simp; ring
  convert congrArg (fun x : ℝ => (x : EReal)) h_eq using 2

/-- Given ‘c ∈ [-∞,∞]‘ and ‘f(n) → 0‘ as ‘n → ∞‘, ‘f(n) + c → c‘ as ‘n → ∞‘ -/
private lemma tendsto_const_add_vanishing (c : EReal) (f : ℕ → EReal)
    (h : Tendsto f atTop (𝓝 0)) : Tendsto (fun n => c + f n) atTop (𝓝 c) := by
  simpa using (EReal.continuousAt_add (by simp) (by simp)).tendsto.comp
    (tendsto_const_nhds.prodMk_nhds h)
```

```
/-- Given `x, y ∈ [-∞, ∞]`, if `∀ε ∈ ℝ⁺, x - ε ≤ y`, then `x ≤ y`. -/
private lemma EReal.le_of_forall_pos_sub_le {x y : EReal}
    (h : ∀ ε : ℝ, 0 < ε → x - (ε : EReal) ≤ y) : x ≤ y := by
  induction x using EReal.rec with
  | bot => exact bot_le
  | top =>
    have := h 1 zero_lt_one
    rwa [EReal.top_sub_coe] at this
  | coe x_val =>
    induction y using EReal.rec with
    | bot =>
      have := h 1 zero_lt_one
      simp only [EReal.coe_one] at this
      exact absurd (le_bot_iff.mp this) (EReal.coe_ne_bot _)
    | top => exact le_top
    | coe y_val =>
      exact_mod_cast _root_.le_of_forall_sub_le fun ε hε => by exact_mod_cast h ε hε

/-! ### Lemmas requiring IsProbabilityMeasure -/

variable [IsProbabilityMeasure (ℙ : Measure Ω)]

include h_indep h_ident h_meas h_mgf h_non_deg in
/-- `ℚ_nt(S_n/n ∈ [a, a+δ])` is eventually always positive as `n → ∞`.
That is, `∃ c > 0` s.t. `ℚ_nt(S_n/n ∈ [a, a+δ]) > c` for all sufficiently large `n`.
This is a consequence of the Central Limit Theorem assumption. -/
private lemma tilted_window_lower_bound_from_concentration (a t δ : ℝ) (hδ : 0 < δ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    ∃ c > 0, ∀ᶠ n in atTop,
      c ≤ ((ℚ_nt X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}).toReal := by
  refine ⟨1/4, by norm_num, ?_⟩
  filter_upwards [eventually_ℚ_nt_empiricalMean_mem_Icc_ge X h_indep h_ident h_meas h_mgf
    h_non_deg t a δ hδ (1/4) (by norm_num) ht_deriv] with n hn
  linarith

/-- For an event `E ⊆ Ω` and a random variable `f : Ω → ℝ`, we have  `ℙ(E) = 𝔼[e^f] ∫_E exp(-f)
    dℙ_f`
where `ℙ_f` is the measure `ℙ` exponentially tilted with respect to `f`. That is, it has density
proportional to `exp(f)`. -/
private lemma measure_eq_integral_exp_neg_tilted (f : Ω → ℝ) (E : Set Ω)
    (h_int : Integrable (fun ω => Real.exp (f ω)) ℙ)
    (hE : MeasurableSet E) :
    (ℙ E).toReal =
      (𝔼[fun ω => Real.exp (f ω)]) * (∫ ω in E, Real.exp (-f ω) ∂(Measure.tilted ℙ f)) := by
  rw [setIntegral_tilted' f (fun ω => Real.exp (-f ω)) hE]
  have h_ne : 𝔼[fun x => Real.exp (f x)] ≠ 0 := (integral_exp_pos h_int).ne'
  simp_rw [smul_eq_mul, div_mul_eq_mul_div, ← Real.exp_add, add_neg_cancel, Real.exp_zero,
    one_div,
    setIntegral_const, Measure.real, smul_eq_mul]
  field_simp
```

```
include h_indep h_ident h_meas h_mgf in
/-- `ℙ(Sₙ/n ∈ [a, a + δ]) ≥ exp(-n(ta - Λ(t))) · ℚₙₜ(Sₙ/n ∈ [a, a + δ])` -/
lemma change_of_measure_lower_bound (a δ t : ℝ) (n : ℕ) (ht : 0 < t)
    (h_int : Integrable (fun ω => Real.exp (t * partialSum X n ω)) ℙ) :
    let E := {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}
    (ℙ E).toReal ≥
      Real.exp (-n * (t * (a + δ) - cgf (X 0) ℙ t)) *
      ((ℚₙₜ X ℙ n t) E).toReal := by
  intro E
  have hE : MeasurableSet E :=
    measurableSet_Icc.preimage (measurable_empiricalMean X h_meas n)
  rw [measure_eq_integral_exp_neg_tilted (fun ω => t * partialSum X n ω) E h_int hE]
  change mgf (partialSum X n) ℙ t * _ ≥ _
  rw [mgf_sum_eq_exp_n_prod_cgf X h_indep h_ident h_meas h_mgf n t]
  haveI : IsProbabilityMeasure (ℚₙₜ X ℙ n t) :=
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  have h_bound :
      ∫ ω in E, Real.exp (-t * partialSum X n ω)
        ∂(ℚₙₜ X ℙ n t) ≥
      Real.exp (-t * n * (a + δ)) *
        ((ℚₙₜ X ℙ n t) E).toReal := by
    calc ∫ ω in E, Real.exp (-t * partialSum X n ω)
        ∂(ℚₙₜ X ℙ n t)
        ≥ ∫ ω in E, Real.exp (-t * n * (a + δ))
          ∂(ℚₙₜ X ℙ n t) :=
          setIntegral_mono_on (integrable_const _).integrableOn
            (Integrable.integrableOn <| by
              rw [show (ℚₙₜ X ℙ n t) = Measure.tilted ℙ (fun ω => t * partialSum X n ω) from rfl,
                integrable_tilted_iff h_int]
              simp [← Real.exp_add])
            hE (exp_neg_mul_S_ge_on_set X t n a δ ht.le)
      _ = ((ℚₙₜ X ℙ n t).real E) ·
            Real.exp (-t * n * (a + δ)) := setIntegral_const _
      _ = Real.exp (-t * n * (a + δ)) *
            ((ℚₙₜ X ℙ n t) E).toReal := by
          rw [Measure.real, smul_eq_mul]; ring
  have key : Real.exp (n * cgf (X 0) ℙ t) *
      (Real.exp (-t * n * (a + δ)) *
        ((ℚₙₜ X ℙ n t) E).toReal) =
    Real.exp (-n * (t * (a + δ) - cgf (X 0) ℙ t)) *
      ((ℚₙₜ X ℙ n t) E).toReal := by
    rw [← mul_assoc, ← Real.exp_add]; ring_nf
  rw [← key]
  gcongr
  convert h_bound.le using 2
  ring_nf
```

```
include h_indep h_ident h_meas h_mgf h_non_deg in
/-- The error term `n⁻¹ * log(ℚ_{nt}(S_n/n ∈ [a, a+δ])) → 0` as `n → ∞` -/
private lemma error_term_vanishes (a t δ : ℝ) (hδ : 0 < δ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    Tendsto (fun n : ℕ =>
      ((1 : ℝ) / n : EReal) * ENNReal.log ((ℚ_{nt} X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)})) atTop (𝓝 0) := by
  obtain ⟨c, hc_pos, h_bounded⟩ :=
    tilted_window_lower_bound_from_concentration X h_indep h_ident h_meas h_mgf h_non_deg
      a t δ hδ ht_deriv
  haveI : ∀ m, IsProbabilityMeasure (ℚ_{nt} X ℙ m t) := fun m =>
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t m
  have h_lower_tendsto : Tendsto (fun m : ℕ =>
      ((1 : ℝ) / m : EReal) * ENNReal.log (ENNReal.ofReal c)) atTop (𝓝 0) := by
    rw [ENNReal.log_ofReal_of_pos hc_pos]
    exact ereal_inv_nat_mul_const_tendsto_zero (Real.log c)
  have h_upper_tendsto : Tendsto (fun (_ : ℕ) => (0 : EReal)) atTop (𝓝 0) := tendsto_const_nhds
  have h_eventually : ∀ᶠ (m : ℕ) in atTop,
      ((1 : ℝ) / m : EReal) * ENNReal.log (ENNReal.ofReal c)
      ≤ ((1 : ℝ) / m : EReal) * ENNReal.log ((ℚ_{nt} X ℙ m t)
          {ω | empiricalMean X m ω ∈ Set.Icc a (a + δ)})
      ∧ ((1 : ℝ) / m : EReal) * ENNReal.log ((ℚ_{nt} X ℙ m t)
          {ω | empiricalMean X m ω ∈ Set.Icc a (a + δ)})
      ≤ 0 := by
    filter_upwards [h_bounded] with m hm_bound
    refine ⟨mul_le_mul_of_nonneg_left ?_ (ereal_one_div_nat_nonneg m),
      mul_nonpos_of_nonneg_of_nonpos (ereal_one_div_nat_nonneg m) ?_⟩
    · exact ENNReal.log_le_log <| (ENNReal.ofReal_le_iff_le_toReal (measure_ne_top _ _)).mpr
        hm_bound
    · exact ENNReal.log_le_zero_iff.mpr prob_le_one
  exact tendsto_of_tendsto_of_tendsto_of_le_of_le' h_lower_tendsto h_upper_tendsto
    (h_eventually.mono fun m h => h.1) (h_eventually.mono fun m h => h.2)
```

```
include h_indep h_ident h_meas h_mgf h_non_deg in
/-- For ‘δ,t > 0‘ with ‘Λ’(t) = a‘, we have ‘liminf_n n^{-1} log ℙ(S_n/n ≥ a) ≥ -(ta - Λ(t)) - tδ ‘ -/
private lemma lower_bound_via_tilted (a t δ : ℝ) (hδ : 0 < δ) (ht : 0 < t)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    liminf (fun n : ℕ =>
      ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop
    ≥ (-(t * a - cgf (X 0) ℙ t) : EReal) - (t * δ : EReal) := by
  -- ‘n^{-1} log ℙ(S_n/n ≥ a) ≥ (ta - Λ(t)) - tδ + n^{-1} log ℚ_{nt}(S_n/n ∈ [a, a + δ])‘
  have h_pointwise : ∀ n : ℕ, n ≥ 1 →
      ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})
      ≥ (-(t * a - cgf (X 0) ℙ t) - t * δ : EReal)
        + ((1 : ℝ) / n : EReal) * ENNReal.log ((ℚ_{nt} X ℙ n t)
            {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}) := by
    intro n hn
    haveI : IsProbabilityMeasure (ℚ_{nt} X ℙ n t) :=
      isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
    have h_subset : {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)} ⊆
        {ω | empiricalMean X n ω ≥ a} := fun _ hω => hω.1
    let E := {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)}
    let F := {ω | empiricalMean X n ω ≥ a}
    have h_prob_mono : (ℙ F).toReal ≥ (ℙ E).toReal :=
      ENNReal.toReal_mono (measure_ne_top _ _) (measure_mono h_subset)
    have h_log_ineq : ENNReal.log (ℙ F) ≥
        ENNReal.log (ENNReal.ofReal (Real.exp (-n * (t * (a + δ) - cgf (X 0) ℙ t)) *
          ((ℚ_{nt} X ℙ n t) E).toReal)) := by
      apply ENNReal.log_le_log
      rw [ENNReal.ofReal_le_iff_le_toReal (measure_ne_top _ _)]
      linarith [h_prob_mono, change_of_measure_lower_bound X h_indep h_ident h_meas h_mgf
        a δ t n ht (integrable_exp_sum X h_indep h_ident h_meas h_mgf t n)]
    calc ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ F)
        ≥ ((1 : ℝ) / n : EReal) * ENNReal.log (ENNReal.ofReal
            (Real.exp (-n * (t * (a + δ) - cgf (X 0) ℙ t)) *
              ((ℚ_{nt} X ℙ n t) E).toReal)) :=
          mul_le_mul_of_nonneg_left (by exact_mod_cast h_log_ineq)
            (ereal_one_div_nat_nonneg n)
      _ = (-(t * a - cgf (X 0) ℙ t) - t * δ : EReal)
          + ((1 : ℝ) / n : EReal) *
            ENNReal.log ((ℚ_{nt} X ℙ n t) E) := by
        by_cases h_tilted_zero : (ℚ_{nt} X ℙ n t) E = 0
        · rw [h_tilted_zero]
          simp only [ENNReal.toReal_zero, mul_zero, ENNReal.ofReal_zero, ENNReal.log_zero]
          have h1n_pos : (0 : EReal) < ((1 : ℝ) / n : EReal) := EReal.coe_pos.mpr (by positivity)
          rw [EReal.mul_bot_of_pos h1n_pos]
          simp only [EReal.add_bot]
        · exact log_exp_product_eq_neg_coef_plus_log n t a δ (cgf (X 0) ℙ t) _ hn
            h_tilted_zero (measure_ne_top _ _)
```

```
  -- The error term `n^{-1} log ℚ_{nt}(S_n/n ∈ [a, a + δ])` vanishes as `n → ∞`.
  have h_error_vanish := @error_term_vanishes _ _ X h_indep h_ident h_meas h_mgf h_non_deg
    _ a t δ hδ ht_deriv
  let rhs_seq : ℕ → EReal := fun n =>
    (-(t * a - cgf (X 0) ℙ t) - t * δ : EReal)
    + ((1 : ℝ) / n : EReal) * ENNReal.log ((ℚ_{nt} X ℙ n t)
        {ω | empiricalMean X n ω ∈ Set.Icc a (a + δ)})
  have h_rhs_limit : Tendsto rhs_seq atTop
      (𝓝 ((-(t * a - cgf (X 0) ℙ t) : EReal) - (t * δ : EReal))) :=
    tendsto_const_add_vanishing _ _ h_error_vanish
  -- Combine the pointwise inequality and vanishing error term to conclude
  have h_eventually : ∀ᶠ (n : ℕ) in atTop,
      ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})
      ≥ rhs_seq n :=
    Filter.eventually_atTop.mpr ⟨1, h_pointwise⟩
  rw [← h_rhs_limit.liminf_eq]
  exact liminf_le_liminf h_eventually

include h_mgf h_non_deg in
/-- If `a ≥ 𝔼[X₁]` and `Λ'(t) = a`, then `0 ≤ t` -/
private lemma deriv_cgf_nonneg_of_ge_mean (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) (t : ℝ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    0 ≤ t := by
  by_contra! ht_neg
  have h_strict_mono : StrictMono (deriv (cgf (X 0) ℙ)) :=
    strictMono_of_deriv_pos fun x => by simpa [iteratedDeriv_succ] using h_non_deg x
  have := h_strict_mono ht_neg
  rw [ht_deriv, deriv_cgf_zero (mem_interior_integrableExpSet X h_mgf 0)] at this
  simp at this
  linarith

omit [IsProbabilityMeasure (ℙ : Measure Ω)] in
include h_mgf in
/-- The CGF of `X 0` is analytic on all of `ℝ`. -/
private lemma analyticOn_cgf_univ : AnalyticOn ℝ (cgf (X 0) ℙ) Set.univ :=
  Set.eq_univ_of_forall (mem_interior_integrableExpSet X h_mgf) ▷ analyticOn_cgf
```

```
omit [IsProbabilityMeasure (ℙ : Measure Ω)] in
include h_mgf h_bdd h_non_deg in
/-- Given `Λ'(t) = a`, the rate function satisfies `I(a) = ta - Λ(t)`. -/
private lemma rateFunction_eq_of_deriv_eq (a t : ℝ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) :
    I X a = t * a - cgf (X 0) ℙ t := by
  rw [I]
  have h_analytic : AnalyticOn ℝ (cgf (X 0) ℙ) Set.univ := analyticOn_cgf_univ X h_mgf
  -- We proceed by proving inequality in both directions
  refine le_antisymm (ciSup_le fun s => ?_) (le_ciSup (h_bdd a) t)
  -- Sub-goal `sa - Λ(s) ≤ ta - Λ(t)`, i.e. `Λ(s) ≥ Λ(t) + a(s - t)`: the CGF is above its
  -- tangent line at `t`, as it is strictly convex (from `h_non_deg`: `Λ''(x) > 0`).
  have h_convex : StrictConvexOn ℝ Set.univ (cgf (X 0) ℙ) :=
    strictConvexOn_of_deriv2_pos' convex_univ h_analytic.continuousOn
      fun x _ => by simpa [← iteratedDeriv_eq_iterate] using h_non_deg x
  have h_diff : DifferentiableAt ℝ (cgf (X 0) ℙ) t :=
    h_analytic.differentiableOn.differentiableAt (isOpen_univ.mem_nhds (Set.mem_univ t))
  suffices h_tangent : cgf (X 0) ℙ s ≥ cgf (X 0) ℙ t + a * (s - t) by linarith
  rcases lt_trichotomy s t with hst | rfl | hts
  · have h_slope := h_convex.convexOn.slope_le_of_hasDerivAt
      (Set.mem_univ s) (Set.mem_univ t) hst h_diff.hasDerivAt
    rw [ht_deriv, slope_def_field] at h_slope
    rw [ge_iff_le, ← sub_nonneg]
    have : 0 < t - s := sub_pos.mpr hst
    nlinarith [(div_le_iff₀ this).mp h_slope]
  · simp
  · have h_slope := h_convex.convexOn.deriv_le_slope
      (Set.mem_univ t) (Set.mem_univ s) hts h_diff
    rw [ht_deriv, slope_def_field] at h_slope
    rw [ge_iff_le, ← sub_nonneg]
    have : 0 < s - t := sub_pos.mpr hts
    nlinarith [(le_div_iff₀ this).mp h_slope]
```

```
include h_indep h_ident h_meas h_mgf h_non_deg in
/-- Edge-case of `cramer_lower_bound` at `a = 𝔼[X 0]` -/
private lemma cramer_lower_bound_at_mean (a : ℝ) (ht_deriv : deriv (cgf (X 0) ℙ) 0 = a) :
    (0 : EReal) ≤
      liminf (fun n : ℕ =>
        ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop := by
  -- `Var[X] > 0`
  have h_var_pos : 0 < variance (X 0) ℙ := by
    have h_int_zero : (0 : ℝ) ∈ interior (integrableExpSet (X 0) ℙ) :=
      mem_interior_integrableExpSet X h_mgf 0
    have h := variance_tilted_mul (X := X 0) (μ := ℙ) h_int_zero
    simp only [zero_mul, show (fun _ : Ω => (0 : ℝ)) = 0 from rfl, tilted_zero] at h
    exact h ▷ h_non_deg 0
  -- `∃ c > 0` such that for all sufficiently large `n`, `ℙ(S_n/n ≥ a) ≥ c`
  -- i.e. the asymptotic lower bound `ℙ(S_n/n ≥ a)` is greater than 0, which we derive from CLT
  -- on the tilted measures, noting that at `t = 0`, the tilted measures `ℚ_nt` coincide with `ℙ`.
  have h_prob_lower_bound : ∃ c > 0, ∀ᶠ n in atTop,
      c ≤ (ℙ {ω | a ≤ empiricalMean X n ω}).toReal := by
    have h_bound := eventually_ℚ_nt_empiricalMean_mem_Icc_ge X h_indep h_ident h_meas h_mgf
      h_non_deg 0 a 1 (by norm_num) (1/4) (by norm_num) ht_deriv
    refine ⟨1/4, by norm_num, ?_⟩
    filter_upwards [h_bound] with n hn
    have h_eq_meas : ℚ_nt X ℙ n 0 = ℙ := by
      change Measure.tilted ℙ (fun ω => 0 * partialSum X n ω) = ℙ
      simp_rw [zero_mul]; exact tilted_zero ℙ
    rw [h_eq_meas] at hn
    calc (1 / 4 : ℝ)
        = 1 / 2 - 1 / 4 := by norm_num
      _ ≤ (ℙ {ω | empiricalMean X n ω ∈ Set.Icc a (a + 1)}).toReal := hn
      _ ≤ (ℙ {ω | a ≤ empiricalMean X n ω}).toReal :=
        ENNReal.toReal_mono (measure_ne_top _ _) (measure_mono fun _ hω => hω.1)
  obtain ⟨c, hc_pos, h_eventually_lower⟩ := h_prob_lower_bound
  have h_lower_tendsto :
      Tendsto (fun n : ℕ => ((1 : ℝ) / n : EReal) * ENNReal.log (ENNReal.ofReal c))
        atTop (𝓝 0) := by
    rw [ENNReal.log_ofReal_of_pos hc_pos]
    exact ereal_inv_nat_mul_const_tendsto_zero (Real.log c)
  have h_upper_tendsto : Tendsto (fun (_ : ℕ) => (0 : EReal)) atTop (𝓝 0) := tendsto_const_nhds
  -- Establish bounds to apply squeeze theorem `n^{-1} log (c) ≤ n^{-1} log ℙ(S_n/n ≥ a) ≤ 0`
  have h_squeeze : ∀ᶠ (m : ℕ) in atTop,
      ((1 : ℝ) / (m : ℝ) : EReal) * ENNReal.log (ENNReal.ofReal c)
      ≤ ((1 : ℝ) / (m : ℝ) : EReal) * ENNReal.log (ℙ {ω | a ≤ empiricalMean X m ω})
      ∧ ((1 : ℝ) / (m : ℝ) : EReal) * ENNReal.log (ℙ {ω | a ≤ empiricalMean X m ω}) ≤ 0 := by
    filter_upwards [h_eventually_lower] with m hm_lower
    refine ⟨mul_le_mul_of_nonneg_left ?_ (ereal_one_div_nat_nonneg m),
      mul_nonpos_of_nonneg_of_nonpos (ereal_one_div_nat_nonneg m) ?_⟩
    · exact ENNReal.log_le_log <|
        (ENNReal.ofReal_le_iff_le_toReal (measure_ne_top _ _)).mpr hm_lower
    · exact ENNReal.log_le_zero_iff.mpr prob_le_one
```

```
  -- ‘n⁻¹ log ℙ(Sₙ/n ≥ a) → 0‘ by squeeze theorem
  have h_tendsto :
      Tendsto (fun n : ℕ => ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ {ω | a ≤ empiricalMean X n ω
    }))
        atTop (𝓝 0) :=
    tendsto_of_tendsto_of_tendsto_of_le_of_le' h_lower_tendsto h_upper_tendsto
      (h_squeeze.mono fun _ hn => hn.1) (h_squeeze.mono fun _ hn => hn.2)
  change (0 : EReal) ≤
    liminf (fun n : ℕ => ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ {ω | a ≤ empiricalMean X n ω}))
      atTop
  rw [h_tendsto.liminf_eq]

include h_indep h_ident h_meas h_mgf h_bdd h_non_deg h_exposed in
/-- **Cramér's Theorem (Lower Bound)**: Given ‘a ≥ E[X 0]‘, ‘-I(a) ≤ liminfₙ n⁻¹ log ℙ(Sₙ/n ≥
    a)‘ -/
theorem cramer_lower_bound (a : ℝ) (h_mean : 𝔼[X 0] ≤ a) :
    (- I X a : EReal) ≤
      liminf (fun n : ℕ =>
        ((1 : ℝ) / (n : ℝ) : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a})) atTop := by
  -- Get the ‘t‘ such that ‘Λ'(t) = a‘.
  obtain ⟨t, ht_deriv⟩ := h_exposed a h_mean
  have ht_nonneg : 0 ≤ t := deriv_cgf_nonneg_of_ge_mean X h_mgf h_non_deg a h_mean t ht_deriv
  have h_rate_eq : I X a = t * a - cgf (X 0) ℙ t :=
    rateFunction_eq_of_deriv_eq X h_mgf h_bdd h_non_deg a t ht_deriv
  rw [h_rate_eq]
  let LHS_val :=
    liminf (fun n : ℕ => ((1 : ℝ) / n : EReal) * ENNReal.log (ℙ {ω | empiricalMean X n ω ≥ a}))
      atTop
  -- Casework on whether ‘t = 0‘
  by_cases ht_zero : t = 0
  · -- ‘t = 0‘ (which implies ‘a = 𝔼[X]‘ from solving ‘a = Λ'(t) = Λ'(0)‘).
    subst ht_zero
    simp only [zero_mul, cgf_zero, sub_zero, EReal.coe_zero, neg_zero]
    exact cramer_lower_bound_at_mean X h_indep h_ident h_meas h_mgf h_non_deg a ht_deriv
  have ht_pos : 0 < t := lt_of_le_of_ne ht_nonneg (Ne.symm ht_zero)
  -- Bound the liminf from below for any fixed positive ‘δ‘ using the tilted measure bounds.
  have h_bound_for_all_delta : ∀ (δ : ℝ), 0 < δ →
      (-(t * a - cgf (X 0) ℙ t) - t * δ : EReal) ≤ LHS_val := fun δ hδ =>
    (lower_bound_via_tilted X h_indep h_ident h_meas h_mgf h_non_deg a t δ hδ ht_pos ht_deriv).le
  -- Conclude the lower bound by taking ‘δ → 0‘.
  refine EReal.le_of_forall_pos_sub_le fun ε hε => ?_
  have h := h_bound_for_all_delta (ε / t) (div_pos hε ht_pos)
  rwa [show ((t : EReal) * (ε / t : ℝ) : EReal) = ((ε : ℝ) : EReal) from by
    rw [← EReal.coe_mul]; norm_cast; field_simp] at h
```

```
/-! ### Extras: Chebyshev-based concentration of the tilted measure
The following results are not used by the current lower-bound proof. They establish additional
elementary results on the concentration by Chebyshev's inequality, instead of the CLT on
tilted measures approach used by the actual result. -/
/-- For constant `C` and `δ > 0`, `n⁻¹ C / δ² → 0` as `n → ∞`. -/
private lemma variance_term_tendsto_zero (C : ℝ) (δ : ℝ) :
    Tendsto (fun n : ℕ => ENNReal.ofReal ((1 / n) * C / δ ^ 2)) atTop (𝓝 0) := by
  rw [← ENNReal.ofReal_zero]
  refine ENNReal.continuous_ofReal.continuousAt.tendsto.comp ?_
  convert tendsto_const_div_atTop_nhds_zero_nat (C / δ ^ 2) using 1
  ext n
  ring

/-- If `f ≤ c` a.e. under `μ`, `f` is integrable, and `∫ f dμ = c`, then `f = c` a.e. under `μ` -/
private lemma ae_eq_const_of_integral_eq_const_of_le {α : Type*} {m : MeasurableSpace α}
    (μ : Measure α) [IsProbabilityMeasure μ] {f : α → ℝ} (c : ℝ)
    (hf : Integrable f μ) (h_le : ∀ᵐ ω ∂μ, f ω ≤ c) (h_int : μ[f] = c) : ∀ᵐ ω ∂μ, f ω = c := by
  have h_diff_nonneg : ∀ᵐ ω ∂μ, 0 ≤ c - f ω := by
    filter_upwards [h_le] with ω hω using sub_nonneg.mpr hω
  have h_int_diff : μ[fun ω => c - f ω] = 0 := by
    rw [integral_sub (integrable_const c) hf]; simp [h_int]
  have h_diff_ae_zero : ∀ᵐ ω ∂μ, c - f ω = 0 :=
    (integral_eq_zero_iff_of_nonneg_ae h_diff_nonneg ((integrable_const c).sub hf)).mp h_int_diff
  filter_upwards [h_diff_ae_zero] with ω h using by linarith

include h_indep h_ident h_meas h_mgf in
/-- `Λ_Sₙ = fun t => n · Λ_X(t)` -/
private lemma cgf_partialSum_eq_smul_cgf (n : ℕ) :
    cgf (partialSum X n) ℙ = fun s => n * cgf (X 0) ℙ s :=
  funext <| cgf_sum_eq_n_prod_cgf X h_indep h_ident h_meas h_mgf n

include h_indep h_ident h_meas h_mgf in
/-- `Λ'_Sₙ(t) = n · Λ'_X(t)` -/
private lemma deriv_cgf_sum (t : ℝ) (n : ℕ) :
    deriv (cgf (partialSum X n) ℙ) t = n * deriv (cgf (X 0) ℙ) t := by
  rw [cgf_partialSum_eq_smul_cgf X h_indep h_ident h_meas h_mgf n]
  exact deriv_const_mul (n : ℝ) ((analyticOn_cgf_univ X h_mgf).differentiableOn.differentiableAt
    (isOpen_univ.mem_nhds (Set.mem_univ t)))

include h_indep h_ident h_meas h_mgf in
/-- `Λ''_Sₙ(t) = n · Λ''_X(t)` -/
private lemma iteratedDeriv_two_cgf_sum (t : ℝ) (n : ℕ) :
    iteratedDeriv 2 (cgf (partialSum X n) ℙ) t =
      n * iteratedDeriv 2 (cgf (X 0) ℙ) t := by
  rw [cgf_partialSum_eq_smul_cgf X h_indep h_ident h_meas h_mgf n]
  exact iteratedDeriv_const_mul_field (n : ℝ) (cgf (X 0) ℙ)

include h_indep h_ident h_meas h_mgf in
/-- `𝔼_ℚ_{nt}[Sₙ/n] = Λ'_Sₙ(t)` and `𝕍_ℚ_{nt}[Sₙ/n] = Λ''_Sₙ(t)` -/
private lemma tilted_Sn_moments (t : ℝ) (n : ℕ) :
    let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
    μ_t[partialSum X n] = deriv (cgf (partialSum X n) ℙ) t ∧
    variance (partialSum X n) μ_t = iteratedDeriv 2 (cgf (partialSum X n) ℙ) t := by
  have ht_Sn : t ∈ interior (integrableExpSet (partialSum X n) ℙ) :=
    mem_interior_integrableExpSet_partialSum X h_indep h_ident h_meas h_mgf t n
  exact ⟨integral_tilted_mul_self ht_Sn, variance_tilted_mul ht_Sn⟩
```

```
include h_indep h_ident h_meas h_mgf in
/-- `𝔼_ℚ_nt[S_n/n] = Λ'(t)` and `𝕍_ℚ_nt[S_n/n] = n⁻¹ Λ''(t)` -/
private lemma tilted_empirical_moments (t : ℝ) (n : ℕ) (hn : n ≠ 0) :
    let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
    μ_t[empiricalMean X n] = deriv (cgf (X 0) ℙ) t ∧
    variance (empiricalMean X n) μ_t = (1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ) t := by
  intro μ_t
  have h_Sn := @tilted_Sn_moments _ _ X h_indep h_ident h_meas h_mgf _ t n
  refine ⟨?_, ?_⟩
  · change μ_t[fun ω => partialSum X n ω / n] = deriv (cgf (X 0) ℙ) t
    rw [integral_div, h_Sn.1, @deriv_cgf_sum _ _ X h_indep h_ident h_meas h_mgf _ t n]
    field_simp
  · change variance (fun ω => partialSum X n ω / n) μ_t = (1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ)
    t
    simp_rw [div_eq_inv_mul]
    rw [show (fun ω => ((n : ℝ))⁻¹ * partialSum X n ω) = ((n : ℝ))⁻¹ · (partialSum X n) from rfl,
      variance_smul, h_Sn.2, @iteratedDeriv_two_cgf_sum _ _ X h_indep h_ident h_meas h_mgf _ t n]
    field_simp

include h_indep h_ident h_meas h_mgf in
/-- Given `Λ'(t) = a`, `𝔼_{ℚ_nt}[S_n/n] = a` -/
private lemma tilted_empirical_mean_eq_of_deriv (t a : ℝ) (n : ℕ) (hn : n ≠ 0)
    (h_deriv : deriv (cgf (X 0) ℙ) t = a) :
    (Measure.tilted ℙ (fun ω => t * partialSum X n ω))[empiricalMean X n] = a :=
  (tilted_empirical_moments X h_indep h_ident h_meas h_mgf t n hn).1.trans h_deriv

include h_indep h_ident h_meas h_mgf in
/-- For `δ > 0`, given `Λ'(t) = a`, `ℚ_nt(δ ≤ |S_n/n - a|) ≤ n⁻¹ Λ''(t) / δ^2` -/
private lemma tilted_deviation_bound (t a : ℝ) (n : ℕ) (hn : n ≠ 0) (δ : ℝ) (hδ : 0 < δ)
    (h_match : deriv (cgf (X 0) ℙ) t = a) :
    let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
    μ_t {ω | δ ≤ |empiricalMean X n ω - a|} ≤
      ENNReal.ofReal ((1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ) t / δ ^ 2) := by
  intro μ_t
  have h_mean : μ_t[empiricalMean X n] = a :=
    tilted_empirical_mean_eq_of_deriv X h_indep h_ident h_meas h_mgf t a n hn h_match
  haveI : IsProbabilityMeasure μ_t :=
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  have h_memLp : MemLp (empiricalMean X n) 2 μ_t := by
    have ht_Sn : t ∈ interior (integrableExpSet (partialSum X n) ℙ) :=
      mem_interior_integrableExpSet_partialSum X h_indep h_ident h_meas h_mgf t n
    have : (empiricalMean X n) = fun ω => (n : ℝ)⁻¹ * partialSum X n ω := by
      ext ω; simp [empiricalMean, div_eq_inv_mul]
    rw [this]
    exact (memLp_tilted_mul ht_Sn 2 : MemLp (partialSum X n) 2 μ_t).const_mul _
  calc μ_t {ω | δ ≤ |empiricalMean X n ω - a|}
      = μ_t {ω | δ ≤ |empiricalMean X n ω - μ_t[empiricalMean X n]|} := by
        simp_rw [h_mean]
    _ ≤ ENNReal.ofReal (variance (empiricalMean X n) μ_t / δ ^ 2) :=
        meas_ge_le_variance_div_sq h_memLp hδ
    _ = ENNReal.ofReal ((1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ) t / δ ^ 2) := by
        rw [(tilted_empirical_moments X h_indep h_ident h_meas h_mgf t n hn).2]
```

```
include h_indep h_ident h_meas h_mgf in
/-- For ‘δ > 0‘, given ‘Λ'(t) = a‘, ‘ℚ_nt(|S_n/n - a| < δ) → 1‘ as ‘n → ∞‘ -/
private lemma tilted_measure_concentrates (t a δ : ℝ) (hδ : 0 < δ)
    (h_match : deriv (cgf (X 0) ℙ) t = a) :
    Tendsto (fun n => ((ℚ_nt X ℙ n t)
      {ω | |empiricalMean X n ω - a| < δ}).toReal) atTop (𝓝 1) := by
  haveI h_prob_n_all : ∀ n, IsProbabilityMeasure (ℚ_nt X ℙ n t) := fun n =>
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  have h_complement_to_zero : Tendsto (fun n => ((ℚ_nt X ℙ n t)
      {ω | δ ≤ |empiricalMean X n ω - a|}).toReal) atTop (𝓝 0) := by
    rw [ENNReal.tendsto_toReal_zero_iff]
    refine ENNReal.tendsto_nhds_zero.2 (fun ε hε => ?_)
    obtain ⟨N, hN⟩ := (ENNReal.tendsto_atTop_zero.1
      (variance_term_tendsto_zero (iteratedDeriv 2 (cgf (X 0) ℙ) t) δ)) ε hε
    filter_upwards [eventually_ge_atTop (max N 1)] with n hn
    calc (ℚ_nt X ℙ n t) {ω | δ ≤ |empiricalMean X n ω - a|}
        ≤ ENNReal.ofReal ((1 / n) * iteratedDeriv 2 (cgf (X 0) ℙ) t / δ ^ 2) :=
          tilted_deviation_bound (X := X) (h_indep := h_indep) (h_ident := h_ident)
            (h_meas := h_meas) (h_mgf := h_mgf) t a n (by omega) δ hδ h_match
      _ ≤ ε := hN n (by omega)
  have h_compl : ∀ n, {ω | |empiricalMean X n ω - a| < δ}^c =
      {ω | δ ≤ |empiricalMean X n ω - a|} := fun _ => by
    ext; simp [not_lt]
  have h_eq : ∀ n, (ℚ_nt X ℙ n t)
      {ω | |empiricalMean X n ω - a| < δ} =
      1 - (ℚ_nt X ℙ n t)
        {ω | δ ≤ |empiricalMean X n ω - a|} := fun n => by
    have h_meas' : MeasurableSet {ω | |empiricalMean X n ω - a| < δ} :=
      measurableSet_lt ((measurable_empiricalMean X h_meas n).sub_const a).abs measurable_const
    rw [← h_compl n, prob_compl_eq_one_sub h_meas',
      ENNReal.sub_sub_cancel ENNReal.one_ne_top prob_le_one]
  simp_rw [h_eq]
  have h_measure_to_zero : Tendsto (fun n => (ℚ_nt X ℙ n t)
      {ω | δ ≤ |empiricalMean X n ω - a|}) atTop (𝓝 0) :=
    (ENNReal.tendsto_toReal_zero_iff (fun n => measure_ne_top _ _)).mp h_complement_to_zero
  have h_sub_to_one : Tendsto (fun n => 1 - (ℚ_nt X ℙ n t)
      {ω | δ ≤ |empiricalMean X n ω - a|}) atTop (𝓝 1) := by
    simpa using ENNReal.Tendsto.sub tendsto_const_nhds h_measure_to_zero
      (Or.inl ENNReal.one_ne_top)
  rw [← ENNReal.toReal_one]
  exact (ENNReal.tendsto_toReal ENNReal.one_ne_top).comp h_sub_to_one
```

```
include h_indep h_ident h_meas h_mgf in
/-- `ℚ_nt(S_n/n ≥ a) > 0`.
This follows from the fact that the mean of `S_n/n` under `ℚ_nt` is `a`, so there must be mass `≥a`
    -/
private lemma tilted_prob_ge_mean_pos (a t : ℝ)
    (ht_deriv : deriv (cgf (X 0) ℙ) t = a) (n : ℕ) (hn : n ≠ 0) :
    0 < (ℚ_nt X ℙ n t) {ω | a ≤ empiricalMean X n ω} := by
  let μ_t := Measure.tilted ℙ (fun ω => t * partialSum X n ω)
  haveI h_prob : IsProbabilityMeasure μ_t :=
    isProbabilityMeasure_tilted_partialSum X h_indep h_ident h_meas h_mgf t n
  have h_mean : μ_t[empiricalMean X n] = a :=
    tilted_empirical_mean_eq_of_deriv X h_indep h_ident h_meas h_mgf t a n hn ht_deriv
  by_contra! h_not_pos
  have h_zero : μ_t {ω | a ≤ empiricalMean X n ω} = 0 := le_antisymm h_not_pos (zero_le _)
  have h_ae_lt : ∀ᵐ ω ∂μ_t, empiricalMean X n ω < a := by
    rw [ae_iff, show {ω | ¬(empiricalMean X n ω < a)} = {ω | a ≤ empiricalMean X n ω} from
      Set.ext fun _ => not_lt, h_zero]
  have h_integrable_em : Integrable (empiricalMean X n) μ_t :=
    (memLp_one_iff_integrable.mp (memLp_tilted_mul
      (mem_interior_integrableExpSet_partialSum X h_indep h_ident h_meas h_mgf t n) 1)).div_const n
  have h_ae_eq : ∀ᵐ ω ∂μ_t, empiricalMean X n ω = a :=
    ae_eq_const_of_integral_eq_const_of_le μ_t a h_integrable_em
      (h_ae_lt.mono fun _ => le_of_lt) h_mean
  exact (h_ae_lt.and h_ae_eq).exists.elim fun _ ⟨hlt, heq⟩ => (heq ▸ hlt).ne rfl

end ProbabilityTheory
```

A.6 Mathlib/Probability/LargeDeviations/Cramers/Theorem.lean

```
module

public import Mathlib.Probability.LargeDeviations.Cramers.LowerBound
public import Mathlib.Probability.LargeDeviations.Cramers.UpperBound
public import Mathlib.Probability.LargeDeviations.Defs
public import Mathlib.Probability.StrongLaw

/-!
# Cramér's Theorem

This file proves the main result:

- `cramers_theorem`: The empirical mean of i.i.d. random variables with finite MGF satisfies
  the Large Deviation Principle with rate function `upperTailRateFunction X`.

The proof combines the upper and lower bounds from `UpperBound.lean` and `LowerBound.lean`,
and handles the case `a < 𝔼[X 0]` using the strong law of large numbers.
-/

open ProbabilityTheory MeasureTheory Filter Topology
open scoped ENNReal

@[expose] public section

namespace ProbabilityTheory

variable {Ω : Type*} [MeasureSpace Ω]
variable (X : ℕ → Ω → ℝ)

/- Assumptions for Cramér's theorem -/
variable (h_indep : iIndepFun X ℙ)
variable (h_ident : ∀ n, IdentDistrib (X n) (X 0) ℙ ℙ)
variable (h_meas : ∀ n, Measurable (X n))
variable (h_int : Integrable (X 0) ℙ)
variable (h_mgf : ∀ t : ℝ, Integrable (fun ω => Real.exp (t * X 0 ω)) ℙ)
variable (h_bdd : ∀ a : ℝ, BddAbove (Set.range (fun t => t * a - cgf (X 0) ℙ t)))
variable (h_non_deg : ∀ t : ℝ, 0 < iteratedDeriv 2 (cgf (X 0) ℙ) t)
variable (h_exposed : ∀ a : ℝ, 𝔼[X 0] ≤ a → ∃ t, deriv (cgf (X 0) ℙ) t = a)
```

```
/-! ### Lemmas requiring `IsProbabilityMeasure` -/

variable [IsProbabilityMeasure (ℙ : Measure Ω)]

include h_indep h_meas h_ident h_int in
/-- If `a < 𝔼[X 0]`, `ℙ(S_n/n ≥ a) → 1` by the strong law of large numbers. -/
private lemma tendsto_prob_empiricalMean_ge_of_lt_mean_one (a : ℝ) (h : a < 𝔼[X 0]) :
  Tendsto (fun n : ℕ => (ℙ {ω | a ≤ empiricalMean X n ω} : ENNReal)) atTop (𝓝 1) := by
  have h_pairwise : Pairwise (fun i j => IndepFun (X i) (X j) ℙ) :=
    fun i j hij => h_indep.indepFun hij
  -- Almost sure convergence: `ℙ(lim_n S_n/n = 𝔼[X]) = 1`
  have h_strong_law :
      ∀ᵐ ω ∂ℙ, Tendsto (fun n : ℕ => empiricalMean X n ω) atTop (𝓝 𝔼[X 0]) := by
    have h_orig := strong_law_ae_real X h_int h_pairwise h_ident
    filter_upwards [h_orig] with ω hω
    have h_eq : (fun n : ℕ => empiricalMean X n ω) =
        (fun n => (Σ i ∈ Finset.range n, X i ω) / n) := by
      ext n
      unfold empiricalMean partialSum
      rw [Finset.sum_apply]
    rwa [h_eq]
  have h_gap : 0 < 𝔼[X 0] - a := sub_pos.mpr h
  set ε := (𝔼[X 0] - a) / 2 with hε_def
  have hε_pos : 0 < ε := by linarith
  have hε_bound : a + ε < 𝔼[X 0] := by linarith
  have h_conv_set : ∀ᵐ ω ∂ℙ, ∀ᶠ n in atTop, |empiricalMean X n ω - 𝔼[X 0]| < ε := by
    filter_upwards [h_strong_law] with ω hω
    have h_tend := (Metric.tendsto_nhds.mp hω) ε hε_pos
    filter_upwards [h_tend] with n hn
    rwa [Real.dist_eq] at hn
  have h_eventually_large : ∀ᵐ ω ∂ℙ, ∀ᶠ n in atTop, a ≤ empiricalMean X n ω := by
    filter_upwards [h_conv_set] with ω hω
    filter_upwards [hω] with n hn
    have : 𝔼[X 0] - ε < empiricalMean X n ω := by
      rw [abs_sub_lt_iff] at hn
      linarith
    linarith
  let S : ℕ → Set Ω := fun k => {ω | ∀ n ≥ k, a ≤ empiricalMean X n ω}
  have h_mono : Monotone S := by
    intro k₁ k₂ hk ω hω n hn
    exact hω n (le_trans hk hn)
  have h_union : ⋃ k, S k = {ω | ∀ᶠ n in atTop, a ≤ empiricalMean X n ω} := by
    ext ω
    simp only [Set.mem_iUnion, Set.mem_setOf_eq, Filter.eventually_atTop, S]
  have h_union_meas : ℙ (⋃ k, S k) = 1 := by
    have h_union_meas_set : MeasurableSet (⋃ k, S k) := by
      refine MeasurableSet.iUnion fun k => ?_
      change MeasurableSet {ω | ∀ n ≥ k, a ≤ empiricalMean X n ω}
      have : {ω | ∀ n ≥ k, a ≤ empiricalMean X n ω} =
          ⋂ n, ⋂ (_ : k ≤ n), {ω | a ≤ empiricalMean X n ω} := by
        ext; simp
      rw [this]
      refine MeasurableSet.iInter fun n => MeasurableSet.iInter fun _ => ?_
      exact measurableSet_le measurable_const (measurable_empiricalMean X h_meas n)
    rw [h_union]
```

```
  have h_compl : ℙ {ω | ¬∀ᶠ n in atTop, a ≤ empiricalMean X n ω} = 0 := by
    rw [← ae_iff]
    exact h_eventually_large
  have h_compl_eq : {ω | ∀ᶠ n in atTop, a ≤ empiricalMean X n ω}ᶜ =
      {ω | ¬∀ᶠ n in atTop, a ≤ empiricalMean X n ω} := by
    ext; simp
  rw [← prob_add_prob_compl (μ := ℙ) (h_union ▸ h_union_meas_set), h_compl_eq, h_compl,
   add_zero]
have h_tend_S : Tendsto (fun k => ℙ (S k)) atTop (𝓝 1) := by
  have := tendsto_measure_iUnion_atTop (μ := ℙ) h_mono
  rw [h_union_meas] at this
  exact this
have h_superset : ∀ k n, k ≤ n → S k ⊆ {ω | a ≤ empiricalMean X n ω} := by
  intro k n hkn ω hω
  exact hω n hkn
have h_measure_ge : ∀ k n, k ≤ n → ℙ (S k) ≤ ℙ {ω | a ≤ empiricalMean X n ω} := by
  intro k n hkn
  exact measure_mono (h_superset k n hkn)
have h_measure_le : ∀ n, ℙ {ω | a ≤ empiricalMean X n ω} ≤ 1 := by
  intro n
  exact prob_le_one
refine tendsto_of_tendsto_of_tendsto_of_le_of_le h_tend_S tendsto_const_nhds
  (fun n => h_measure_ge n n le_rfl) h_measure_le
```

```
include h_indep h_meas h_ident h_mgf h_int h_bdd h_non_deg h_exposed in
/-- **Cramér's Theorem**: For iid. random variables with finite MGF, the empirical mean satisfies
 a Large Deviation Principle with rate function being the Legendre transform of the CGF. -/
theorem cramers_theorem :
    LargeDeviationPrinciple (empiricalMean X) (upperTailRateFunction X) := by
  constructor
  · intro a
    by_cases h : 𝔼[X 0] ≤ a
    · rw [upperTailRateFunction, if_pos h]
      exact cramer_upper_bound X h_indep h_ident h_meas h_int h_mgf h_bdd a h
    · norm_cast
      rw [upperTailRateFunction, if_neg h]
      have h_prob_bound_2 : ∀ n : ℕ, n ≠ 0 →
          1 / ↑n * (ℙ {ω | empiricalMean X n ω ≥ a}).log ≤ 0 := by
        intro n h_n_nonneg
        have h_log_nonpos : (ℙ {ω | empiricalMean X n ω ≥ a}).log ≤ 0 := by
          rw [ENNReal.log_le_zero_iff]
          exact prob_le_one
        rw [EReal.mul_nonpos_iff]
        left
        constructor
        · rw [div_eq_mul_inv, one_mul]
          apply EReal.inv_nonneg_of_nonneg
          have : 0 < (n : ℝ) := Nat.cast_pos.mpr (Nat.pos_of_ne_zero h_n_nonneg)
          exact EReal.coe_nonneg.mpr (le_of_lt this)
        · exact h_log_nonpos
      simp only [neg_zero]
      apply Filter.limsup_le_of_le
      · exact isCoboundedUnder_le_of_le atTop (fun _ => bot_le)
      · apply Filter.eventually_atTop.mpr
        use 1
        intro n hn
        exact h_prob_bound_2 n (Nat.one_le_iff_ne_zero.mp hn)
  · intro a
    by_cases h : 𝔼[X 0] ≤ a
    · rw [upperTailRateFunction, if_pos h]
      exact cramer_lower_bound X h_indep h_ident h_meas h_mgf h_bdd h_non_deg h_exposed a h
    · rw [upperTailRateFunction, if_neg h]
      norm_cast
      rw [neg_zero]
      have h_a_lt_mean : a < 𝔼[X 0] := not_le.mp h
      have h_prob_to_one :
          Tendsto (fun n => (ℙ {ω | a ≤ empiricalMean X n ω} : ENNReal)) atTop (𝓝 1) :=
        tendsto_prob_empiricalMean_ge_of_lt_mean_one X h_indep h_ident h_meas h_int a
          h_a_lt_mean
      have h_seq_to_zero : Tendsto (fun (n : ℕ) =>
          1 / ((n : ℝ) : EReal) * (ℙ {ω | empiricalMean X n ω ≥ a}).log) atTop
          (𝓝 (0 : EReal)) := by
        have h_log_to_zero : Tendsto (fun n => (ℙ {ω | empiricalMean X n ω ≥ a}).log) atTop
            (𝓝 (0 : EReal)) := by
          simpa [ENNReal.log_one] using (ENNReal.continuous_log.tendsto 1).comp h_prob_to_one
        have h_inv_to_zero : Tendsto (fun n : ℕ => 1 / ((n : ℝ) : EReal)) atTop (𝓝 0) := by
          simpa using ereal_inv_nat_mul_const_tendsto_zero (1 : ℝ)
        simpa using EReal.Tendsto.mul h_inv_to_zero h_log_to_zero
          (Or.inr (EReal.coe_ne_bot 0)) (Or.inr (EReal.coe_ne_top 0))
          (Or.inl (EReal.coe_ne_bot 0)) (Or.inl (EReal.coe_ne_top 0))
```

```
      have h_lim_eq : liminf (fun (n : ℕ) =>
          1 / ((n : ℝ) : EReal) * (ℙ {ω | empiricalMean X n ω ≥ a}).log) atTop
          = (0 : EReal) := Filter.Tendsto.liminf_eq h_seq_to_zero
      have : liminf (fun n : ℕ =>
          1 / (n : EReal) * (ℙ {ω | empiricalMean X n ω ≥ a}).log) atTop = 0 := by
        convert h_lim_eq using 2
      rw [this]
      norm_cast

end ProbabilityTheory
```

B Mathlib Definitions and Theorems

In this appendix, we list verbatim several existing definitions and theorems from the `mathlib` library that are used in the proofs listed above.

B.1 Probability Theory

B.1.1 Mathlib/Probability/Moments/Basic.lean

```
/-- Moment-generating function of a real random variable `X`: `fun t => μ[exp(t*X)]`. -/
def mgf (X : Ω → ℝ) (μ : Measure Ω) (t : ℝ) : ℝ :=
  μ[fun ω => exp (t * X ω)]

/-- Cumulant-generating function of a real random variable `X`: `fun t => log μ[exp(t*X)]`. -/
def cgf (X : Ω → ℝ) (μ : Measure Ω) (t : ℝ) : ℝ :=
  log (mgf X μ t)

/-- **Chernoff bound** on the upper tail of a real random variable. -/
theorem measure_ge_le_exp_cgf [IsFiniteMeasure μ] (ε : ℝ) (ht : 0 ≤ t)
    (h_int : Integrable (fun ω => exp (t * X ω)) μ) :
    μ.real {ω | ε ≤ X ω} ≤ exp (-t * ε + cgf X μ t)

@[simp]
theorem cgf_zero [IsZeroOrProbabilityMeasure μ] : cgf X μ 0 = 0

theorem mgf_sum_of_identDistrib
    {X : ι → Ω → ℝ}
    {s : Finset ι} {j : ι}
    (h_meas : ∀ i, Measurable (X i))
    (h_indep : iIndepFun X μ)
    (hident : ∀ i ∈ s, ∀ j ∈ s, IdentDistrib (X i) (X j) μ μ)
    (hj : j ∈ s) (t : ℝ) : mgf (Σ i ∈ s, X i) μ t = mgf (X j) μ t ^ #s

lemma mgf_id_map (hX : AEMeasurable X μ) : mgf id (μ.map X) = mgf X μ
```

B.1.2 Mathlib/MeasureTheory/Measure/Tilted.lean

```
/-- Exponentially tilted measure. When `x ↦ exp (f x)` is integrable, `μ.tilted f` is the
probability measure with density with respect to `μ` proportional to `exp (f x)`. Otherwise it is
    0.
-/
noncomputable
def Measure.tilted (μ : Measure α) (f : α → ℝ) : Measure α :=
  μ.withDensity (fun x ↦ ENNReal.ofReal (exp (f x) / ∫ x, exp (f x) ∂μ))

lemma isProbabilityMeasure_tilted [NeZero μ]
    (hf : Integrable (fun x ↦ exp (f x)) μ) :
    IsProbabilityMeasure (μ.tilted f)

@[simp]
lemma tilted_zero' (μ : Measure α) : μ.tilted 0 = (μ Set.univ)⁻¹ • μ

lemma integrable_tilted_iff {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    {f : α → ℝ} (hf : Integrable (fun x ↦ exp (f x)) μ) (g : α → E) :
    Integrable g (μ.tilted f) ↔ Integrable (fun x ↦ exp (f x) • g x) μ
```

```
lemma integral_tilted (f : α → ℝ) (g : α → E) :
    ∫ x, g x ∂(μ.tilted f) = ∫ x, (exp (f x) / ∫ x, exp (f x) ∂μ) • (g x) ∂μ

lemma integral_exp_tilted (f g : α → ℝ) :
    ∫ x, exp (g x) ∂(μ.tilted f) = (∫ x, exp ((f + g) x) ∂μ) / ∫ x, exp (f x) ∂μ

lemma setIntegral_tilted' (f : α → ℝ) (g : α → E) {s : Set α} (hs : MeasurableSet s) :
    ∫ x in s, g x ∂(μ.tilted f) = ∫ x in s, (exp (f x) / ∫ x, exp (f x) ∂μ) • (g x) ∂μ

lemma lintegral_tilted (f : α → ℝ) (g : α → ℝ≥0∞) :
    ∫⁻ x, g x ∂(μ.tilted f)
      = ∫⁻ x, ENNReal.ofReal (exp (f x) / ∫ x, exp (f x) ∂μ) * (g x) ∂μ

lemma tilted_apply' (μ : Measure α) (f : α → ℝ) {s : Set α} (hs : MeasurableSet s) :
    μ.tilted f s = ∫⁻ a in s, ENNReal.ofReal (exp (f a) / ∫ x, exp (f x) ∂μ) ∂μ
```

B.1.3 Mathlib/Probability/Moments/Tilted.lean

```
/-- The integral of `X` against the tilted measure `μ.tilted (t * X ·)` is the first derivative of
the cumulant-generating function of `X` at `t`. -/
lemma integral_tilted_mul_self (ht : t ∈ interior (integrableExpSet X μ)) :
    (μ.tilted (t * X ·))[X] = deriv (cgf X μ) t

lemma memLp_tilted_mul (ht : t ∈ interior (integrableExpSet X μ)) (p : ℝ≥0) :
    MemLp X p (μ.tilted (t * X ·))

/-- The variance of `X` under the tilted measure `μ.tilted (t * X ·)` is the second derivative of
the cumulant-generating function of `X` at `t`. -/
lemma variance_tilted_mul (ht : t ∈ interior (integrableExpSet X μ)) :
    Var[X; μ.tilted (t * X ·)] = iteratedDeriv 2 (cgf X μ) t
```

B.1.4 Mathlib/Probability/StrongLaw.lean

```
/-- **Strong law of large numbers**, almost sure version: if `X n` is a sequence of independent
identically distributed integrable real-valued random variables, then `Σ i ∈ range n, X i / n`
converges almost surely to `𝔼[X 0]`. We give here the strong version, due to Etemadi, that only
requires pairwise independence. Superseded by `strong_law_ae`, which works for random variables
taking values in any Banach space. -/
theorem strong_law_ae_real {Ω : Type*} {m : MeasurableSpace Ω} {μ : Measure Ω}
    (X : ℕ → Ω → ℝ) (hint : Integrable (X 0) μ)
    (hindep : Pairwise ((· ⊥ᵢ[μ] ·) on X))
    (hident : ∀ i, IdentDistrib (X i) (X 0) μ μ) :
    ∀ᵐ ω ∂μ, Tendsto (fun n : ℕ => (Σ i ∈ range n, X i ω) / n) atTop (𝓝 μ[X 0])
```

B.1.5 Mathlib/Probability/Moments/Variance.lean

```
/-- The ‘ℝ‘-valued variance of a real-valued random variable defined by applying ‘ENNReal.toReal‘
to ‘evariance‘. -/
def variance : ℝ := (evariance X μ).toReal

/-- **Chebyshev's inequality**: one can control the deviation probability of a real random variable
from its expectation in terms of the variance. -/
theorem meas_ge_le_variance_div_sq [IsFiniteMeasure μ] {X : Ω → ℝ} (hX : MemLp X 2 μ) {c : ℝ}
    (hc : 0 < c) : μ {ω | c ≤ |X ω - μ[X]|} ≤ ENNReal.ofReal (variance X μ / c ^ 2)

theorem variance_smul (c : ℝ) (X : Ω → ℝ) (μ : Measure Ω) :
    variance (c · X) μ = c ^ 2 * variance X μ

lemma variance_eq_integral (hX : AEMeasurable X μ) : Var[X; μ] = ∫ ω, (X ω - μ[X]) ^ 2 ∂μ
```

B.1.6 Mathlib/Probability/Moments/IntegrableExpMul.lean

```
/-- The interval of reals ‘t‘ for which ‘exp (t * X)‘ is integrable. -/
def integrableExpSet (X : Ω → ℝ) (μ : Measure Ω) : Set ℝ :=
  {t | Integrable (fun ω ↦ exp (t * X ω)) μ}
```

B.1.7 Mathlib/Probability/Independence/Basic.lean

```
/-- A family of functions defined on the same space ‘Ω‘ and taking values in possibly different
spaces, each with a measurable space structure, is independent if the family of measurable space
structures they generate on ‘Ω‘ is independent. For a function ‘g‘ with codomain having measurable
space structure ‘m‘, the generated measurable space structure is ‘MeasurableSpace.comap g m‘. -/
def iIndepFun {_mΩ : MeasurableSpace Ω} {β : ι → Type*} [m : ∀ x : ι, MeasurableSpace (β x)]
    (f : ∀ x : ι, Ω → β x) (μ : Measure Ω := by volume_tac) : Prop :=
  Kernel.iIndepFun f (Kernel.const Unit μ) (Measure.dirac () : Measure Unit)

/-- Two functions are independent if the two measurable space structures they generate are
independent. For a function ‘f‘ with codomain having measurable space structure ‘m‘, the generated
measurable space structure is ‘MeasurableSpace.comap f m‘.
We use the notation ‘f ⊥ᵢ[μ] g‘ for ‘IndepFun f g μ‘ (scoped in ‘ProbabilityTheory‘). -/
def IndepFun {β γ} {_mΩ : MeasurableSpace Ω} [MeasurableSpace β] [MeasurableSpace γ]
    (f : Ω → β) (g : Ω → γ) (μ : Measure Ω := by volume_tac) : Prop :=
  Kernel.IndepFun f g (Kernel.const Unit μ) (Measure.dirac () : Measure Unit)

nonrec lemma iIndepFun.comp {β γ : ι → Type*} {mβ : ∀ i, MeasurableSpace (β i)}
    {mγ : ∀ i, MeasurableSpace (γ i)} {f : ∀ i, Ω → β i}
    (h : iIndepFun f μ) (g : ∀ i, β i → γ i) (hg : ∀ i, Measurable (g i)) :
    iIndepFun (fun i ↦ g i ∘ f i) μ

theorem iIndepFun.indepFun {β : ι → Type*}
    {m : ∀ x, MeasurableSpace (β x)} {f : ∀ i, Ω → β i} (hf_Indep : iIndepFun f μ) {i j : ι}
    (hij : i ≠ j) :
    f i ⊥ᵢ[μ] f j

@[to_additive]
lemma iIndepFun.indepFun_finset_prod_of_notMem (hf_Indep : iIndepFun f μ)
    (hf_meas : ∀ i, Measurable (f i)) {s : Finset ι} {i : ι} (hi : i ∉ s) :
    IndepFun (∏ j ∈ s, f j) (f i) μ
```

B.1.8 Mathlib/Probability/Independence/Integration.lean

```
/-- The product of two independent, integrable, real-valued random variables is integrable. -/
theorem IndepFun.integrable_mul {β : Type*} [MeasurableSpace β] {X Y : Ω → β}
    [NormedDivisionRing β] [BorelSpace β] (hXY : X ⟂ᵢ[μ] Y) (hX : Integrable X μ)
    (hY : Integrable Y μ) : Integrable (X * Y) μ

theorem lintegral_prod_eq_prod_lintegral_of_indepFun {ι : Type*}
    (s : Finset ι) (X : ι → Ω → ℝ≥0∞) (hX : iIndepFun X μ)
    (x_mea : ∀ i, Measurable (X i)) :
    ∫⁻ ω, ∏ i ∈ s, (X i ω) ∂μ = ∏ i ∈ s, ∫⁻ ω, X i ω ∂μ
```

B.1.9 Mathlib/Probability/IdentDistrib.lean

```
/-- Two functions defined on two (possibly different) measure spaces are identically distributed if
their image measures coincide. This only makes sense when the functions are ae measurable
(as otherwise the image measures are not defined), so we require this as well in the definition. -/
structure IdentDistrib (f : α → γ) (g : β → γ)
    (μ : Measure α := by volume_tac)
    (ν : Measure β := by volume_tac) : Prop where
  aemeasurable_fst : AEMeasurable f μ
  aemeasurable_snd : AEMeasurable g ν
  map_eq : Measure.map f μ = Measure.map g ν

protected theorem IdentDistrib.comp {u : γ → δ} (h : IdentDistrib f g μ ν) (hu : Measurable u) :
    IdentDistrib (u ∘ f) (u ∘ g) μ ν

protected theorem IdentDistrib.symm (h : IdentDistrib f g μ ν) : IdentDistrib g f ν μ

protected theorem IdentDistrib.trans {ρ : Measure δ} {h : δ → γ} (h₁ : IdentDistrib f g μ ν)
    (h₂ : IdentDistrib g h ν ρ) : IdentDistrib f h μ ρ

theorem IdentDistrib.integrable_iff [NormedAddCommGroup γ] [BorelSpace γ]
    (h : IdentDistrib f g μ ν) :
    Integrable f μ ↔ Integrable g ν
```

B.1.10 Mathlib/Probability/Moments/MGFAnalytic.lean

```
/-- The cumulant-generating function is analytic on the interior of the interval
  `integrableExpSet X μ`. -/
lemma analyticOn_cgf :
    AnalyticOn ℝ (cgf X μ) (interior (integrableExpSet X μ))

lemma deriv_cgf (h : v ∈ interior (integrableExpSet X μ)) :
    deriv (cgf X μ) v =
      μ[fun ω ↦ X ω * exp (v * X ω)] / mgf X μ v

lemma deriv_cgf_zero (h : 0 ∈ interior (integrableExpSet X μ)) :
    deriv (cgf X μ) 0 = μ[X] / μ.real Set.univ
```

B.1.11 Mathlib/MeasureTheory/Measure/Typeclasses/Probability.lean

```
lemma prob_le_one {μ : Measure α} [IsZeroOrProbabilityMeasure μ] {s : Set α} : μ s ≤ 1

theorem prob_add_prob_compl [IsProbabilityMeasure μ] (h : MeasurableSet s) : μ s + μ sᶜ = 1

/-- Note that this is not quite as useful as it looks because the measure takes values in `ℝ≥0∞`.
Thus the subtraction appearing is the truncated subtraction of `ℝ≥0∞`, rather than the
better-behaved subtraction of `ℝ`. -/
theorem prob_compl_eq_one_sub (hs : MeasurableSet s) : μ sᶜ = 1 - μ s

@[simp] lemma probReal_univ : μ.real univ = 1
```

B.1.12 Mathlib/MeasureTheory/Measure/ProbabilityMeasure.lean

```
/-- Probability measures are defined as the subtype of measures that have the property of being
probability measures (i.e., their total mass is one). -/
def ProbabilityMeasure (Ω : Type*) [MeasurableSpace Ω] : Type _ :=
  { μ : Measure Ω // IsProbabilityMeasure μ }

theorem Measure.isProbabilityMeasure_map {f : α → β} (hf : AEMeasurable f μ) :
    IsProbabilityMeasure (map f μ)
```

B.2 Characteristic Functions and Limit Theorems

B.2.1 Mathlib/MeasureTheory/Measure/CharacteristicFunction/Basic.lean

```
/-- The characteristic function of a measure in an inner product space. -/
noncomputable def charFun [Inner ℝ E] (μ : Measure E) (t : E) : ℂ := ∫ x, exp (⟪x, t⟫ * I) ∂μ

lemma charFun_map_mul_comp {X : Type*} {mX : MeasurableSpace X} {μ : Measure X}
    {f : X → ℝ} (hf : AEMeasurable f μ) (r t : ℝ) :
    charFun (μ.map (fun x ↦ r * (f x))) t = charFun (μ.map f) (r * t)
```

B.2.2 Mathlib/Probability/Independence/CharacteristicFunction.lean

```
lemma iIndepFun.charFun_map_fun_finset_sum_eq_prod [InnerProductSpace ℝ E]
    (mX : ∀ i ∈ s, AEMeasurable (X i) P) (hX : iIndepFun (s.restrict X) P) :
    charFun (P.map (fun ω ↦ Σ i ∈ s, X i ω)) = ∏ i ∈ s, charFun (P.map (X i))
```

B.2.3 Mathlib/Probability/CentralLimitTheorem.lean

```
lemma tendsto_charFun_inv_sqrt_mul_pow {X : Ω → ℝ}
    (hX : AEMeasurable X P) (h0 : P[X] = 0) (h1 : P[X ^ 2] = 1) (t : ℝ) :
    Tendsto (fun (n : ℕ) ↦ (charFun (P.map X) ((√n)⁻¹ * t)) ^ n) atTop (𝓝 (exp (- t ^ 2 / 2)))
```

B.2.4 Mathlib/MeasureTheory/Measure/LevyConvergence.lean

```
/-- The **Lévy convergence theorem**: the weak convergence of probability measures is equivalent
to the pointwise convergence of their characteristic functions. -/
theorem ProbabilityMeasure.tendsto_iff_tendsto_charFun :
    Tendsto μ atTop (𝓝 μ₀) ↔
      ∀ t : E, Tendsto (fun n ↦ charFun (μ n) t) atTop (𝓝 (charFun μ₀ t))
```

B.2.5 Mathlib/MeasureTheory/Measure/Portmanteau.lean

```
theorem ProbabilityMeasure.tendsto_measure_of_null_frontier_of_tendsto' {Ω ι : Type*}
    {L : Filter ι} [MeasurableSpace Ω] [TopologicalSpace Ω] [OpensMeasurableSpace Ω]
    [HasOuterApproxClosed Ω] {μ : ProbabilityMeasure Ω} {μs : ι → ProbabilityMeasure Ω}
    (μs_lim : Tendsto μs L (𝓝 μ)) {E : Set Ω} (E_nullbdry : (μ : Measure Ω) (frontier E) = 0) :
    Tendsto (fun i ↦ (μs i : Measure Ω) E) L (𝓝 ((μ : Measure Ω) E))
```

B.2.6 Mathlib/Probability/Distributions/Gaussian/Real.lean

```
/-- A Gaussian distribution on `ℝ` with mean `μ` and variance `v`. -/
noncomputable
def gaussianReal (μ : ℝ) (v : ℝ≥0) : Measure ℝ :=
  if v = 0 then Measure.dirac μ else volume.withDensity (gaussianPDF μ v)

/-- The characteristic function of a Gaussian distribution with mean `μ` and variance `v`
is given by `t ↦ exp (t * μ - v * t ^ 2 / 2)`. -/
theorem charFun_gaussianReal (t : ℝ) :
    charFun (gaussianReal μ v) t = cexp (t * μ * I - v * t ^ 2 / 2)

lemma noAtoms_gaussianReal {μ : ℝ} {v : ℝ≥0} (h : v ≠ 0) : NoAtoms (gaussianReal μ v)

lemma gaussianReal_map_neg : (gaussianReal μ v).map (fun x ↦ -x) = gaussianReal (-μ) v
```

B.3 Measure Theory

B.3.1 Mathlib/MeasureTheory/Measure/Typeclasses/Probability.lean

```
/-- A measure `μ` is called a probability measure if `μ univ = 1`. -/
class IsProbabilityMeasure (μ : Measure α) : Prop where
  measure_univ : μ univ = 1
```

B.3.2 Mathlib/MeasureTheory/Measure/Typeclasses/Finite.lean

```
theorem measure_ne_top (μ : Measure α) [IsFiniteMeasure μ] (s : Set α) : μ s ≠ ∞
```

B.3.3 Mathlib/MeasureTheory/Measure/Typeclasses/NoAtoms.lean

```
/-- Measure `μ` *has no atoms* if the measure of each singleton is zero.

NB: Wikipedia assumes that for any measurable set `s` with positive `μ`-measure,
there exists a measurable `t ⊆ s` such that `0 < μ t < μ s`. While this implies `μ {x} = 0`,
the converse is not true. -/
class NoAtoms {m0 : MeasurableSpace α} (μ : Measure α) : Prop where
  measure_singleton : ∀ x, μ {x} = 0

theorem Iio_ae_eq_Iic : Iio a =ᵐ[μ] Iic a
theorem Ioi_ae_eq_Ici : Ioi a =ᵐ[μ] Ici a
```

B.3.4 Mathlib/MeasureTheory/MeasurableSpace/Defs.lean

```
@[class] structure MeasurableSpace (α : Type*) where
  /-- Predicate saying that a given set is measurable. Use ‘MeasurableSet‘ in the root namespace
  instead. -/
  MeasurableSet' : Set α → Prop
  /-- The empty set is a measurable set. Use ‘MeasurableSet.empty‘ instead. -/
  measurableSet_empty : MeasurableSet' ∅
  /-- The complement of a measurable set is a measurable set. Use ‘MeasurableSet.compl‘ instead. -/
  measurableSet_compl : ∀ s, MeasurableSet' s → MeasurableSet' sᶜ
  /-- The union of a sequence of measurable sets is a measurable set. Use a more general
  ‘MeasurableSet.iUnion‘ instead. -/
  measurableSet_iUnion : ∀ f : ℕ → Set α, (∀ i, MeasurableSet' (f i)) → MeasurableSet' (⋃ i, f
    i)

/-- ‘MeasurableSet s‘ means that ‘s‘ is measurable (in the ambient measure space on ‘α‘) -/
def MeasurableSet [MeasurableSpace α] (s : Set α) : Prop :=
  ⟨MeasurableSpace α⟩.MeasurableSet' s

/-- A function ‘f‘ between measurable spaces is measurable if the preimage of every
  measurable set is measurable. -/
@[fun_prop]
def Measurable [MeasurableSpace α] [MeasurableSpace β] (f : α → β) : Prop :=
  ∀ {t : Set β}, MeasurableSet t → MeasurableSet (f ⁻¹' t)

@[simp, fun_prop]
theorem measurable_const {_ : MeasurableSpace α} {_ : MeasurableSpace β} {a : α} :
    Measurable fun _ : β => a

theorem measurable_id {_ : MeasurableSpace α} : Measurable (@id α)

@[measurability]
protected theorem MeasurableSet.iUnion [Countable ι] {f : ι → Set α}
    (h : ∀ b, MeasurableSet (f b)) : MeasurableSet (⋃ b, f b)
@[measurability]
theorem MeasurableSet.iInter [Countable ι] {f : ι → Set α} (h : ∀ b, MeasurableSet (f b)) :
    MeasurableSet (⋂ b, f b)
```

B.3.5 Mathlib/MeasureTheory/Group/Arithmetic.lean

```
@[to_additive (attr := fun_prop)]
theorem Measurable.mul [MeasurableMul₂ M] (hf : Measurable f) (hg : Measurable g) :
    Measurable fun a => f a * g a
@[to_additive (attr := fun_prop)]
theorem Measurable.const_mul [MeasurableMul M] (hf : Measurable f) (c : M) :
    Measurable fun x => c * f x
@[to_additive (attr := fun_prop)]
theorem Measurable.div_const [MeasurableDiv G] (hf : Measurable f) (c : G) :
    Measurable fun x => f x / c

@[to_additive (attr := measurability, fun_prop)]
theorem Finset.measurable_prod (s : Finset ι) (hf : ∀ i ∈ s, Measurable (f i)) :
    Measurable fun a ↦ ∏ i ∈ s, f i a
```

B.3.6 Mathlib/MeasureTheory/Function/SpecialFunctions/Basic.lean

```
@[fun_prop]
protected theorem Measurable.exp : Measurable fun x => Real.exp (f x)
```

B.3.7 Mathlib/MeasureTheory/Constructions/BorelSpace/Order.lean

```
@[simp, measurability]
theorem measurableSet_Icc [OrderClosedTopology α] : MeasurableSet (Icc a b)

theorem measurableSet_lt [SecondCountableTopology α] [OrderClosedTopology α]
    {f g : δ → α} (hf : Measurable f) (hg : Measurable g) :
    MeasurableSet { a | f a < g a }

theorem measurableSet_le {f g : δ → α} (hf : Measurable f) (hg : Measurable g) :
    MeasurableSet { a | f a ≤ g a }
```

B.3.8 Mathlib/MeasureTheory/Function/L1Space/Integrable.lean

```
/-- ‘Integrable f μ‘ means that ‘f‘ is measurable and that the integral ‘∫⁻ a, ‖f a‖ ∂μ‘ is
    finite.
  ‘Integrable f‘ means ‘Integrable f volume‘. -/
@[fun_prop]
def Integrable {α} {_ : MeasurableSpace α} (f : α → ε)
    (μ : Measure α := by volume_tac) : Prop :=
  AEStronglyMeasurable f μ ∧ HasFiniteIntegral f μ

@[fun_prop]
theorem integrable_const [IsFiniteMeasure μ] (c : β) : Integrable (fun _ : α => c) μ

theorem integrable_map_measure {f : α → α'} {g : α' → ε}
    (hg : AEStronglyMeasurable g (Measure.map f μ)) (hf : AEMeasurable f μ) :
    Integrable g (Measure.map f μ) ↔ Integrable (g ∘ f) μ

@[fun_prop]
theorem Integrable.const_mul {f : α → 𝕜} (h : Integrable f μ) (c : 𝕜) :
    Integrable (fun x => c * f x) μ
@[fun_prop]
theorem Integrable.div_const {f : α → 𝕜} (h : Integrable f μ) (c : 𝕜) :
    Integrable (fun x => f x / c) μ
@[fun_prop]
theorem Integrable.sub {f g : α → β} (hf : Integrable f μ) (hg : Integrable g μ) :
    Integrable (f - g) μ

theorem MemLp.integrable {q : ℝ≥0∞} (hq1 : 1 ≤ q) {f : α → ε} [IsFiniteMeasure μ]
    (hfq : MemLp f q μ) : Integrable f μ
```

B.3.9 Mathlib/MeasureTheory/Integral/IntegrableOn.lean

```
/-- A function is ‘IntegrableOn‘ a set ‘s‘ if it is almost everywhere strongly measurable on ‘s‘
and if the integral of its pointwise norm over ‘s‘ is less than infinity. -/
def IntegrableOn (f : α → ε) (s : Set α) (μ : Measure α := by volume_tac) : Prop :=
  Integrable f (μ.restrict s)
```

B.3.10 Mathlib/MeasureTheory/Function/LpSeminorm/Defs.lean

```
/-- The property that `f : α → E` is a.e. strongly measurable and `(∫ ‖f a‖ ^ p ∂μ) ^ (1/p)`
is finite if `p < ∞`, or `essSup ‖f‖ < ∞` if `p = ∞`. -/
def MemLp [TopologicalSpace ε] (f : α → ε) (p : ℝ≥0∞) (μ : Measure α := by volume_tac) : Prop :=
  AEStronglyMeasurable f μ ∧ eLpNorm f p μ < ∞
```

B.3.11 Mathlib/MeasureTheory/Function/LpSeminorm/Basic.lean

```
theorem memLp_const (c : E) [IsFiniteMeasure μ] : MemLp (fun _ : α => c) p μ

theorem memLp_map_measure_iff (hg : AEStronglyMeasurable g (Measure.map f μ))
    (hf : AEMeasurable f μ) : MemLp g p (Measure.map f μ) ↔ MemLp (g ∘ f) p μ
```

B.3.12 Mathlib/MeasureTheory/Integral/Bochner/Basic.lean

```
theorem integral_map {β} [MeasurableSpace β] {φ : α → β} (hφ : AEMeasurable φ μ) {f : β → G}
    (hfm : AEStronglyMeasurable f (Measure.map φ μ)) :
    ∫ y, f y ∂Measure.map φ μ = ∫ x, f (φ x) ∂μ

@[simp]
theorem integral_const (c : E) : ∫ _ : α, c ∂μ = μ.real univ • c

theorem integral_sub {f g : α → G} (hf : Integrable f μ) (hg : Integrable g μ) :
    ∫ a, f a - g a ∂μ = ∫ a, f a ∂μ - ∫ a, g a ∂μ

theorem integral_const_mul {L : Type*} [RCLike L] (r : L) (f : α → L) :
    ∫ a, r * f a ∂μ = r * ∫ a, f a ∂μ

theorem integral_div {L : Type*} [RCLike L] (r : L) (f : α → L) :
    ∫ a, f a / r ∂μ = (∫ a, f a ∂μ) / r

lemma integral_exp_pos {μ : Measure α} {f : α → ℝ} [hμ : NeZero μ]
    (hf : Integrable (fun x ↦ Real.exp (f x)) μ) :
    0 < ∫ x, Real.exp (f x) ∂μ

theorem integral_eq_zero_iff_of_nonneg_ae {f : α → ℝ} (hf : 0 ≤ᵐ[μ] f) (hfi : Integrable f μ) :
    ∫ x, f x ∂μ = 0 ↔ f =ᵐ[μ] 0

theorem ofReal_integral_eq_lintegral_ofReal {f : α → ℝ} (hfi : Integrable f μ) (f_nn : 0 ≤ᵐ[μ]
    f) :
    ENNReal.ofReal (∫ x, f x ∂μ) = ∫⁻ x, ENNReal.ofReal (f x) ∂μ
```

B.3.13 Mathlib/MeasureTheory/Integral/Bochner/Set.lean

```
theorem setIntegral_mono_on (hs : MeasurableSet s) (h : ∀ x ∈ s, f x ≤ g x) :
    ∫ x in s, f x ∂μ ≤ ∫ x in s, g x ∂μ

theorem setIntegral_const [CompleteSpace E] (c : E) : ∫ _ in s, c ∂μ = μ.real s • c
```

B.3.14 Mathlib/MeasureTheory/Integral/Lebesgue/Basic.lean

```
theorem Measure.ext_of_lintegral (ν : Measure α)
    (hμν : ∀ f : α → ℝ≥0∞, Measurable f → ∫⁻ a, f a ∂μ = ∫⁻ a, f a ∂ν) : μ = ν
```

B.3.15 Mathlib/MeasureTheory/Integral/Lebesgue/Map.lean

```
theorem lintegral_map {f : β → ℝ≥0∞} {g : α → β} (hf : Measurable f)
    (hg : Measurable g) : ∫⁻ a, f a ∂map g μ = ∫⁻ a, f (g a) ∂μ
```

B.3.16 Mathlib/MeasureTheory/Integral/Lebesgue/Add.lean

```
theorem lintegral_mul_const' (r : ℝ≥0∞) (f : α → ℝ≥0∞) (hr : r ≠ ∞) :
    ∫⁻ a, f a * r ∂μ = (∫⁻ a, f a ∂μ) * r
```

B.3.17 Mathlib/MeasureTheory/Measure/MeasureSpaceDef.lean

```
/-- The real-valued version of a measure. Maps infinite measure sets to zero. Use as `μ.real s`.
The API is developed in `Mathlib/MeasureTheory/Measure/Real.lean`. -/
protected def Measure.real {α : Type*} {m : MeasurableSpace α} (μ : Measure α) (s : Set α) : ℝ :=
  (μ s).toReal
```

B.3.18 Mathlib/MeasureTheory/OuterMeasure/Basic.lean

```
@[mono, gcongr]
theorem measure_mono (h : s ⊆ t) : μ s ≤ μ t
```

B.3.19 Mathlib/MeasureTheory/OuterMeasure/AE.lean

```
theorem ae_iff {p : α → Prop} : (∀ᵐ a ∂μ, p a) ↔ μ { a | ¬p a } = 0

theorem ae_of_all {p : α → Prop} (μ : F) : (∀ a, p a) → ∀ᵐ a ∂μ, p a
```

B.3.20 Mathlib/MeasureTheory/Measure/MeasureSpace.lean

```
/-- Continuity from below: the measure of the union of an increasing sequence of (not necessarily
measurable) sets is the limit of the measures. -/
theorem tendsto_measure_iUnion_atTop [Preorder ι] [IsCountablyGenerated (atTop : Filter ι)]
    {s : ι → Set α} (hm : Monotone s) : Tendsto (μ ∘ s) atTop (𝓝 (μ (⋃ n, s n)))

/-- `μ s + μ sᶜ = μ univ`. -/
theorem measure_add_measure_compl (h : MeasurableSet s) : μ s + μ sᶜ = μ univ

theorem tendsto_measure_Iic_atTop [Preorder α] [(atTop : Filter α).IsCountablyGenerated]
    (μ : Measure α) : Tendsto (fun x => μ (Iic x)) atTop (𝓝 (μ univ))
```

B.3.21 Mathlib/MeasureTheory/Measure/Map.lean

```
@[simp]
theorem map_apply (hf : Measurable f) {s : Set β} (hs : MeasurableSet s) :
    μ.map f s = μ (f ⁻¹' s)

/-- Mapping a measure twice is the same as mapping the measure with the composition. This version
    is
for measurable functions. See ‘map_map_of_aemeasurable‘ when they are just ae measurable. -/
theorem map_map {g : β → γ} {f : α → β} (hg : Measurable g) (hf : Measurable f) :
    (μ.map f).map g = μ.map (g ∘ f)
```

B.4 Analysis

B.4.1 Mathlib/Analysis/Convex/Integral.lean

```
theorem ConvexOn.map_integral_le [IsProbabilityMeasure μ] (hg : ConvexOn ℝ s g)
    (hgc : ContinuousOn g s) (hsc : IsClosed s) (hfs : ∀ᵐ x ∂μ, f x ∈ s) (hfi : Integrable f μ)
    (hgi : Integrable (g ∘ f) μ) : g (∫ x, f x ∂μ) ≤ ∫ x, g (f x) ∂μ

theorem Convex.integral_mem [IsProbabilityMeasure μ] (hs : Convex ℝ s) (hsc : IsClosed s)
    (hf : ∀ᵐ x ∂μ, f x ∈ s) (hfi : Integrable f μ) : (∫ x, f x ∂μ) ∈ s
```

B.4.2 Mathlib/Analysis/Convex/Function.lean

```
/-- Convexity of functions -/
def ConvexOn : Prop :=
  Convex 𝕜 s ∧ ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, y ∈ s → ∀ ⦃a b : 𝕜⦄, 0 ≤ a → 0 ≤ b → a + b = 1 →
    f (a • x + b • y) ≤ a • f x + b • f y

/-- Strict convexity of functions -/
def StrictConvexOn : Prop :=
  Convex 𝕜 s ∧ ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, y ∈ s → x ≠ y → ∀ ⦃a b : 𝕜⦄, 0 < a → 0 < b → a + b =
    1 →
    f (a • x + b • y) < a • f x + b • f y

theorem StrictConvexOn.convexOn {s : Set E} {f : E → β} (hf : StrictConvexOn 𝕜 s f) :
    ConvexOn 𝕜 s f
```

B.4.3 Mathlib/Analysis/Convex/SpecificFunctions/Basic.lean

```
/-- ‘Real.exp‘ is convex on the whole real line. -/
theorem convexOn_exp : ConvexOn ℝ univ exp
```

B.4.4 Mathlib/Analysis/Convex/Deriv.lean

```
/-- Reformulation of `ConvexOn.le_slope_of_hasDerivAt` using `deriv` -/
lemma deriv_le_slope (hfc : ConvexOn ℝ S f) (hx : x ∈ S) (hy : y ∈ S) (hxy : x < y)
    (hfd : DifferentiableAt ℝ f x) :
    deriv f x ≤ slope f x y

/-- Reformulation of `ConvexOn.slope_le_of_hasDerivAt` using `deriv`. -/
lemma slope_le_deriv (hfc : ConvexOn ℝ S f) (hx : x ∈ S) (hy : y ∈ S) (hxy : x < y)
    (hfd : DifferentiableAt ℝ f y) :
    slope f x y ≤ deriv f y

/-- If a function `f` is continuous on a convex set `D ⊆ ℝ` and `f''` is strictly positive on `D`,
then `f` is strictly convex on `D`.
Note that we don't require twice differentiability explicitly as it is already implied by the
    second
derivative being strictly positive, except at at most one point. -/
theorem strictConvexOn_of_deriv2_pos' {D : Set ℝ} (hD : Convex ℝ D) {f : ℝ → ℝ}
    (hf : ContinuousOn f D) (hf'' : ∀ x ∈ D, 0 < (deriv^[2] f) x) : StrictConvexOn ℝ D f
```

B.4.5 Mathlib/Analysis/SpecialFunctions/Exp.lean

```
@[continuity]
theorem continuous_exp : Continuous exp
```

B.4.6 Mathlib/Analysis/Complex/Exponential.lean

```
theorem exp_sum {α : Type*} (s : Finset α) (f : α → ℝ) :
    exp (∑ x ∈ s, f x) = ∏ x ∈ s, exp (f x)

@[bound]
theorem exp_pos (x : ℝ) : 0 < exp x
```

B.4.7 Mathlib/Analysis/SpecialFunctions/Log/Basic.lean

```
/-- The real logarithm function, equal to the inverse of the exponential for `x > 0`,
to `log |x|` for `x < 0`, and to `0` for `0`. We use this unconventional extension to
`(-∞, 0]` as it gives the formula `log (x * y) = log x + log y` for all nonzero `x` and `y`, and
the derivative of `log` is `1/x` away from `0`. -/
@[pp_nodot]
noncomputable def log (x : ℝ) : ℝ :=
  if hx : x = 0 then 0 else expOrderIso.symm ⟨|x|, abs_pos.2 hx⟩

@[simp, push]
theorem log_exp (x : ℝ) : log (exp x) = x
theorem exp_log (hx : 0 < x) : exp (log x) = x

@[gcongr, bound]
lemma log_le_log (hx : 0 < x) (hxy : x ≤ y) : log x ≤ log y
```

B.4.8 Mathlib/Data/Real/Sqrt.lean

```
/-- The square root of a real number. This returns 0 for negative inputs.

This has notation `√x`. Note that `√x⁻¹` is parsed as `√(x⁻¹)`. -/
@[irreducible] noncomputable def sqrt (x : ℝ) : ℝ :=
  NNReal.sqrt (Real.toNNReal x)

@[simp]
theorem sqrt_pos : 0 < √x ↔ 0 < x
@[simp]
theorem sqrt_mul {x : ℝ} (hx : 0 ≤ x) (y : ℝ) : √(x * y) = √x * √y
@[simp]
theorem sq_sqrt (h : 0 ≤ x) : √x ^ 2 = x
lemma tendsto_sqrt_atTop : Tendsto (√·) atTop atTop
```

B.4.9 Mathlib/Analysis/Calculus/Deriv/Basic.lean

```
/-- Derivative of `f` at the point `x`, if it exists.  Zero otherwise.

If the derivative exists (i.e., `∃ f', HasDerivAt f f' x`), then
`f x' = f x + (x' - x) · deriv f x + o(x' - x)` where `x'` converges to `x`.
-/
def deriv (f : 𝕜 → F) (x : 𝕜) :=
  fderiv 𝕜 f x 1

/-- `f` has the derivative `f'` at the point `x`.

That is, `f x' = f x + (x' - x) · f' + o(x' - x)` where `x'` converges to `x`.
-/
def HasDerivAt (f : 𝕜 → F) (f' : F) (x : 𝕜) :=
  HasDerivAtFilter f f' (𝓝 x ×ˢ pure x)
```

B.4.10 Mathlib/Analysis/Calculus/Deriv/MeanValue.lean

```
/-- Let `f : ℝ → ℝ` be a differentiable function. If `f'` is positive, then
`f` is a strictly monotone function.
Note that we don't require differentiability explicitly as it already implied by the derivative
being strictly positive. -/
theorem strictMono_of_deriv_pos {f : ℝ → ℝ} (hf' : ∀ x, 0 < deriv f x) : StrictMono f
```

B.4.11 Mathlib/Analysis/Calculus/FDeriv/Defs.lean

```
/-- A function `f` is differentiable at a point `x` if it admits a derivative there (possibly
non-unique). -/
@[fun_prop]
def DifferentiableAt (f : E → F) (x : E) :=
  ∃ f' : E →L[𝕜] F, HasFDerivAt f f' x
```

B.4.12 Mathlib/Analysis/Calculus/ContDiff/Defs.lean

```
/-- A function is continuously differentiable up to `n` at a point `x` if, for any integer `k ≤
    n`,
there is a neighborhood of `x` where `f` admits derivatives up to order `n`, which are continuous.
The parameter `n` belongs to `WithTop ℕ∞`, i.e., it can be a natural number, `∞`, or `ω`
(when the `ContDiff` scope is open).

For `n = ∞`, we only require that this holds up to any finite order (where the neighborhood may
depend on the finite order we consider).
For `n = ω`, we require the function to be analytic at `x`. The precise
definition we give (all the derivatives should be analytic) is more involved to work around issues
when the space is not complete, but it is equivalent when the space is complete.
-/
@[fun_prop]
def ContDiffAt (n : WithTop ℕ∞) (f : E → F) (x : E) : Prop :=
  ContDiffWithinAt 𝕜 n f univ x
```

B.4.13 Mathlib/Analysis/Calculus/IteratedDeriv/Defs.lean

```
/-- The `n`-th iterated derivative of a function from `𝕜` to `F`, as a function from `𝕜` to `F`. -/
def iteratedDeriv (n : ℕ) (f : 𝕜 → F) (x : 𝕜) : F :=
  (iteratedFDeriv 𝕜 n f x : (Fin n → 𝕜) → F) fun _ : Fin n => 1

/-- The `n+1`-th iterated derivative can be obtained by differentiating the `n`-th
iterated derivative. -/
theorem iteratedDeriv_succ : iteratedDeriv (n + 1) f = deriv (iteratedDeriv n f)

/-- The `n`-th iterated derivative can be obtained by iterating `n` times the
differentiation operation. -/
theorem iteratedDeriv_eq_iterate : iteratedDeriv n f = deriv^[n] f
```

B.4.14 Mathlib/Analysis/Analytic/Basic.lean

```
/-- `f` is analytic within `s` if it is analytic within `s` at each point of `s`.  Note that
this is weaker than `AnalyticOnNhd 𝕜 f s`, as `f` is allowed to be arbitrary outside `s`. -/
def AnalyticOn (f : E → F) (s : Set E) : Prop :=
  ∀ x ∈ s, AnalyticWithinAt 𝕜 f s x
```

B.4.15 Mathlib/Analysis/Calculus/FDeriv/Analytic.lean

```
theorem AnalyticOn.differentiableOn (h : AnalyticOn 𝕜 f s) : DifferentiableOn 𝕜 f s
```

B.5 Order and Filters

B.5.1 Mathlib/Order/Filter/Defs.lean

```
/-- ‘Filter.Tendsto‘ is the generic "limit of a function" predicate.
  ‘Tendsto f l₁ l₂‘ asserts that for every ‘l₂‘ neighborhood ‘a‘,
  the ‘f‘-preimage of ‘a‘ is an ‘l₁‘ neighborhood. -/
def Tendsto (f : α → β) (l₁ : Filter α) (l₂ : Filter β) :=
  l₁.map f ≤ l₂
```

B.5.2 Mathlib/Order/LiminfLimsup.lean

```
/-- The ‘limsup‘ of a function ‘u‘ along a filter ‘f‘ is the infimum of the ‘a‘ such that
the inequality ‘u x ≤ a‘ eventually holds for ‘f‘. -/
def limsup (u : β → α) (f : Filter β) : α :=
  limsSup (map u f)

/-- The ‘liminf‘ of a function ‘u‘ along a filter ‘f‘ is the supremum of the ‘a‘ such that
the inequality ‘u x ≥ a‘ eventually holds for ‘f‘. -/
def liminf (u : β → α) (f : Filter β) : α :=
  limsInf (map u f)

theorem limsup_le_of_le {f : Filter β} {u : β → α} {a}
    (hf : f.IsCoboundedUnder (· ≤ ·) u := by isBoundedDefault)
    (h : ∀ᶠ n in f, u n ≤ a) : limsup u f ≤ a

theorem liminf_le_liminf {α : Type*} [ConditionallyCompleteLattice β] {f : Filter α} {u v : α → β
    }
    (h : ∀ᶠ a in f, u a ≤ v a)
    (hu : f.IsBoundedUnder (· ≥ ·) u := by isBoundedDefault)
    (hv : f.IsCoboundedUnder (· ≥ ·) v := by isBoundedDefault) :
    liminf u f ≤ liminf v f
```

B.5.3 Mathlib/Order/Filter/IsBounded.lean

```
lemma isCoboundedUnder_le_of_le [Preorder α] (l : Filter ι) [NeBot l] {f : ι → α} {x : α}
    (hf : ∀ i, x ≤ f i) :
    IsCoboundedUnder (· ≤ ·) l f
```

B.5.4 Mathlib/Topology/Order/LiminfLimsup.lean

```
/-- If a function has a limit, then its liminf coincides with its limit. -/
theorem Filter.Tendsto.liminf_eq {f : Filter β} {u : β → α} {a : α} [NeBot f]
    (h : Tendsto u f (𝓝 a)) : liminf u f = a
```

B.5.5 Mathlib/Topology/Order/Basic.lean

```
/-- **Squeeze theorem** (also known as **sandwich theorem**). This version assumes that
    inequalities
hold everywhere. -/
theorem tendsto_of_tendsto_of_tendsto_of_le_of_le [OrderTopology α] {f g h : β → α} {b : Filter β
    }
    {a : α} (hg : Tendsto g b (𝓝 a)) (hh : Tendsto h b (𝓝 a)) (hgf : g ≤ f) (hfh : f ≤ h) :
    Tendsto f b (𝓝 a)
```

B.5.6 Mathlib/Order/Filter/AtTopBot/Defs.lean

```
/-- `atTop` is the filter representing the limit `→ ∞` on an ordered set.
It is generated by the collection of up-sets `{b | a ≤ b}`.
(The preorder need not have a top element for this to be well defined,
and indeed is trivial when a top element `x` exists, i.e., it coincides with `pure x`.) -/
@[to_dual
/-- `atBot` is the filter representing the limit `→ -∞` on an ordered set.
It is generated by the collection of down-sets `{b | b ≤ a}`.
(The preorder need not have a bottom element for this to be well defined,
and indeed is trivial when a bottom element `x` exists, i.e., it coincides with `pure x`.) -/]
def atTop [Preorder α] : Filter α :=
  inf a, 𝒫 (Ici a)

@[to_dual eventually_le_atBot]
theorem eventually_ge_atTop [Preorder α] (a : α) : ∀ᶠ x in atTop, a ≤ x
```

B.5.7 Mathlib/Order/Filter/AtTopBot/Basic.lean

```
@[simp] lemma eventually_atTop : (∀ᶠ x in atTop, p x) ↔ ∃ a, ∀ b ≥ a, p b := mem_atTop_sets
```

B.5.8 Mathlib/Order/Filter/AtTopBot/Archimedean.lean

```
theorem tendsto_natCast_atTop_atTop [Semiring R] [PartialOrder R] [IsOrderedRing R]
    [Archimedean R] :
    Tendsto ((↑) : ℕ → R) atTop atTop
```

B.5.9 Mathlib/Order/Filter/AtTopBot/Field.lean

```
/-- If `r` is a positive constant, `fun x ↦ r * f x` tends to infinity along a filter
if and only if `f` tends to infinity along the same filter. -/
theorem tendsto_const_mul_atTop_of_pos (hr : 0 < r) :
    Tendsto (fun x => r * f x) l atTop ↔ Tendsto f l atTop
```

B.5.10 Mathlib/Topology/Neighborhoods.lean

```
theorem tendsto_const_nhds {f : Filter α} : Tendsto (fun _ : α => x) f (𝓝 x)
```

B.5.11 Mathlib/Topology/Order/DenselyOrdered.lean

```
@[simp]
theorem frontier_Icc [NoMinOrder α] [NoMaxOrder α] {a b : α} (h : a ≤ b) :
    frontier (Icc a b) = {a, b}
```

B.5.12 Mathlib/Order/Bounds/Defs.lean

```
/-- A set is bounded above if there exists an upper bound. -/
@[to_dual /-- A set is bounded below if there exists a lower bound. -/]
def BddAbove (s : Set α) :=
  (upperBounds s).Nonempty
```

B.5.13 Mathlib/Order/ConditionallyCompleteLattice/Defs.lean

```
/-- A conditionally complete lattice is a lattice in which
every nonempty subset which is bounded above has a supremum, and
every nonempty subset which is bounded below has an infimum.
Typical examples are real numbers or natural numbers.

To differentiate the statements from the corresponding statements in (unconditional)
complete lattices, we prefix ‘sInf‘ and ‘sSup‘ by a ‘c‘ everywhere. The same statements should
hold in both worlds, sometimes with additional assumptions of nonemptiness or
boundedness. -/
class ConditionallyCompleteLattice (α : Type*) extends Lattice α, SupSet α, InfSet α where
  /-- Every nonempty subset which is bounded above has a least upper bound. -/
  isLUB_csSup : ∀ s : Set α, s.Nonempty → BddAbove s → IsLUB s (sSup s)
  /-- Every nonempty subset which is bounded below has a greatest lower bound. -/
  isGLB_csInf : ∀ s : Set α, s.Nonempty → BddBelow s → IsGLB s (sInf s)
```

B.5.14 Mathlib/Order/ConditionallyCompleteLattice/Indexed.lean

```
/-- The indexed supremum of a function is bounded above by a uniform bound -/
theorem ciSup_le [Nonempty ι] {f : ι → α} {c : α} (H : ∀ x, f x ≤ c) : iSup f ≤ c

/-- The indexed supremum of a function is bounded below by the value taken at one point -/
theorem le_ciSup {f : ι → α} (H : BddAbove (range f)) (c : ι) : f c ≤ iSup f
```

B.6 Foundations

B.6.1 Mathlib/Logic/Pairwise.lean

```
/-- A relation ‘r‘ holds pairwise if ‘r i j‘ for all ‘i ≠ j‘. -/
def Pairwise (r : α → α → Prop) :=
  ∀ ⦃i j⦄, i ≠ j → r i j
```

B.7 Extended Reals

B.7.1 Mathlib/Data/EReal/Basic.lean

```
/-- The type of extended real numbers `[-∞, ∞]`, constructed as `WithBot (WithTop ℝ)`. -/
def EReal := WithBot (WithTop ℝ)

@[simp, norm_cast]
theorem coe_mul (x y : ℝ) : (↑(x * y) : EReal) = x * y

@[simp, norm_cast]
protected theorem coe_nonneg {x : ℝ} : (0 : EReal) ≤ x ↔ 0 ≤ x

@[simp]
theorem coe_ne_bot (x : ℝ) : (x : EReal) ≠ ⊥

@[simp]
theorem coe_ne_top (x : ℝ) : (x : EReal) ≠ ⊤
```

B.7.2 Mathlib/Data/EReal/Operations.lean

```
@[simp, norm_cast] theorem coe_neg (x : ℝ) : (↑(-x) : EReal) = -↑x

lemma mul_nonpos_iff {a b : EReal} : a * b ≤ 0 ↔ 0 ≤ a ∧ b ≤ 0 ∨ a ≤ 0 ∧ 0 ≤ b
```

B.7.3 Mathlib/Data/EReal/Inv.lean

```
lemma inv_nonneg_of_nonneg {a : EReal} (h : 0 ≤ a) : 0 ≤ a⁻¹
```

B.7.4 Mathlib/Topology/Instances/EReal/Lemmas.lean

```
protected theorem Tendsto.mul {f : Filter α} {ma : α → EReal} {mb : α → EReal} {a b : EReal}
    (hma : Tendsto ma f (𝓝 a)) (hmb : Tendsto mb f (𝓝 b)) (h₁ : a ≠ 0 ∨ b ≠ ⊥)
    (h₂ : a ≠ 0 ∨ b ≠ ⊤) (h₃ : a ≠ ⊥ ∨ b ≠ 0) (h₄ : a ≠ ⊤ ∨ b ≠ 0) :
    Tendsto (fun x ↦ ma x * mb x) f (𝓝 (a * b))
```

B.7.5 Mathlib/Data/ENNReal/Basic.lean

```
/-- The extended nonnegative real numbers. This is usually denoted [0, ∞],
  and is relevant as the codomain of a measure. -/
def ENNReal := WithTop ℝ≥0

/-- `toReal x` returns `x` if it is real, `0` otherwise. -/
protected def toReal (a : ℝ≥0∞) : Real := a.toNNReal

/-- `ofReal x` returns `x` if it is nonnegative, `0` otherwise. -/
protected def ofReal (r : Real) : ℝ≥0∞ := r.toNNReal

@[simp]
theorem ofReal_toReal_eq_iff : ENNReal.ofReal a.toReal = a ↔ a ≠ ⊤
```

B.7.6 Mathlib/Data/ENNReal/Real.lean

```
@[gcongr, bound]
theorem ofReal_le_ofReal {p q : ℝ} (h : p ≤ q) : ENNReal.ofReal p ≤ ENNReal.ofReal q

@[gcongr]
theorem toReal_mono (hb : b ≠ ∞) (h : a ≤ b) : a.toReal ≤ b.toReal
```

B.7.7 Mathlib/Topology/Instances/ENNReal/Lemmas.lean

```
theorem tendsto_toReal {a : ℝ≥0∞} (ha : a ≠ ∞) : Tendsto ENNReal.toReal (𝓝 a) (𝓝 a.toReal)

protected theorem Tendsto.sub {f : Filter α} {ma : α → ℝ≥0∞} {mb : α → ℝ≥0∞} {a b : ℝ≥0∞}
    (hma : Tendsto ma f (𝓝 a)) (hmb : Tendsto mb f (𝓝 b)) (h : a ≠ ∞ ∨ b ≠ ∞) :
    Tendsto (fun a => ma a - mb a) f (𝓝 (a - b))
```

B.7.8 Mathlib/Analysis/SpecialFunctions/Log/ENNRealLog.lean

```
/-- The logarithm function defined on the extended nonnegative reals `ℝ≥0∞`
to the extended reals `EReal`. Coincides with the usual logarithm function
and with `Real.log` on positive reals, and takes values `log 0 = ⊥` and `log ⊤ = ⊤`.
Conventions about multiplication in `ℝ≥0∞` and addition in `EReal` make the identity
`log (x * y) = log x + log y` unconditional. -/
noncomputable def log (x : ℝ≥0∞) : EReal :=
  if x = 0 then ⊥
    else if x = ⊤ then ⊤
    else Real.log x.toReal

lemma log_ofReal_of_pos {x : ℝ} (hx : 0 < x) : log (ENNReal.ofReal x) = Real.log x

@[gcongr] lemma log_le_log {x y : ℝ≥0∞} (h : x ≤ y) : log x ≤ log y

@[simp] lemma log_le_zero_iff {x : ℝ≥0∞} : log x ≤ 0 ↔ x ≤ 1

@[simp] lemma log_zero : log 0 = ⊥
@[simp] lemma log_one : log 1 = 0
```

B.7.9 Mathlib/Analysis/SpecialFunctions/Log/ENNRealLogExp.lean

```
@[continuity, fun_prop]
lemma continuous_log : Continuous log
```

B.8 Metric Spaces and Topology

B.8.1 Mathlib/Topology/MetricSpace/Pseudo/Defs.lean

```
theorem tendsto_nhds {f : Filter β} {u : β → α} {a : α} :
    Tendsto u f (𝓝 a) ↔ ∀ ε > 0, ∀ᶠ x in f, dist (u x) a < ε

theorem Real.dist_eq (x y : ℝ) : dist x y = |x - y|
```

References

[1] Harald Cramér. Sur un nouveau théorème-limite de la théorie des probabilités. *Actualités Scientifiques et Industrielles*, 736:2–23, 1938.

[2] Amir Dembo and Ofer Zeitouni. *Large Deviations Techniques and Applications*. Springer-Verlag, New York, 2nd edition, 1998.

[3] S. R. S. Varadhan. *Large Deviations and Applications*, volume 46 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, 1984.

[4] Richard S. Ellis. *Entropy, Large Deviations, and Statistical Mechanics*, volume 271 of *Grundlehren der mathematischen Wissenschaften*. Springer-Verlag, 1985.

[5] Frank den Hollander. *Large Deviations*, volume 14 of *Fields Institute Monographs*. American Mathematical Society, 2000.

[6] Roman Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, Cambridge, 2nd edition, 2026.

[7] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer International Publishing, 2021.

[8] The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pages 367–381, New York, NY, USA, 2020. Association for Computing Machinery.

[9] Johannes Hölzl and Armin Heller. Three chapters of measure theory in Isabelle/HOL. In Marko van Eekelen, Herman Geuvers, Julien Schmaltz, and Freek Wiedijk, editors, *Interactive Theorem Proving*, pages 135–151, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[10] Jeremy Avigad, Johannes Hölzl, and Luke Serafin. A formally verified proof of the Central Limit Theorem. *Journal of Automated Reasoning*, 59(4):389–423, 2017.

[11] Patrick Billingsley. *Probability and Measure*. Wiley Series in Probability and Mathematical Statistics. Wiley, New York, 3rd edition, 1995.

[12] Michikazu Hirata. A formalization of the Lévy-Prokhorov metric in Isabelle/HOL. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPIcs*, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[13] Reynald Affeldt and Cyril Cohen. Measure construction by extension in dependent type theory with application to integration. *Journal of Automated Reasoning*, 67(3):28, 2023.

[14] Reynald Affeldt, Alessandro Bruni, Cyril Cohen, Pierre Roux, and Takafumi Saikawa. Formalizing concentration inequalities in Rocq: Infrastructure and automation. In *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *LIPIcs*, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[15] Kexing Ying and Rémy Degenne. A formalization of Doob's martingale convergence theorems in mathlib. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2023, pages 334–347, New York, NY, USA, 2023. Association for Computing Machinery.

[16] Rémy Degenne. Markov kernels in Mathlib's probability library. *arXiv preprint arXiv:2510.04070*, 2025.

[17] Sho Sonoda, Kazumi Kasaura, Yuma Mizuno, Kei Tsukamoto, and Naoto Onda. Lean formalization of generalization error bound by Rademacher complexity and Dudley's entropy integral. *arXiv preprint arXiv:2503.19605*, 2025.

[18] N. Etemadi. An elementary proof of the Strong Law of Large Numbers. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 55(1):119–122, 1981.

[19] Étienne Marion, Thomas Zhu, and Rémy Degenne. Central Limit Theorem in Mathlib. Mathlib pull request #36208, 2026. Merged March 28, 2026.

[20] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean Copilot: Large language models as copilots for theorem proving in Lean. In *Proceedings of the International Conference on Neuro-symbolic Systems (NeuS 2025)*, volume 288 of *PMLR*, 2025. arXiv:2404.12534.

[21] Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, et al. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.